

ROS 2-Robotik-Handbuch

Theoretische Grundlagen, Hardware, Simulation und vier Projekte

Stepper-Robotik mit Zykloidgetriebe · Stereokamera · Kalman-Filter · ROS 2

Arbeitsdokument

Stand: Juni 2026

ROS 2 Jazzy Jalisco / Gazebo Harmonic

Über dieses Dokument

Wie dieses Dokument zu lesen ist

Das Handbuch ist in sechs Teile gegliedert. **Teil I** legt die Theorie fest (ROS2, Kalman/EKF, Odometrie, Stereosehen, Kinematik, Dynamik, Trajektorien, Schrittmotoren, Zykloidgetriebe). **Teil II** behandelt die Hardware-Auswahl inklusive Strom-/Spannungsmessung zur Drehmomentschätzung und PCB-Optimierung. **Teil III** ist die Schritt-für-Schritt-Installationsanleitung (VM, Linux, ROS2). **Teil IV** erklärt die Simulation, den CAD-zu-URDF-Workflow und die entscheidende Frage, wo sich Simulation und reale Implementierung trennen. **Teil V** arbeitet die vier Projekte mit Knoten-Architektur und `rqt_graph` durch. **Teil VI** ist die praktische Stereokamera-Integration mit Codebeispielen. Den Abschluss bildet eine empfohlene Dokumentationsmethodik.

Inhaltsverzeichnis

Über dieses Dokument	i
I Theoretische Grundlagen	1
1 ROS 2 – Konzepte und Architektur	2
1.1 Warum ROS 2 und nicht ROS 1	2
1.2 Die fünf Kommunikationsprimitive	2
1.3 QoS – Quality of Service	3
1.4 Workspace, Pakete, Build	3
1.5 TF2 – der Transformationsbaum	3
1.6 Wichtige CLI-Werkzeuge	3
2 Zustandsschätzung: Kalman-Filter und EKF	5
2.1 Das Grundproblem	5
2.2 Lineares Zustandsraummodell	5
2.3 Die zwei Phasen	5
2.4 Extended Kalman Filter (EKF)	6
2.5 Anwendung Tischtennisball (konstante Beschleunigung)	6
3 Odometrie und Sensorfusion	7
3.1 Radodometrie: Pose-Integration aus Encoder-Ticks	7
3.2 Visuelle Odometrie (VO)	8
3.3 Visual-Inertial Odometry (VIO)	9
4 Stereosehen und Merkmalerkennung	11
4.1 Lochkameramodell und Tiefe aus Disparität	11
4.2 Kalibrierung	11
4.3 Merkmalerkennung ohne eigenes KI-Training	11
4.4 Posenschätzung – 6D-Lage eines Bauteils	12
5 6-DoF-Posenschätzung mit gelernten Surface Embeddings	13
5.1 Das Problem: sechs Freiheitsgrade	13
5.2 Rotationsrepräsentationen	13
5.3 Warum gelernte Embeddings statt Regression	14
5.4 Kontrastives Lernen der Embeddings	14
5.5 Das Two-Tower-Modell (Query-Key)	14
5.6 Von Korrespondenzen zur Pose: PnP	15
5.7 Robuste Schätzung: RANSAC	15
5.8 Registrierung: grob und fein	16
5.9 Punktwolken-Netze: PointNet und PointNet++	17
5.10 Stereo, Tiefe und Skalierung	19

5.11	Zeitliche Konsistenz: Kalman-Pose-Tracking	19
5.12	Sim-to-Real und Trainingsdaten	19
5.13	Die Pipeline: von CAD zu Trainingsdaten	19
5.14	Werkzeuge: Renderer im Vergleich	20
5.15	Benchmarking und Metriken	21
6	Kinematik, Dynamik und Trajektorien	22
6.1	Vorwärtskinematik	22
6.2	Inverse Kinematik	22
6.3	Dynamik	22
6.4	Trajektorienplanung	22
7	Regelungstechnik der Robotergrundlagen	23
7.1	Feedback: die Grundlage	23
7.2	Vorsteuerung (Feedforward)	24
7.3	Kaskadenregelung	24
7.4	Feedback-Entkopplung (Computed Torque)	24
7.5	Regelung im Arbeitsraum (Operational Space Control)	25
7.6	Zustandsbeobachter	25
7.7	Störgrößenkompensation	26
7.7.1	Statische vs. dynamische Störgrößenausschaltung	26
7.7.2	Störentkopplung (Disturbance Decoupling)	26
7.7.3	Dynamische Vorsteuerung – mit oder ohne Zustandsregelung	26
7.8	Überblick: Wann was?	27
8	Schrittmotoren und Drehmomentschätzung	28
8.1	NEMA-17-Grundlagen	28
8.2	Indirekte Drehmomentschätzung	28
9	Zykloidgetriebe	29
10	Maschinelles Lernen in ROS 2-Projekten	30
10.1	Wann ML – und wann nicht	30
10.2	Architektur: Training offline, Inferenz im Knoten	31
10.3	Die vier Bibliotheken und ihre Rolle	32
10.4	Klassisches ML mit SciPy & scikit-learn	33
10.4.1	Regression – Kennlinien und Vorhersage	33
10.4.2	Clustering – Punktwolken und Objekte gruppieren	33
10.4.3	Bayes & Gauß-Prozesse – Schätzen mit Unsicherheit	34
10.5	Neuronale Netze mit PyTorch	34
10.5.1	ANN (MLP) – gelernte inverse Kinematik	34
10.5.2	CNN – Objekt- und Ball-Erkennung	35
10.5.3	Autoencoder – Anomalie- und Zustandsüberwachung	35
10.5.4	PINN – physik-informierte Netze	36
10.6	Verfahren-Wegweiser	36
II	Stereokamera in ROS 2 – Praxis mit Code	38
11	Kamera anbinden, Frame splitten	39
12	Kalibrieren und Tiefe erzeugen	40
12.1	Warum kalibrieren? Das Kameramodell	40
12.2	Kalibrierung und Tiefenpipeline in ROS 2	40

13 Kalman-Filter-Knoten (Ballverfolgung)	42
14 Merkmalerkennung ohne KI-Training	43
14.1 Detektoren im Vergleich: SIFT, SURF, KAZE, AKAZE, ORB, BRISK	43
15 Echtzeit-Stereoverarbeitung: Scheduling, Frame-Dropping und Zustandsauto-	46
mat	
15.1 Das Zeitbudget eines Frames	46
15.2 Frame-Dropping: DDS macht die Arbeit	47
15.3 Scheduling: Executor und Callback-Gruppen	47
15.4 Ablauflogik als Zustandsautomat	48
 III Hardware-Auswahl	 50
16 Komponenten im günstigen Hobby-Bereich	51
16.1 Schrittmotor und Treiber	51
16.2 Recheneinheiten	51
16.3 Stereokamera	52
17 Optimierungspotenzial für ein eigenes PCB	53
 IV Software-Setup: VM, Linux, ROS 2	 54
18 Betriebssystem und Distribution	55
18.1 Distributionswahl (Stand Juni 2026)	55
18.2 VM, Dual-Boot oder WSL2?	55
19 ROS 2 installieren – Schritt für Schritt	56
19.1 Pakete, die du für die Projekte brauchst	56
19.2 Die Umgebung dauerhaft verfügbar machen (<code>source</code>)	57
19.2.1 Underlay und Overlay	57
19.2.2 Automatisch bei jedem Terminal – via <code>~/.bashrc</code>	57
19.2.3 Was sich <i>nicht</i> automatisch sourcen lässt (und warum)	58
 V Simulation und der Weg vom CAD-Modell zur Realität	 59
20 Simulationswerkzeuge im ROS 2-Ökosystem	60
20.1 Was lässt sich vorab simulieren?	60
20.2 MuJoCo und Isaac Sim im Detail	61
20.2.1 MuJoCo – Multi-Joint dynamics with Contact	61
20.2.2 NVIDIA Isaac Sim – photorealistisch und GPU-parallel	61
20.3 Lassen sie sich integriert einsetzen?	62
20.4 Praktische Einrichtung auf dem Dual-Boot-System	62
20.4.1 Schritt 0 – System aktualisieren und Basiswerkzeuge	63
20.4.2 Schritt 1 – ROS 2 Jazzy installieren	63
20.4.3 Schritt 2 – Simulations- und Projektpakete (inkl. Gazebo Harmonic)	63
20.4.4 Schritt 3 – colcon-Workspace anlegen	64
20.4.5 Schritt 4 – MuJoCo in einer Python-Umgebung	64
20.4.6 Schritt 5 – NVIDIA Isaac Sim (optional)	64
20.4.7 Schritt 6 – VSCode-Erweiterungen	65
20.4.8 Schritt 7 – micro-ROS-Agent für den ESP32	65

20.4.9	Schritt 8 – GPU-Rendering (EGL) für Gazebo-Kamerasensoren	65
21	Wo sich Simulation und reale Implementierung trennen	67
21.1	Der vollständige Weg: von der frischen Ubuntu-Installation zum Roboter	67
22	Vorabschritte: vom CAD-Modell zur Simulation	69
22.1	Getrennte STL-Exporte pro Gelenk	69
22.2	Gelenktypen	70
22.3	Getriebeübersetzung in <code>ros2_control</code>	70
VI	Einstiegsprojekte: Erste Schritte mit ROS 2, RViz und Gazebo	72
23	Projekt 0.A – Planarer 2-Gelenk-Arm: Nodes, Topics, Services, Actions, IK	73
23.1	ROS 2-Bausteine	73
23.2	<code>rqt_graph</code> (Zielbild)	74
23.3	Schritt für Schritt	75
23.3.1	Schritt 1 – Pakete anlegen und die Struktur verstehen	75
23.3.2	Schritt 2 – Geometrie als URDF/Xacro	76
23.3.3	Schritt 3 – In RViz darstellen (über eine Launch-Datei)	78
23.3.4	Schritt 4 – In Gazebo simulieren (<code>ros2_control</code>)	80
23.3.5	Schritt 5 – Topics: kommandieren und auslesen	82
23.3.6	Schritt 6 – Service: in eine benannte Pose fahren	84
23.3.7	Schritt 7 – Inverse Kinematik allgemein: Denavit-Hartenberg + Jacobi	87
23.3.8	Schritt 8 – Action: kartesisches Ziel anfahren	89
23.4	Implementierungs-Referenz: <code>rcpy</code> und <code>Launch</code>	93
23.4.1	Das Knotengerüst	93
23.4.2	<code>create_publisher</code> – senden	93
23.4.3	<code>create_subscription</code> – empfangen	93
23.4.4	<code>create_timer</code> – periodisch handeln	94
23.4.5	<code>create_service</code> – auf Anfragen antworten	94
23.4.6	Interfaces definieren – <code>srv</code> und <code>action</code>	94
23.4.7	<code>ActionServer</code> – langlaufende Aufträge umsetzen	94
23.4.8	<code>ReentrantCallbackGroup</code> + <code>MultiThreadedExecutor</code>	95
23.4.9	Aufbau einer Launch-Datei	95
23.5	Häufige Fehler und Checkliste (Projekt 0.A)	96
23.6	Was du danach beherrscht	97
24	Projekt 0.B – Stereokamera, Objekterkennung und Kalman-Tracking	98
24.1	ROS 2-Bausteine	98
24.2	<code>rqt_graph</code> (Zielbild)	98
24.3	Schritt für Schritt	99
24.3.1	Schritt 0 – Paket anlegen (ein Workspace für alle Projekte)	99
24.3.2	Schritt 1 – Welt: Ball und Stereokamera	99
24.3.3	Schritt 2 – Bilder und Zeit nach ROS bruecken	101
24.3.4	Schritt 3 – Objekterkennung (OpenCV, HSV)	101
24.3.5	Schritt 4 – Triangulation und der vollständige Detektor	101
24.3.6	Schritt 5 – Kalman-Tracker mit Vorhersage	102
24.3.7	Schritt 6 – Marker in RViz und alles starten	103
24.4	Implementierungs-Referenz: Parameter und Synchronisation	104
24.4.1	Parameter – <code>declare_parameter</code> / <code>get_parameter</code>	105
24.4.2	Zwei Topics zeitlich koppeln – <code>message_filters</code>	105
24.5	Sichtfeld visualisieren und Sim-to-Real	106

24.5.1	Wie das Sichtfeld definiert ist	106
24.5.2	Das Frustum als Pyramide zeigen (fov_viz)	106
24.6	Häufige Fehler und Checkliste (Projekt 0.B)	108
24.7	Was du danach beherrscht	108
VII	Die vier Projekte: Architektur und rqt_graph	109
25	Projekt 1 – Stepper-Roboter mit Trajektorienbefehl vom PC	110
25.1	ROS 2-Bausteine	110
25.2	rqt_graph (vereinfacht)	110
26	Projekt 2 – Tischtennisroboter	111
26.1	Vorüberlegungen	111
26.1.1	ROS 2-Bausteine	111
26.1.2	Architektur und Wahrnehmungskette	111
26.1.3	Kamera-Architektur – wo wird gerechnet?	112
26.1.3.1	Die Bandbreitenrechnung entscheidet	112
26.1.3.2	Verdikt pro Plattform	112
26.1.3.3	Sparse statt dense: der Ball braucht keine volle Tiefenkarte	113
26.1.3.4	ROS-Service, UDP oder Topic?	113
26.1.4	Mechanischer Aufbau: Portalsystem (Gantry)	113
26.1.4.1	Grundkonzept	113
26.1.4.2	Geschachtelte Achsen	113
26.1.4.3	Antriebstopologie und Hardware-Anschluss	113
26.1.5	Das Hebelarm- und Schwingungsproblem	114
26.1.5.1	Die Physik	114
26.1.5.2	Gegenmaßnahmen	114
26.1.6	Simulierbarkeit in Gazebo und RViz	115
26.1.6.1	Was Gazebo kann – und was nicht	115
26.1.6.2	Wege, die Nachgiebigkeit doch abzubilden	115
26.1.7	CAD-Module und URDF	115
26.1.7.1	Notwendige CAD-Module mit Designanforderungen	116
26.1.7.2	Kinematische Kette	116
26.1.7.3	URDF/Xacro-Gerüst	117
26.1.8	Ablauf- und Parallellogik	117
26.2	Vorgehensweise	118
26.2.1	Phase 1 – Kamera und Balltracking in Simulation	118
26.2.1.1	Tischtennisplatte als CAD/SDF-Modell	118
26.2.1.2	Gegnerischen Ballschlag parametrisch simulieren	118
26.2.1.3	Stereokamera-Tracking des Balls	118
26.2.1.4	Auftreffpunkt über schiefen Wurf und EKF	118
26.2.1.5	Zielebene, Plattenrand und Fehlschuss-Erkennung	119
26.2.1.6	Ausblick: Luftwiderstand	120
26.2.1.7	Ausblick: Ballspin (Magnus)	121
26.2.2	Phase 2 – Roboteranbindung	121
26.2.2.1	Frühe Abfangbewegung (iterative Verfeinerung)	121
26.2.2.2	Stellgrößen und Motorsteuerung	121
26.2.2.3	Regelstrategie: entkoppelte Achsregelung statt MIMO	121
26.2.2.4	Schläger-Reichweite: Position grob, Schlag fein	121
26.2.2.5	Inverse Kinematik bei laufendem EKF-Update	122
26.2.2.6	Auftreffzeitpunkt, Timing und Balltypen	122
26.3	Umsetzung	122

26.3.1	Schritt 1 – Tischtennisplatte als STL in Gazebo einbinden	122
26.3.2	Schritt 2 – Stereo-Rig und Ball in die Welt integrieren	124
26.3.3	Schritt 3 – Balltracking: fliegender Ball	126
26.3.4	Schritt 4 – Flugschätzung mit EKF und Luftwiderstand	127
26.3.5	Schritt 5 – Aufschlagregeln, Fehlschuss und Begrenzungswand	129
26.3.6	Genauigkeit der Flugschätzung: Messrauschen, Kamerastandort, FPS . . .	133
26.3.7	Schritt 6 – Auftreffzeitpunkt und Ball-Klassifikation	136
26.3.8	Schritt 7 – Roboteranbindung	137
27	Projekt 3 – Mobiler Roboter: Odometrie, Navigation, Bewegungserkennung	138
27.1	ROS 2-Bausteine	138
28	Projekt 4 – CAD-basierte Posenerkennung und Greifen	139
28.1	ROS 2-Bausteine	139
28.2	Zwei Wege zur 6D-Pose	139
28.3	Inferenz-Pipeline (lernbasiert)	140
28.4	Stereo als metrische Skala	140
28.5	Bewegte Teile: Pose-Tracking mit Kalman	140
28.6	Hyperparameter und Evaluation	140
VIII	Dokumentationsmethodik	142
29	Vorgehensweise festhalten	143
	Literaturverzeichnis	144

Teil I

Theoretische Grundlagen

Kapitel 1

ROS 2 – Konzepte und Architektur

1.1 Warum ROS 2 und nicht ROS 1

ROS 2 ersetzt den zentralen `rosmaster` von ROS 1 durch eine dezentrale Discovery über **DDS** (Data Distribution Service), einen Industriestandard aus Luft- und Raumfahrt. Daraus folgen die für deine Projekte relevanten Eigenschaften: kein Single-Point-of-Failure, Multi-Maschinen-Betrieb über das Netzwerk ohne Master, konfigurierbare Echtzeit-/Zuverlässigkeitsgarantien (QoS) und ein einheitliches API in C++ (`rclcpp`) und Python (`rclpy`). ROS 1 (Noetic) ist seit Mai 2025 End-of-Life; alle Neuprojekte gehören auf ROS 2.

1.2 Die fünf Kommunikationsprimitive

Das gesamte Verhalten eines ROS 2-Systems lässt sich auf fünf Bausteine zurückführen. Sie zu verstehen ist die halbe Miete – der Rest ist Anwendung.

Node (Knoten)

Eine Recheneinheit mit genau einer Verantwortung – z. B. `ball_detector`, `trajectory_controller`. Knoten kommunizieren ausschließlich über die vier folgenden Mechanismen, nie direkt.

Topic (Thema)

Asynchroner Publish/Subscribe-Kanal, many-to-many. Ein Publisher sendet *Nachrichten* (eines `.msg`-Typs), beliebig viele Subscriber empfangen sie. Geeignet für kontinuierliche Datenströme: Bilder, Gelenkzustände, Odometrie. *Feuern und vergessen* – der Sender weiß nicht, wer zuhört.

Service (Dienst)

Synchroner Request/Response, one-to-one. Ein Client schickt eine Anfrage, ein Server antwortet genau einmal. Für kurze, abgeschlossene Aktionen: “Schalte Controller um”, “Gib aktuelle Kalibrierung zurück”. Blockierend.

Action (Aktion)

Für lange, zielgerichtete Aufgaben mit Zwischen-Feedback und Abbruchmöglichkeit. Technisch aus Topics + Services zusammengesetzt: *Goal* (Ziel senden), *Feedback* (Fortschritt), *Result* (Endergebnis). Der Paradefall für “fahre diese Trajektorie ab” – du willst Fortschritt sehen und abbrechen können.

Parameter

Pro Knoten gehaltene Konfigurationswerte (Gelenklimits, PID-Gains, Getriebeübersetzung), zur Laufzeit les- und setzbar, deklariert beim Knotenstart.

Faustregel zur Wahl

Kontinuierlicher Datenstrom → **Topic**. Kurze Frage mit sofortiger Antwort → **Service**. Langer Auftrag mit Fortschritt/Abbruch (Trajektorie, Navigation, Greifen) → **Action**. Statische Konfiguration → **Parameter**.

1.3 QoS – Quality of Service

Weil DDS darunterliegt, hat jedes Topic eine QoS-Konfiguration. Die wichtigsten Profile:

- **Reliability:** RELIABLE (jede Nachricht garantiert, Wiederübertragung) vs. BEST_EFFORT (verlustbehaftet, aber latenzarm). Sensorbilder fahren oft BEST_EFFORT, Steuerbefehle RELIABLE.
- **Durability:** TRANSIENT_LOCAL hält die letzte Nachricht für später startende Subscriber vor (typisch für /map, statische TF).
- **History/Depth:** Puffergröße.

Eine häufige Fehlerquelle: Publisher und Subscriber mit *inkompatiblen* QoS-Profilen “sehen” sich nicht. Bei stummen Topics ist das der erste Verdacht.

1.4 Workspace, Pakete, Build

ROS 2-Code lebt in einem **Workspace** (Verzeichnis mit `src/`). Darin liegen **Pakete**. Gebaut wird mit **colcon**.

```
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws
# ... Pakete nach src/ ...
rosdep install --from-paths src --ignore-src -y      # Abhaengigkeiten
colcon build --symlink-install                      # bauen
source install/setup.bash                          # Overlay aktivieren
```

Listing 1.1: Workspace anlegen und bauen

Python-Pakete nutzen `ament_python` (`setup.py`), C++-Pakete `ament_cmake` (`CMakeLists.txt`). Beide haben eine `package.xml` mit Metadaten und Abhängigkeiten.

1.5 TF2 – der Transformationsbaum

Jeder Roboter ist ein Baum von Koordinatensystemen (Frames): `map` → `odom` → `base_link` → `camera_link` → **TF2** verwaltet diese zeitgestempelten Transformationen, sodass du jederzeit fragen kannst “Wo ist der Ball im `base_link`-Frame?”, obwohl er in `camera_link` gemessen wurde. Rotationen werden hier als **Quaternionen** gespeichert (`geometry_msgs/Quaternion`) – genau aus den Gründen, die wir besprochen haben: kein Gimbal Lock, kompakt, sauber interpolierbar. Konventionen sind in REP-103 (Einheiten, rechtshändig, *x* vorne, *z* oben) und REP-105 (Frame-Namen) festgelegt.

1.6 Wichtige CLI-Werkzeuge

Befehl	Zweck
<code>ros2 run <pkg> <node></code>	Einen Knoten starten
<code>ros2 launch <pkg> <file></code>	Mehrere Knoten + Konfiguration starten
<code>ros2 topic list / echo / hz</code>	Topics auflisten, Inhalt sehen, Rate messen
<code>ros2 node list / info</code>	Knoten und ihre Schnittstellen

<code>ros2 service call</code>	Dienst manuell aufrufen
<code>ros2 action send_goal</code>	Aktion manuell starten
<code>ros2 param get / set / dump</code>	Parameter lesen/schreiben/sichern
<code>rqt_graph</code>	Knoten-Topic-Graph visualisieren
<code>rviz2</code>	3D-Visualisierung (<i>kein</i> Simulator!)
<code>ros2 bag record / play</code>	Datenströme aufzeichnen/abspielen

Häufige Begriffsverwechslung

RViz2 ist ein *Visualisierer* (zeigt Daten an), **Gazebo** ist ein *Simulator* (erzeugt Daten per Physik). Du wirst beide gleichzeitig nutzen: Gazebo simuliert, RViz2 zeigt an.

Kapitel 2

Zustandsschätzung: Kalman-Filter und EKF

2.1 Das Grundproblem

Du willst eine verborgene Größe (Position und Geschwindigkeit des Tischtennisballs, Pose des Roboters) aus verrauschten, unvollständigen Messungen schätzen – und das, während sich der Zustand zeitlich ändert. Der Kalman-Filter ist der optimale lineare Schätzer für genau dieses Problem unter gaußischem Rauschen. Er hält nicht nur einen Schätzwert $\hat{\mathbf{x}}$, sondern auch dessen **Unsicherheit** als Kovarianzmatrix $\mathbf{P}$.

2.2 Lineares Zustandsraummodell

$$\mathbf{x}_k = \mathbf{F} \mathbf{x}_{k-1} + \mathbf{B} \mathbf{u}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}) \quad (\text{Prozessrauschen}) \quad (2.1)$$

$$\mathbf{z}_k = \mathbf{H} \mathbf{x}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R}) \quad (\text{Messrauschen}) \quad (2.2)$$

$\mathbf{F}$ = Systemmatrix (wie entwickelt sich der Zustand), $\mathbf{B}$ = Steuermatrix, $\mathbf{H}$ = Messmatrix (was misst der Sensor vom Zustand), $\mathbf{Q}/\mathbf{R}$ = Rausch-Kovarianzen (deine wichtigsten Tuning-Größen).

2.3 Die zwei Phasen

Der Filter alterniert zwischen **Prädiktion** (Modell rechnet voraus, Unsicherheit wächst) und **Korrektur** (Messung zieht zurück, Unsicherheit sinkt).

Prädiktion (Zeitschritt)

$$\begin{aligned} \hat{\mathbf{x}}_k^- &= \mathbf{F} \hat{\mathbf{x}}_{k-1} + \mathbf{B} \mathbf{u}_k \\ \mathbf{P}_k^- &= \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q} \end{aligned}$$

Korrektur (Messupdate)

$$\begin{aligned} \mathbf{K}_k &= \mathbf{P}_k^- \mathbf{H}^\top (\mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R})^{-1} && (\text{Kalman-Gain}) \\ \hat{\mathbf{x}}_k &= \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^-) && (\text{Innovation gewichtet}) \\ \mathbf{P}_k &= (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- \end{aligned}$$

Der Gain $\mathbf{K}$ ist die Gewichtung: Ist die Messung präzise ($\mathbf{R}$ klein), vertraut der Filter ihr; ist das Modell präzise ($\mathbf{Q}$ klein), glättet er stärker.

2.4 Extended Kalman Filter (EKF)

Sobald System oder Messung **nichtlinear** sind – und das sind sie in der Robotik fast immer (Rotationen, Projekttilflug mit Luftwiderstand, Kameraprojektion) –, ersetzt man die Matrizen durch nichtlineare Funktionen $\mathbf{f}(\cdot)$ und $\mathbf{h}(\cdot)$ und linearisiert sie pro Schritt über die **Jacobi-Matrizen**:

$$\mathbf{F}_k = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}}, \quad \mathbf{H}_k = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-} \quad (2.3)$$

Prädiktion nutzt dann $\hat{\mathbf{x}}_k^- = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$, die Kovarianz-Updates verwenden $\mathbf{F}_k, \mathbf{H}_k$. Der **UKF** (Unscented) vermeidet die Jacobi-Matrizen, indem er “Sigma-Punkte” durch die Nichtlinearität propagiert – genauer bei starker Krümmung, aber rechenintensiver. Für Lageschätzung mit Quaternionen nimmt man den **Error-State/Multiplicative EKF** (Fehlerzustand als kleiner 3D-Rotationsvektor im Tangentialraum, multiplikativ aufs Quaternion angewandt).

2.5 Anwendung Tischtennisball (konstante Beschleunigung)

Zustand $\mathbf{x} = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top$, Schwerkraft als bekannte Steuerung. Mit Zeitschritt Δt :

$$\mathbf{F} = \begin{pmatrix} \mathbf{I}_3 & \Delta t \mathbf{I}_3 \\ \mathbf{0} & \mathbf{I}_3 \end{pmatrix}, \quad \mathbf{B}\mathbf{u} = \begin{pmatrix} \frac{1}{2}\Delta t^2 \mathbf{g} \\ \Delta t \mathbf{g} \end{pmatrix}, \quad \mathbf{g} = \begin{pmatrix} 0 \\ 0 \\ -9.81 \end{pmatrix}, \quad \mathbf{H} = \begin{pmatrix} \mathbf{I}_3 & \mathbf{0} \end{pmatrix} \quad (2.4)$$

Die Stereokamera liefert nur die Position (daher $\mathbf{H}$ projiziert auf die ersten drei Komponenten); der Filter schätzt die Geschwindigkeit mit und extrapoliert per Prädiktion (ohne Messupdate) den Flug bis zur Schlagebene – das ist die Vorhersage des Abschlagzeitpunkts. Luftwiderstand und Magnus-Effekt (Spin) machen das Modell nichtlinear $\rightarrow$ EKF, falls nötig.

Kapitel 3

Odometrie und Sensorfusion

Odometrie schätzt die Eigenbewegung aus inkrementellen Sensordaten.

- **Radodometrie:** Aus Encoder-Ticks und Differentialantriebs-Kinematik die Pose integrieren. Einfach, aber driftet (Schlupf, Rundungsfehler akkumulieren).
- **Visuelle Odometrie (VO):** Bewegung aus der Verfolgung von Bildmerkmalen über aufeinanderfolgende Frames schätzen. Mit Stereokamera bekommst du den Maßstab gratis (Tiefe ist bekannt).
- **Visual-Inertial Odometry (VIO):** VO + IMU per EKF fusioniert – robuster, glatter.

In ROS2 ist `robot_localization` das Standardpaket: ein `ekf_node/ukf_node` fusioniert mehrere Quellen (Rad-Odom + IMU + visuelle Odom) zu einer konsistenten `odom→base_link`-Transformation. Drift wird später durch SLAM/Loop-Closure korrigiert (`map→odom`).

3.1 Radodometrie: Pose-Integration aus Encoder-Ticks

Die Radodometrie ist die billigste und am weitesten verbreitete Quelle für die Eigenbewegung: zwei Drehgeber (Encoder) zählen, wie weit sich das linke und das rechte Rad gedreht haben, und daraus rekonstruiert man die Bewegung des Fahrzeugmittelpunkts. Der Grundgedanke ist eine **Bogen-Approximation**: Zwischen zwei Abtastschritten fährt jedes Rad ein kurzes Kreisbogenstück um ein gemeinsames *Momentanzentrum* C (Instantaneous Center of Rotation, ICR).

Aus den Encoder-Ticks erhält man die zurückgelegten Bogenlängen Δd_l (links) und Δd_r (rechts). Bei der Radspurbreite b (Abstand der Räder) ergeben sich die mittlere Strecke und die Drehung zu:

$$\Delta d_c = \frac{\Delta d_r + \Delta d_l}{2}, \quad \Delta \Psi = \frac{\Delta d_r - \Delta d_l}{b} \quad (3.1)$$

Die neue Pose folgt durch Integration entlang des Bogens (Mittelpunktsregel):

$$\begin{aligned} X_f &= X_i + \Delta d_c \cos\left(\Psi_i + \frac{\Delta \Psi}{2}\right), \\ Y_f &= Y_i + \Delta d_c \sin\left(\Psi_i + \frac{\Delta \Psi}{2}\right), \\ \Psi_f &= \Psi_i + \Delta \Psi. \end{aligned} \quad (3.2)$$

Warum driftet das? Jeder Schritt addiert seinen Fehler zum Zustand – die Schätzung ist *rein integrativ*, ohne absoluten Bezug. Radschlupf, ungenaue Radradien, eine falsch gemessene Spurweite b und die endliche Encoder-Auflösung akkumulieren sich. Besonders teuer sind **Drehfehler**: ein kleiner Fehler in $\Delta \Psi$ wird über die folgende Fahrstrecke zu einem großen Positionsfehler gehandelt. In GNSS-freien Umgebungen wird die Radodometrie deshalb mit visuell-inertialen Messungen per EKF fusioniert [1] – die Radodometrie liefert den *metrischen Maßstab*, die anderen Sensoren korrigieren den Drift.

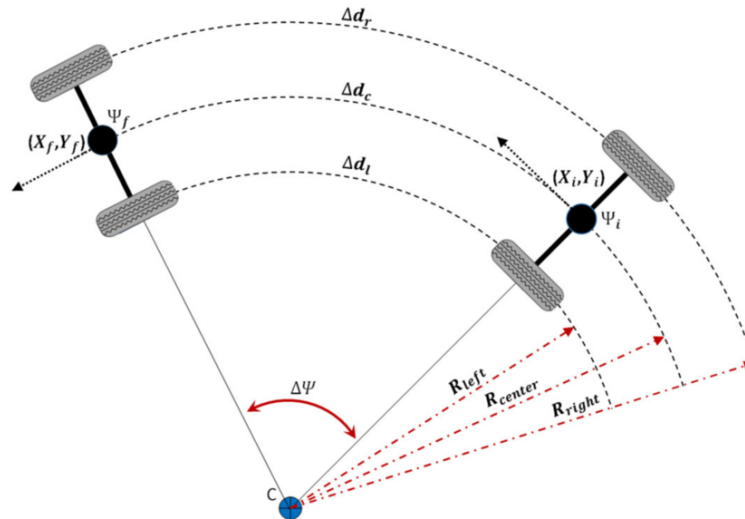


Abbildung 3.1: Geometrie der Radodometrie: Zwischen Anfangspose (X_i, Y_i, Ψ_i) und Endpose (X_f, Y_f, Ψ_f) legen links, mittleres und rechtes Rad die Bogenstücke Δd_l , Δd_c , Δd_r um das Momentanzentrum C zurück. Die unterschiedlichen Radradien R_{left} , R_{center} , R_{right} erzeugen die Kursänderung $\Delta \Psi$. Nach [1].

3.2 Visuelle Odometrie (VO)

Visuelle Odometrie schätzt die inkrementelle Kamerabewegung aus dem optischen Strom markanter Bildmerkmale. Sie hat keinen mechanischen Schlupf und driftet daher anders (und oft langsamer) als die Radodometrie, ist dafür aber empfindlich gegen Textur-Armut, Bewegungsunschärfe und schlechte Beleuchtung [2].

3D – 2D motion estimation

- **Given:**
 - 3D world coordinates of features in frame $k-1$
 - 2D image coordinates in frame k
- **Camera Projection:**

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = K[R|t] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$
- **K is known** from calibration
- Estimate **$[R|t]$**

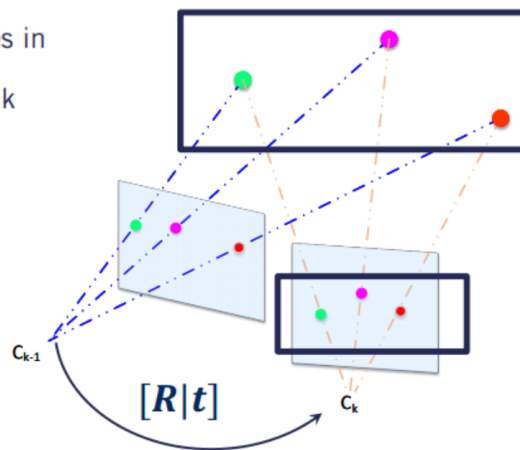


Abbildung 3.2: 3D-2D-Bewegungsschätzung in der visuellen Odometrie: Gegeben sind die 3D-Weltkoordinaten der Merkmale aus Frame $k-1$ und ihre 2D-Bildkoordinaten in Frame k . Über die Kameraprojektion $s\mathbf{u} = \mathbf{K}[\mathbf{R}|\mathbf{t}]\mathbf{X}$ mit der aus der Kalibrierung bekannten Intrinsik $\mathbf{K}$ wird die relative Pose $[\mathbf{R}|\mathbf{t}]$ zwischen den Kamerazentren C_{k-1} und C_k geschätzt. Nach [3].

Die typische **VO-Pipeline** pro Frame:

1. **Merkmale erkennen** (ORB, SIFT, FAST – vgl. Kapitel 4) und über aufeinanderfolgende Frames **verfolgen** (Matching oder optischer Fluss).
2. **Ausreißer filtern** (Lowe-Ratio-Test, RANSAC) – bewegte Objekte und Fehlzuordnungen müssen

raus.

3. **Relative Pose schätzen.** Mit *bekannter* Tiefe (Stereo oder RGB-D) wird das zum **3D-2D-Problem** und über PnP+RANSAC gelöst (Abb. 3.2). Bei Mono-VO liefert die Essential-Matrix nur eine Pose *bis auf Skalierung*.
4. **Pose akkumulieren** und optional über eine lokale Bündelausgleichung (Bundle Adjustment) verfeinern.

Der Stereo-Vorteil: Da die Tiefe pro Merkmal aus der Disparität bekannt ist (vgl. Stereo-Kapitel 4), gibt es keine Skalenmehrdeutigkeit – die geschätzte Translation ist sofort metrisch. Die verbleibende Schwäche ist der *akkumulierte Drift*, gegen den erst ein Loop-Closure (SLAM) oder die Fusion mit weiteren Sensoren hilft [2].

3.3 Visual-Inertial Odometry (VIO)

VIO fusioniert die visuelle Odometrie mit einer **IMU** (Beschleunigungsmesser + Gyroskop). Die beiden Sensoren ergänzen sich komplementär: Die Kamera liefert genaue, aber langsame und (bei Mono) skalenmehrdeutige Posen; die IMU liefert hochfrequente Bewegungsinformation und löst über die gemessene Erdbeschleunigung die **metrische Skala** sowie die absolute Neigung (Roll/Pitch), driftet aber bei reiner Integration schnell weg.

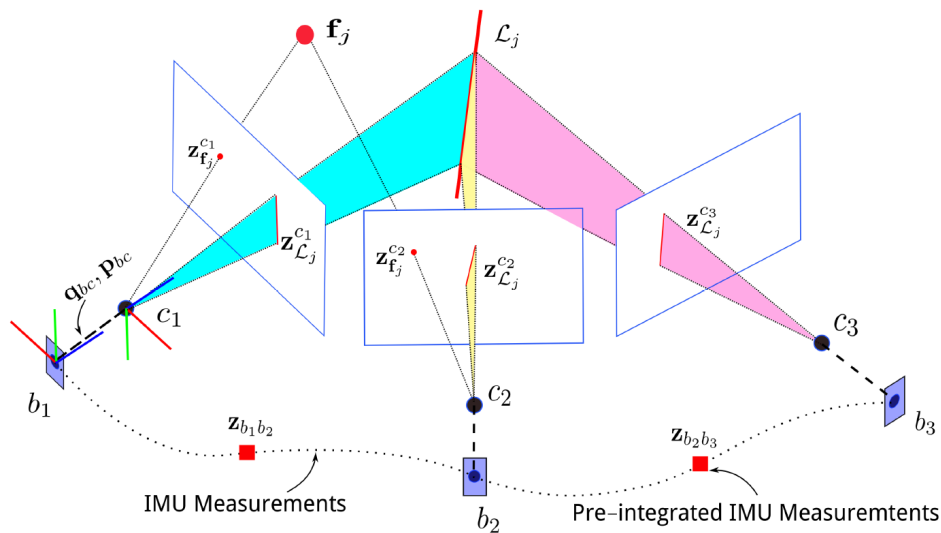


Abbildung 3.3: VIO-Messmodell über ein gleitendes Fenster von Kameraposen c_1, c_2, c_3 und IMU-Körperposen b_1, b_2, b_3 . Jedes Merkmal f_j wird in mehreren Bildern als visuelle Messung $z_{f_j}^{c_i}$ beobachtet; zwischen den Bildzeitpunkten verbinden *vorintegrierte* IMU-Messungen $z_{b_i b_{i+1}}$ die Posen. Die gemeinsame Optimierung beider Messtypen ergibt die fusionierte Trajektorie. Nach [4].

Zwei Fusionsphilosophien sind gebräuchlich:

- **Filterbasiert (loosely/tightly coupled EKF):** Ein EKF prädiziert den Zustand (Pose, Geschwindigkeit, IMU-Biases) mit der IMU und korrigiert ihn mit den visuellen Messungen. So arbeitet das ROS 2-Paket `robot_localization` und der in [1] beschriebene Aufbau, der VIO zusätzlich mit Radodometrie verschmilzt.
- **Optimierungsbasiert (Sliding-Window-Bundle-Adjustment):** Über ein gleitendes Fenster werden Kameraposen, Merkmale und *vorintegrierte* IMU-Faktoren *gemeinsam* optimiert (Abb. 3.3). Die *IMU-Vorintegration* fasst viele hochfrequente IMU-Messungen zwischen zwei Bildern zu einem einzigen Bewegungsfaktor zusammen – das macht die Optimierung handhabbar. Semi-direkte Verfahren wie SVO gehören hierher [4].

Praktische Auszahlung: VIO ist robuster und glatter als VO allein, übersteht kurze textur- oder

merkmalsarme Phasen (die IMU überbrückt sie) und liefert eine metrisch skalierte, neigungsstabile Trajektorie – die Grundlage für die `odom→base_link`-Transformation in GNSS-freien Innenräumen [4, 1].

Kapitel 4

Stereosehen und Merkmalerkennung

4.1 Lochkameramodell und Tiefe aus Disparität

Eine kalibrierte Kamera bildet einen 3D-Punkt über die Intrinsik-Matrix $\mathbf{K}$ ab:

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix} \quad (4.1)$$

Bei zwei horizontal um die **Basislinie** b versetzten Kameras erscheint ein Punkt im linken und rechten Bild an leicht verschiedener Spalte; die Differenz ist die **Disparität** d . Die Tiefe folgt direkt:

$$Z = \frac{f \cdot b}{d} \quad (4.2)$$

Große Basislinie $\rightarrow$ bessere Tiefenaufösung in der Ferne, aber kleinerer überlappender Sichtbereich in der Nähe. Die Disparität wird auf *rektifizierten* Bildern berechnet (beide Bildebenen koplanar, Epipolarlinien horizontal), typisch mit **StereoBM** (schnell) oder **StereoSGBM** (genauer) aus OpenCV.

4.2 Kalibrierung

Beide Kameras müssen kalibriert werden (Intrinsik + Verzerrung) und zueinander (Extrinsik: $\mathbf{R}, \mathbf{t}$ zwischen den Linsen). Standard ist ein Schachbrettmuster [5] und das ROS 2-Paket `camera_calibration` aus dem `image_pipeline`. Ergebnis sind `.yaml`-Dateien, die der Treiber lädt. **Ohne saubere Kalibrierung ist jede Tiefenmessung wertlos** – das ist der erste reale Stolperstein gegenüber der Simulation, wo die “Kamera” perfekt kalibriert ist. Schon kleine Kalibrierfehler skalieren systematisch in die Objektgenauigkeit hinein [6]; die praktische Durchführung samt detektiertem Schachbrettmuster ist in Kapitel 12 ausgeführt.

4.3 Merkmalerkennung ohne eigenes KI-Training

Es gibt ausgereifte *klassische* Detektoren, die ohne Trainingsdaten und ohne neuronale Netze auskommen. Sie finden markante, wiedererkennbare Bildpunkte (Ecken, Blobs) und beschreiben deren lokale Umgebung:

Detektor	Eigenschaft	Einsatz
ORB	schnell, binär, lizenzfrei	Echtzeit-Matching, VO, Standardwahl für Hobby

SIFT	skalierungs- /rotationsinvariant, robust	genaues Matching, CAD-Posenschätzung
AKAZE	gute Balance	Alternative zu SIFT/ORB
BRISK	binär, schnell	ressourcenarme Plattformen

Alle sind in OpenCV enthalten (`cv2.ORB_create()` usw.) und über `cv_bridge` in ROS 2 nutzbar. Das Matching erfolgt mit `BFMatcher` (Brute Force) oder `FLANN`, gefolgt von Ausreißerfilterung (Lowe-Ratio-Test, RANSAC). Eine quantitative Gegenüberstellung von Genauigkeit und Geschwindigkeit dieser Detektoren findet sich in Kapitel 14 [7]. Für **Bewegungserkennung** (Projekt 3) brauchst du nicht einmal Merkmale: Hintergrundsubtraktion (`cv2.createBackgroundSubtractorMOG2`) oder optischer Fluss (`calcOpticalFlowFarneback`) reichen.

4.4 Posenschätzung – 6D-Lage eines Bauteils

Für Projekt 4 (CAD-Matching) gibt es zwei Hauptwege:

1. **2D-3D über PnP:** Merkmale im Kamerabild gegen bekannte 3D-Punkte des CAD-Modells matchen, dann `cv2.solvePnPRansac()` liefert Rotation + Translation des Bauteils relativ zur Kamera.
2. **3D-3D über ICP:** Aus der Stereokamera eine Punktwolke erzeugen und per **Iterative Closest Point** an die aus dem CAD-Modell gesampelte Punktwolke anpassen (z. B. mit Open3D). Liefert die volle 6D-Pose, robust bei guter Initialschätzung.

Das Ergebnis (Rotation als Quaternion + Translation) wird in TF veröffentlicht und vom Greifplaner (MoveIt 2) konsumiert.

Kapitel 5

6-DoF-Posenschätzung mit gelernten Surface Embeddings

Dieses Kapitel vertieft die theoretischen Grundlagen hinter Projekt 4. Es baut direkt auf dem Stereosehen (Kapitel zuvor) auf: Dort wurde die 6D-Pose über klassische Merkmale (PnP, ICP) bestimmt; hier wird der moderne, lernbasierte Weg über **dichte Surface Embeddings** (in der Linie von **SurfEmb** [8]) entwickelt, der bei texturarmen, glänzenden oder symmetrischen Industrieteilen robuster ist.

5.1 Das Problem: sechs Freiheitsgrade

6-DoF-Posenschätzung bestimmt *Position und Orientierung* eines bekannten Bauteils – drei Translations- und drei Rotationsfreiheitsgrade. **Warum überhaupt?** Ein Werkstück liegt selten zweimal gleich (Schüttgut in Kisten, verschobene Vorrichtung, Förderband). Der Roboter muss das Teil *sehen*, bevor er greift. Gesucht ist die starre Transformation, die das bekannte CAD-Modell in das Kamera-/Roboterkoordinatensystem überführt:

$$\boxed{\mathbf{p}_{\text{cam}} = \mathbf{R} \mathbf{p}_{\text{model}} + \mathbf{t}} \quad \mathbf{R} \in SO(3), \quad \mathbf{t} \in \mathbb{R}^3 \quad (5.1)$$

Die rücktransformierte (“ausgerichtete”) Punktwolke ist nur die visuelle Bestätigung – das eigentliche Ergebnis ist $(\mathbf{R}, \mathbf{t})$.

5.2 Rotationsrepräsentationen

Die Translation ist trivial ($\mathbb{R}^3$); die Rotation ist der heikle Teil, weil $SO(3)$ gekrümmt ist. Gängige Darstellungen mit ihren Fallstricken:

Darstellung	Eigenschaft / Fallstrick
Rotationsmatrix (3×3)	9 Zahlen, 6 Nebenbedingungen (orthonormal); redundant aber stabil
Euler-Winkel	nur 3 Zahlen, aber Gimbal Lock (Freiheitsgrad-Verlust)
Quaternion	4 Zahlen, singularitätsfrei, aber Double Cover ($\mathbf{q}$ und $-\mathbf{q}$ = gleiche Drehung)
Kontinuierliche 6D-Form	beste Wahl für die NN-Regression – stetig, keine Sprünge [9]

Für neuronale Netze ist die Stetigkeit entscheidend: Euler und Quaternion haben Unstetigkeiten,

an denen das Netz schlecht lernt. Die 6D-Darstellung (zwei Spaltenvektoren $\rightarrow$ Gram-Schmidt $\rightarrow \mathbf{R}$) ist überall stetig.

5.3 Warum gelernte Embeddings statt Regression

Zwei naive Wege scheitern in der Industrie:

- **Handgefertigte Deskriptoren** (FPFH [10], SIFT) versagen bei texturlosen, glänzenden, symmetrischen Teilen und unter Verdeckung – genau die typischen Fälle in der Fertigung.
- **Direkte Pose-Regression** (Netz $\rightarrow$ ein $(\mathbf{R}, \mathbf{t})$) nimmt *eine* Antwort an (unimodal) und **bricht bei Symmetrie**: ein symmetrisches Teil hat mehrere korrekte Posen, der Mittelwert ist falsch.

Gelernte dichte Surface Embeddings lösen beides: Ein Netz bildet jeden Oberflächenpunkt in einen Merkmalsraum ab; gemessene Punkte werden über die *Nähe im Merkmalsraum* den Punkten des bekannten CAD-Modells zugeordnet. Statt einer Punktschätzung lernt das Modell eine **Verteilung** über Korrespondenzen – Symmetrie wird implizit behandelt (mehrere Oberflächenpunkte teilen sich dann schlicht ein Embedding) [8].

5.4 Kontrastives Lernen der Embeddings

Trainiert werden die Embeddings mit einem **kontrastiven Verlust**: wahre Pixel- $\leftrightarrow$ -Punkt-Paare werden im Merkmalsraum zusammengezogen, alle anderen Paare auseinandergedrückt. Die *Positiv*-Paare sind die unter bekannter Render-Pose bekannten Korrespondenzen, die *Negativ*-Paare andere gesampelte Oberflächenpunkte. In InfoNCE-Form für ein Query-Embedding $\mathbf{q}$ mit positivem Schlüssel $\mathbf{k}^+$:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(\mathbf{q}, \mathbf{k}^+)/\tau)}{\sum_j \exp(\text{sim}(\mathbf{q}, \mathbf{k}_j)/\tau)} \quad (5.2)$$

mit Kosinus-Ähnlichkeit $\text{sim}(\cdot)$ und **Temperatur** τ (Tuning-Größe). **Der Clou**: die Labels sind quasi gratis – man rendert das CAD-Modell in bekannten Posen, womit Ground-Truth-Pose *und* Korrespondenzen automatisch vorliegen; keine manuellen Symmetrie-Labels nötig.

5.5 Das Two-Tower-Modell (Query–Key)

Die Architektur besteht aus zwei getrennten Türmen, die erst im gemeinsamen Embedding-Raum zusammentreffen:

- **Query-Tower** – *Eingang*: Bildausschnitt (2D-Pixel, RGB/RGB-D); *Ausgang*: Embedding pro Pixel; *Architektur*: großes CNN-Encoder-Decoder-Netz (U-Net / ResNet).
- **Key-Tower** – *Eingang*: eine 3D-Oberflächenkoordinate (x, y, z) vom CAD; *Ausgang*: Embedding dieses Punktes im selben Raum; *Architektur*: kleines MLP (wenige Schichten reichen, da der Eingang simpel ist – die **Asymmetrie** der Türme ist gewollt).

Beide werden *gemeinsam* mit dem kontrastiven Verlust trainiert, sodass zusammengehörige Pixel- und Punkt-Embeddings nahe beieinander landen. Ein Turm spricht “2D-Bild”, der andere “3D-Oberfläche”; der gemeinsame Raum ist die gemeinsame Sprache.

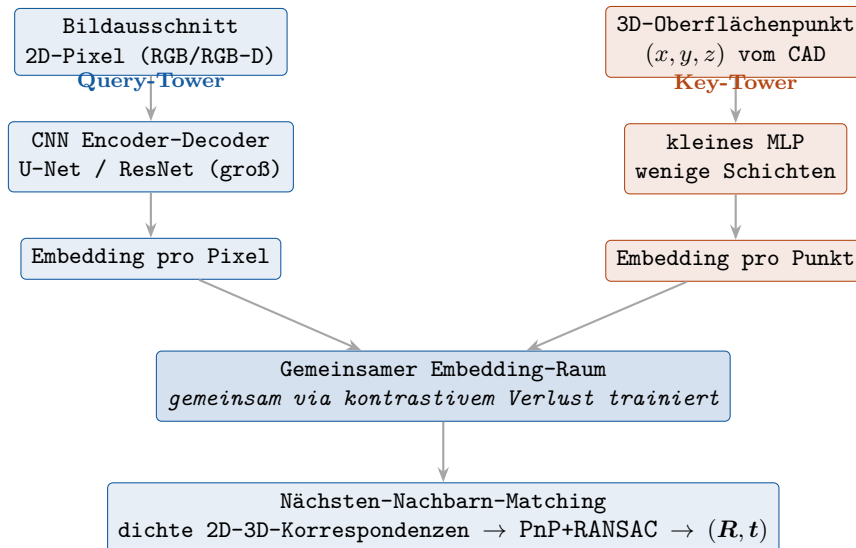


Abbildung 5.1: Two-Tower-Modell: die Türme bleiben getrennt bis zum gemeinsamen Embedding-Raum – genau diese Trennung *ist* die Architektur. Bei der Inferenz wird jedes Pixel-Embedding seinem nächsten Oberflächenpunkt-Embedding zugeordnet.

5.6 Von Korrespondenzen zur Pose: PnP

Aus den dichten 2D-3D-Korrespondenzen wird die Pose über **Perspective-n-Point** (PnP) gelöst: Gesucht ist die starre Transformation $(\mathbf{R}, \mathbf{t})$, die die bekannten 3D-Modellpunkte $\mathbf{P}_i$ so ins Kamerasystem dreht und schiebt, dass ihre Projektion möglichst genau auf die beobachteten Pixel $\mathbf{u}_i$ fällt.

Die Geometrie dahinter (Abb. 5.2): Ein 3D-Punkt $\mathbf{P}$ lebt zunächst im *Weltkoordinatensystem*. Die gesuchte Außenorientierung $[\mathbf{R} | \mathbf{t}]$ überführt ihn ins *Kamerakoordinatensystem*; die Brennweite f projiziert ihn auf die *normalisierte Bildebene*; die Intrinsik $\mathbf{K}$ (mit Hauptpunkt (u_0, v_0)) rechnet schließlich in Pixel um. PnP kehrt diese Kette um: aus n bekannten Punkt-Pixel-Paaren und bekanntem $\mathbf{K}$ rekonstruiert es $[\mathbf{R} | \mathbf{t}]$. Mathematisch ist es die Minimierung des **Reprojektionsfehlers**:

$$(\mathbf{R}, \mathbf{t}) = \arg \min_{\mathbf{R}, \mathbf{t}} \sum_i \|\mathbf{u}_i - \pi(\mathbf{K}(\mathbf{R}\mathbf{P}_i + \mathbf{t}))\|^2 \quad (5.3)$$

mit der Projektion $\pi(\cdot)$ (Division durch die Tiefe). Drei Korrespondenzen ($n = 3$, das *P3P*-Minimalproblem) reichen geometrisch für eine Lösung mit mehreren Kandidaten; ab $n \geq 4$ wird die Pose eindeutig und durch die Redundanz genauer und stabiler [11].

Klassische Löser: *DLT* (direkte lineare Transformation) ist schnell, aber rauschempfindlich; *EP-nP* löst das allgemeine Problem in $O(n)$; iterative Verfahren (Levenberg-Marquardt) verfeinern das Ergebnis durch nichtlineare Minimierung des Reprojektionsfehlers. Numerisch heikel bleiben der *planare* Fall (alle Punkte in einer Ebene) und *quasi-singuläre* Konfigurationen, weshalb genaue Verfahren die Rotation gesondert parametrisieren [11]. In der Praxis ruft man `cv2.solvePnP` auf – und genau dieses RANSAC-Mantelverfahren ist der nächste Baustein.

5.7 Robuste Schätzung: RANSAC

Reale Korrespondenzmengen enthalten **Ausreißer** (Fehlzuordnungen, bewegte Objekte). Eine Kleinste-Quadrate-Lösung über *alle* Punkte wird schon von wenigen groben Ausreißern ruiniert, weil deren quadrierter Fehler die Summe dominiert. **RANSAC** (Random Sample Consensus) umgeht das mit einer *Hypothesize-and-Verify*-Schleife [12]:

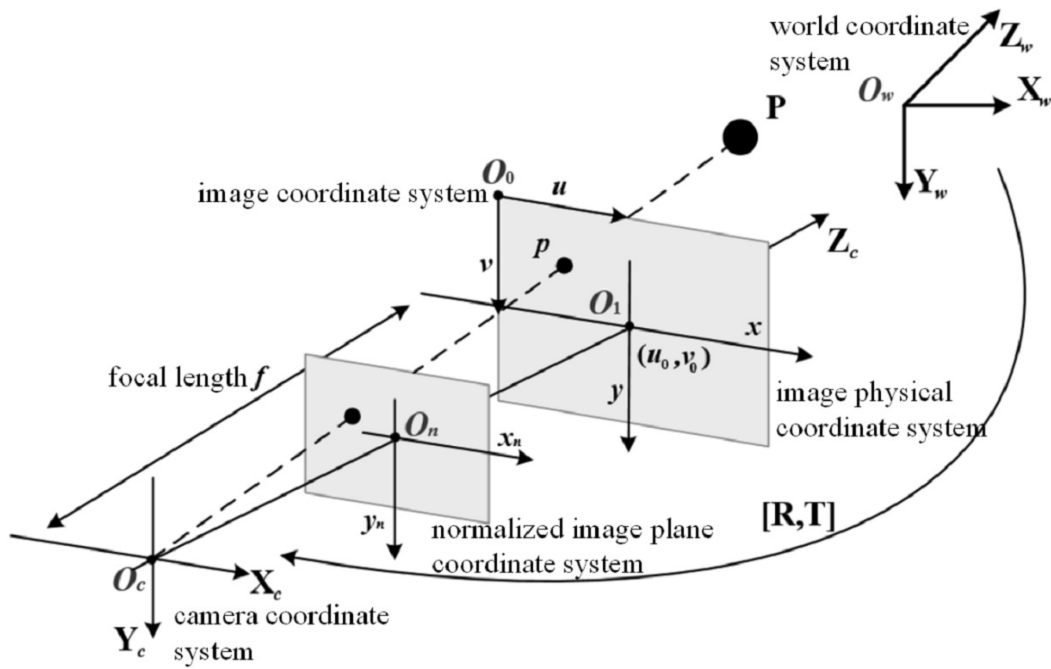


Abbildung 5.2: Koordinatensysteme der PnP-Posenschätzung: Ein Weltpunkt P wird über die Außenorientierung $[R, t]$ ins Kamerasystem O_c , dann über die Brennweite f auf die normalisierte Bildebene und schließlich über die Intrinsik (Hauptpunkt (u_0, v_0)) in Pixelkoordinaten abgebildet. PnP löst die Umkehrung: aus Punkt-Pixel-Paaren und bekanntem K die gesuchte Pose $[R, t]$. Nach [11].

1. **Sample:** Ziehe zufällig eine *minimale* Punktmenge (für eine Gerade: 2 Punkte; für PnP: 3–4 Korrespondenzen) und schätze daraus ein Modell.
2. **Verify:** Zähle, wie viele aller Punkte näher als eine Schwelle τ am Modell liegen – die **Inlier** (der *Consensus Set*).
3. **Iterate:** Wiederhole; behalte das Modell mit den meisten Inliern.
4. **Refine:** Schätze das finale Modell aus *allen* Inliern neu (Kleinste Quadrate ohne Ausreißer).

Die nötige Iterationszahl N ergibt sich aus der gewünschten Erfolgswahrscheinlichkeit p (z. B. 0,99), dem Inlier-Anteil w und der Sample-Größe s :

$$N = \frac{\log(1 - p)}{\log(1 - w^s)} \quad (5.4)$$

– der Aufwand steigt steil, wenn der Ausreißeranteil wächst oder s größer wird, was die adaptiven Echtzeit-Varianten motiviert [12].

Genau dieses Verfahren steckt in `solvePnP`ransac: Jede minimale Korrespondenz-Stichprobe liefert ein Kandidaten- (R, t) , dessen Reprojektionsfehler über alle Punkte die Inlier zählt. Bei reinen 3D-3D-Daten tritt an die Stelle der Reprojektion die euklidische Punktdistanz, das Mantelverfahren bleibt identisch.

5.8 Registrierung: grob und fein

Eine *einstufige* Registrierung ist unzuverlässig; Standard sind **zwei Stufen**:

1. **Grob (global):** liefert die “richtige Größenordnung” aus Merkmalen – Deskriptoren auf Quelle und Modell matchen, grobes (R, t) lösen (z. B. SAC-IA + FPFH). Braucht *keine* gute Startschätzung, ist aber nur näherungsweise.
2. **Fein = ICP** (*Iterative Closest Point* [13]): zu jedem Quellpunkt den nächsten Zielpunkt suchen,

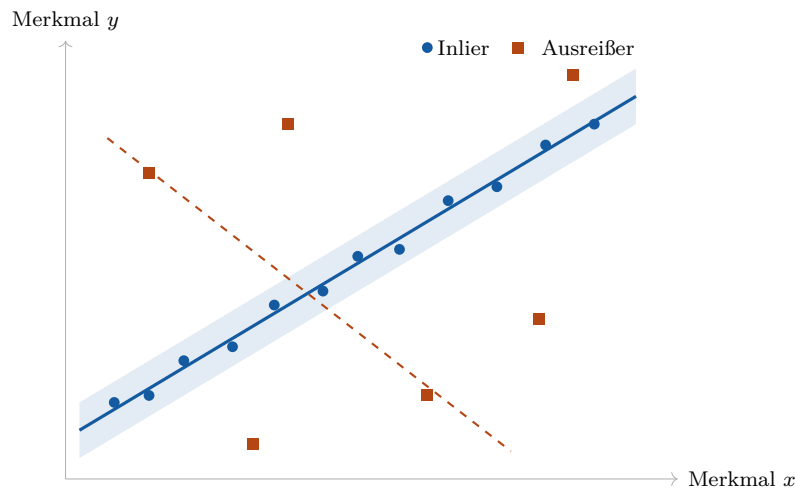


Abbildung 5.3: RANSAC am Beispiel der Geradenanpassung: Die durchgezogene Linie ist das Modell mit dem größten *Consensus Set* – alle Punkte im hellen Band ($\pm\tau$) zählen als Inlier; die mit Kreuzen markierten Ausreißer werden verworfen. Die gestrichelte Linie ist eine zufällig gezogene Hypothese, die wegen zu weniger Inlier wieder verworfen wird. Prinzip nach [12].

die Gesamtdistanz minimieren, wiederholen bis zur Konvergenz:

$$E(\mathbf{R}, \mathbf{t}) = \sum_i \|\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{c(i)}\|^2, \quad c(i) = \text{nächster Nachbar} \quad (5.5)$$

Genau, aber nur **lokal**.

ICP im Detail (Abb. 5.4): Jede Iteration besteht aus zwei Schritten, die abwechselnd ausgeführt werden, bis sich nichts mehr ändert:

- (i) **Korrespondenz:** Ordne jedem transformierten Quellpunkt seinen *nächsten* Zielpunkt zu (Nearest-Neighbor-Suche, beschleunigt per k-d-Baum) – das liefert die aktuellen Punktpaare.
- (ii) **Ausrichtung:** Bestimme bei *festen* Korrespondenzen das optimale $(\mathbf{R}, \mathbf{t})$, das die Summe der quadrierten Abstände minimiert. Das ist das **Procrustes-Problem** und hat eine geschlossene Lösung über die Singulärwertzerlegung der Kreuzkovarianzmatrix [14].

Beide Schritte senken den Fehler monoton, weshalb ICP garantiert konvergiert – allerdings nur in das *nächstgelegene* Minimum.

ICP-Schwächen: (1) *lokale Minima* – es läuft bergab und bleibt stecken; (2) *initialisierungsabhängig* – es funktioniert nur bei nahem Start (daher läuft die Grobstufe zuerst). **Go-ICP** [15] garantiert das globale Optimum, indem es ganz $SE(3)$ (alle starren Transformationen) per *Branch-and-Bound* durchsucht: Raum aufteilen, Fehler je Region beschränken, Regionen verwerfen, die nicht gewinnen können – langsamer, aber global optimal.

5.9 Punktwolken-Netze: PointNet und PointNet++

Verarbeitet man die Punktwolke direkt mit einem Netz, ist die zentrale Schwierigkeit: eine Punktwolke ist eine **ungeordnete Menge** – der Ausgang darf nicht von der Reihenfolge abhängen (**Permutationsinvarianz**).

- **PointNet** [16]: dasselbe geteilte MLP auf jeden Punkt, dann **Max-Pooling** (symmetrisch = reihenfolgeunabhängig). Erfasst die *globale* Form, verpasst aber lokale Details.
- **PointNet++** [17]: wendet PointNet *lokal* in Stufen an – Zentren sampeln (Farthest-Point-Sampling), Nachbarschaften gruppieren (Multi-Scale-Radien), lokale Merkmale extrahieren, auf größerer Skala wiederholen. Baut lokal + global hierarchisch auf, ähnlich einem CNN.

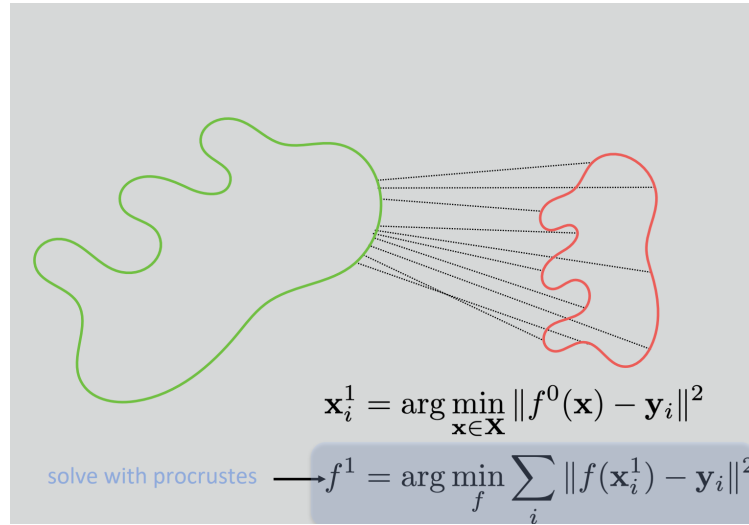


Abbildung 5.4: Eine ICP-Iteration: Zu jedem Punkt der Quellform (grün) wird der nächste Punkt der Zielform (rot) gesucht (gepunktete Korrespondenzen, $\mathbf{x}_i^1 = \arg \min_{\mathbf{x}} \|f^0(\mathbf{x}) - \mathbf{y}_i\|^2$); anschließend liefert die Procrustes-Lösung die starre Transformation f^1 , die die paarweise Distanz minimiert. Beide Schritte werden bis zur Konvergenz wiederholt. Nach [14].

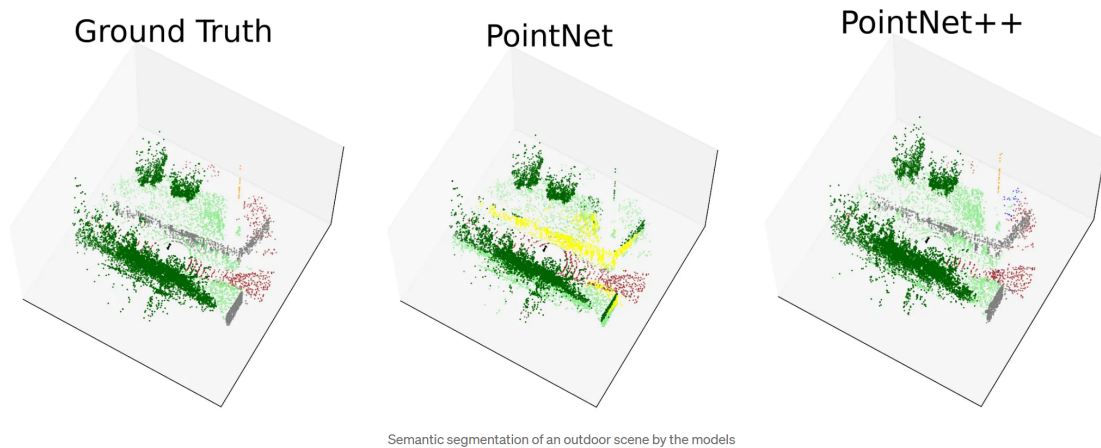


Abbildung 5.5: Semantische Segmentierung einer Außenszene im Vergleich: Gegenüber der Ground Truth (links) verschmiert **PointNet** (Mitte) feine lokale Strukturen, weil das globale Max-Pooling die Nachbarschaftsinformation verliert. **PointNet++** (rechts) gewinnt durch die hierarchische, lokale Merkmalsextraktion deutlich saubere Klassengrenzen zurück. Nach [18].

Genau diese hierarchische Lokalität ist der praktische Unterschied: Für die feine Registrierung texturarmer Industrieteile zählen lokale Oberflächendetails, weshalb PointNet++ (bzw. darauf aufbauende Architekturen) in der Posenschätzung dem ursprünglichen PointNet vorgezogen wird [18, 17].

5.10 Stereo, Tiefe und Skalierung

Warum Stereo? Eine Mono-Kamera hat *Skalenmehrdeutigkeit* (klein-nah $\approx$ groß-fern). Stereo liefert Tiefe und löst die Skala – über Disparität, Basislinie und Brennweite (Triangulation, vgl. Stereo-Kapitel). Zwei Wege in die Pipeline:

- (a) Tiefe als **Zusatzkanal** = RGB-D-Eingang, oder
- (b) Tiefe + Intrinsik **rückprojizieren** $\rightarrow$ Punktwolke $\rightarrow$ 3D-3D-Registrierung.

Vorverarbeitung (vor dem Netz, ja): Kalibrierung (Intrinsik + Extrinsik) und Rektifizierung, Tiefen-Entrauschung, Umgang mit ungültiger Tiefe (glänzendes Metall liefert schlechte Tiefenwerte). **Auszahlung:** die Tiefe gibt *metrische* Skala – $(\mathbf{R}, t)$ in echten Millimetern, mit denen der Roboter handeln kann.

5.11 Zeitliche Konsistenz: Kalman-Pose-Tracking

Eine bildweise geschätzte Pose ist verrauscht/zittrig. Ein **Kalman-Filter** ergänzt *zeitliche Konsistenz* (vgl. Kapitel zu Kalman/EKF): der *Zustand* ist die Pose (ggf. plus Geschwindigkeit), die *Messung* ist die bildweise Netz-Pose. Prädiktion der nächsten Pose $\rightarrow$ Korrektur mit der neuen Schätzung. Das **glättet Jitter**, **verwirft Ausreißer-Frames** und **überbrückt kurze Verdeckung / ausgefallene Detektionen** – aus Einzelschuss wird *Tracking*, relevant für bewegtes Förderband oder armmontierte Kamera. Da Rotationen nichtlinear sind, nutzt man EKF/UKF oder filtert auf einer geeigneten Rotationsdarstellung.

5.12 Sim-to-Real und Trainingsdaten

Da die Labels durch Rendern des CAD-Modells praktisch gratis entstehen, wird synthetisch trainiert – aber es klappt ein **Sim-to-Real-Gap** (synthetisch $\rightarrow$ real: Rauschen, Beleuchtung, Reflektanz). Gegenmittel: **Domain Randomization**, physikalisch-basiertes Rendering (PBR, z. B. via Isaac Sim, vgl. Kapitel zu Simulationswerkzeugen) und ein kleiner realer Feintuning-Datensatz.

5.13 Die Pipeline: von CAD zu Trainingsdaten

Das Vorgehen ist in der Praxis (und in den BOP-Datensätzen [19]) eine feste Kette von Schritten. Sie produziert pro Szene nicht nur ein Bild, sondern den *kompletten* Annotationssatz, den der kontrastive Verlust (Abschnitt zuvor) braucht.

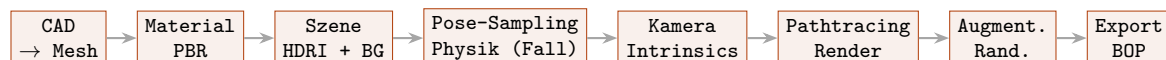


Abbildung 5.6: Synthese-Pipeline. Parallel dazu läuft eine *reale* Validierungsschicht (ein paar hundert annotierte Bilder), die nicht trainiert, sondern nur den Sim-to-Real-Abfall misst.

1. **CAD-Vorbereitung.** Konvertierung in ein renderfähiges Mesh (OBJ, PLY, glTF). Dabei oft: Mesh-Vereinfachung bei überaufgelösten Modellen, Sicherstellen der *Wasserdichtigkeit*, Skalierung in metrische Einheiten (mm) – der Roboter arbeitet später in Millimetern.
2. **Material- und Texturzuweisung.** Realistisches **PBR-Material** (Roughness, Metallic, ggf. Normal Maps). Glänzende Metallteile: hoher Metallic-, niedriger Roughness-Wert – der Hebel, mit dem Reflexionen physikalisch korrekt entstehen.

3. **Szenenaufbau.** Hintergrund (zufällige Domain-Randomization-Textur *oder* realistische Industrieumgebung) und Beleuchtung, meist über **HDRI-Environment-Maps** (360°-Panoramen), die Licht *und* Reflexionen im Material zugleich liefern – der zweite große Realismus-Hebel bei Metall.
4. **Pose-Sampling per Physik.** Statt zufälliger Platzierung wird simuliert, wie Teile in eine Kiste *fallen* (Bullet in BlenderProc, PhysX in Omniverse): mehrere Instanzen kollidieren und kommen in stabilen Lagen zur Ruhe. Extrahiert werden genau die Posen, die real vorkommen – keine schwebenden oder unmöglichen Lagen.
5. **Kamera-Sampling.** Virtuelle Kamera in plausiblen Positionen (typisch schräg von oben, wie eine Bin-Picking-Kamera). Die **Intrinsics** werden auf die spätere reale Kamera (RealSense, Photoneo) gesetzt, damit das Netz keine Brennweiten-Diskrepanz mitlernen muss.
6. **Rendering.** Der Pathtracer erzeugt pro Szene mehrere Ausgänge zugleich: RGB-Bild, Tiefenkarte (für RGB-D), Segmentierungsmaske, 6-DoF-Pose pro Instanz und – entscheidend – die **dichten 2D-3D-Korrespondenzen** (zu jedem Pixel die exakte 3D-Oberflächenkoordinate auf dem CAD). Genau diese Korrespondenzen sind das Trainingssignal für den kontrastiven Verlust in SurfEmb [8].
7. **Augmentierung / Domain Randomization.** Beim Rendern oder als Nachbearbeitung: Beleuchtungsvariation, Hintergrund-Austausch, Sensorrauschen, Defokus- und Bewegungsunschärfe, JPEG-Artefakte – damit das Netz robust gegen den Unterschied zwischen synthetischer und realer Sensorik wird.
8. **Datensatz-Export.** Ausgabe in einem standardisierten Format; bei Evaluation auf **BOP** [19] direkt im BOP-Format, sodass das Trainingsset zur Benchmark-Pipeline kompatibel ist (BlenderProc bringt fertige Writer mit).
9. **Reale Validierungsschicht.** Parallel hält man einen kleinen Satz *real* annotierter Bilder vor (typisch ein paar hundert) – nicht zum Trainieren, sondern zur Messung der Sim-to-Real-Leistung. Fällt die Genauigkeit zu stark ab, wird mit diesem Set fine-getuned oder die Renderpipeline angepasst (mehr Randomization, realistischere Materialien, andere HDRIs).

Der Knackpunkt bei Industrieteilen

Zwei Schritte entscheiden über Sim-to-Real bei glänzendem Metall: das **PBR-Material** (Schritt 2) und die **HDRI-Beleuchtung** (Schritt 3). Sie erzeugen die Reflexionen, an denen klassische Deskriptoren scheitern – und damit genau das Erscheinungsbild, das das Netz robust verarbeiten lernen muss.

5.14 Werkzeuge: Renderer im Vergleich

Vier Werkzeugfamilien decken die Pipeline ab; die Wahl bestimmt vor allem Geschwindigkeit, Integrationstiefe und Verbreitung im Feld.

BlenderProc – der De-facto-Standard.

Python-Wrapper um Blender, speziell für synthetische Trainingsdaten [20]. Damit werden die offiziellen BOP-Datensätze gerendert, und die meisten modernen Methoden (SurfEmb, GDR-Net, CosyPose) nutzen es in ihren Papern. *Vorteile:* kostenlos, Open Source, große Community, eingebaute Physik (Bullet) für plausibles Pose-Sampling, fertige BOP-Writer, photorealistisches Rendering über Blenders Cycles-Pathtracer. Der Name, den man im Gespräch nennt.

NVIDIA Omniverse Replicator / Isaac Sim – die industrielle Alternative.

Auf Omniverse/USD aufgebaut, mit RTX-beschleunigtem Pathtracing und enger Anbindung an robotische Simulation (Isaac Sim) [21, 22]. *Stärken:* schneller bei großen Mengen dank GPU-Pathtracing, gute Domain-Randomization-API, direkte Integration in Robotik-Workflows. Ernst-haft Option, wenn die Posenschätzung später Teil einer größeren Robotik-Simulation werden soll.

Unity Perception / Unreal Engine.

Beide Game Engines haben Pakete für synthetische Daten: Unity das **Perception Package** mit fertigen Labelern [23], Unreal über das NDDS-Plugin oder Python-APIs. In der Pose-Estimation-Community weniger verbreitet als BlenderProc, in der Industrie (besonders Automotive) aber präsent.

Spezialisierte / proprietäre Tools.

CAE-/Robotik-Suiten (z.B. NX, Tecnomatix Process Simulate) können CAD und Robotik simulieren, sind aber keine vollständigen Pathtracer für realistische Trainingsbilder. Möglich, dass ein Hersteller intern eine eigene Pipeline auf einer solchen Basis aufgesetzt hat – eine gute Rückfrage an den Betreuer.

Werkzeug	Stärke	Pathtracer	Verbreitung
BlenderProc	BOP-nativ, Physik, gratis	Cycles (CPU/GPU)	Standard im Feld
Omniverse Replicator	GPU-Tempo, Robotik-Integration	RTX	Industrie/NVIDIA-Stack
Unity / Unreal	fertige Labeler, Engine-Ökosystem	Engine-Renderer	v. a. Automotive
Proprietär (NX etc.)	CAD-/Robotik-Nähe	kein voller PT	herstellerintern

Faustregel: Für Forschung und BOP-Vergleichbarkeit **BlenderProc**; für große Datenmengen und einen durchgängigen Robotik-Stack **Omniverse Replicator / Isaac Sim**.

5.15 Benchmarking und Metriken

Größe	Bedeutung
ADD / ADD-S	mittlerer Punktabstand Modell↔Schätzung; ADD-S nutzt nächsten Punkt (für <i>symmetrische</i> Teile)
Rotations-/Translationsfehler	geodätisch (Rotation) bzw. euklidisch (Translation), dazu RMSE
BOP-Metriken	VSD, MSSD, MSPD → Average Recall [19]
Industrie-KPIs	Taktzeit/Latenz, Pick-Erfolgsrate, Falsch-Positiv-Rate
Datensätze	BOP-Suite, speziell T-LESS und ITODD (texturlos/industriell)

Kernaussage in einem Satz

6-DoF-Posenschätzung über gelernte Surface Embeddings ersetzt fragile handgefertigte Deskriptoren durch ein *Zwei-Turm-Netz*, das Pixel und CAD-Oberflächenpunkte in einen gemeinsamen Merkmalsraum abbildet; das dichte Korrespondenz-Matching liefert über **PnP+RANSAC** (optional ICP-verfeinert) die starre Transformation $(\mathbf{R}, \mathbf{t})$ – robust auch bei texturlosen, glänzenden und symmetrischen Teilen.

Kapitel 6

Kinematik, Dynamik und Trajektorien

6.1 Vorwärtskinematik

Aus den Gelenkwinkeln $\mathbf{q}$ die Endeffektor-Pose berechnen. Klassisch über die Kette homogener 4×4 -Transformationen (Denavit-Hartenberg-Parameter). Der Rotationsblock jeder Transformation ist eine 3×3 -Matrix; intern äquivalent zur Quaternion-Darstellung.

6.2 Inverse Kinematik

Aus der gewünschten Pose die Gelenkwinkel finden. Numerisch über die **Jacobi-Matrix** $\mathbf{J}$, die Gelenkgeschwindigkeiten auf Endeffektorgeschwindigkeiten abbildet ($\dot{\mathbf{x}} = \mathbf{J}\dot{\mathbf{q}}$). Iterativ:

$$\Delta \mathbf{q} = \mathbf{J}^\dagger \mathbf{e}, \quad \mathbf{e} = \begin{pmatrix} \mathbf{e}_{\text{pos}} \\ \mathbf{e}_{\text{ori}} \end{pmatrix} \quad (6.1)$$

mit Pseudo-Inverser $\mathbf{J}^\dagger$ (oder Damped-Least-Squares nahe Singularitäten). Der **Orientierungsfehler** $\mathbf{e}_{\text{ori}}$ wird als Vektorteil des Fehlerquaternions $q_{\text{soll}} \otimes q_{\text{ist}}^{-1}$ gebildet – Euler-Differenzen sind hier wegen Gimbal Lock unbrauchbar. In ROS 2 löst **MoveIt 2** (mit KDL/TracIK) das für dich.

6.3 Dynamik

Das Bewegungsgesetz eines Roboterarms:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} \quad (6.2)$$

$\mathbf{M}$ = Massenmatrix, $\mathbf{C}$ = Coriolis/Zentrifugal, $\mathbf{g}$ = Gravitation, $\boldsymbol{\tau}$ = Gelenkmomente. Für die Simulation ist entscheidend, dass du jedem Link in der URDF korrekte **Masse und Trägheitstensoren** gibst – sonst rechnet die Physik-Engine Unsinn.

6.4 Trajektorienplanung

- **Gelenkraum:** Kubische/quintische Polynome oder ein **trapezförmiges Geschwindigkeitsprofil** (beschleunigen – konstant – abbremesen) erzeugen ruckarme, ableitungsstetige Verläufe.
- **Kartesischer Raum:** Position linear interpolieren, Orientierung per **SLERP** (gleichmäßige Winkelgeschwindigkeit). Für glatte Mehrpunktbahnen **SQUAD**.

In ROS 2 transportiert `trajectory_msgs/JointTrajectory` eine Bahn (Punkte mit Position/Geschwindigkeit/Zeit) und der `joint_trajectory_controller` fährt sie ab.

Kapitel 7

Regelungstechnik der Robotergrundlagen

Die Dynamik aus Kapitel 6 sagt, *welches* Gelenkmoment τ eine gewünschte Bewegung erzeugt. Die Regelung beantwortet die schwierigere Frage: Wie hält der Roboter die Sollbahn **trotz** Modellfehlern, Reibung, Nutzlastwechseln und Stößen? Dieses Kapitel ordnet die gängigen Regelungsarchitekturen ein – mit dem Fokus, **wann** man welche einsetzen *muss* und worin der Unterschied zu den Alternativen liegt.

Grundlage ist überall das Bewegungsgesetz aus dem Kinematik-Kapitel:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + \tau_{\text{stör}} \quad (7.1)$$

mit Massenmatrix M , Coriolis/Zentrifugal-Term C , Gravitation g und einer Störung $\tau_{\text{stör}}$ (Reibung, Kontaktkräfte, unmodellerte Last). Der Bahnfehler ist $e = q_{\text{soll}} - q$.

7.1 Feedback: die Grundlage

Feedback (Rückführung) misst den Fehler e und stellt ihm ein korrigierendes Moment entgegen. Der Klassiker im Gelenkraum ist ein **PD-Regler mit Gravitationskompensation**:

$$\tau = K_p e + K_d \dot{e} + g(q) \quad (7.2)$$

Wann einsetzen

Immer – Feedback ist das Fundament jeder Roboterregelung. Es ist die einzige Komponente, die *unbekannte* Störungen und Modellfehler nachträglich ausbügelt. Ein reiner PD/PID im Gelenkraum genügt bei langsamen Bewegungen, hoher Getriebeuntersetzung (entkoppelt die Gelenke quasi voneinander) und moderaten Genauigkeitsanforderungen – etwa beim Anfahren von Wegpunkten.

Unterschied zu den anderen: Feedback reagiert *erst nach* dem Auftreten des Fehlers – es ist reaktiv und damit zwangsläufig „hinterher“. Hohe K_p verkürzen die Reaktion, fachen aber Rauschen und Schwingungen an. Alles Folgende (Vorsteuerung, Entkopplung, Beobachter) dient dazu, dem Feedback weniger Arbeit zu hinterlassen.

7.2 Vorsteuerung (Feedforward)

Die Vorsteuerung rechnet aus der *bekannten* Sollbahn das nötige Moment **vorab** aus und schaltet es additiv zum Feedback. Mit dem Modell (7.1) ergibt sich die ideale **dynamische Vorsteuerung**:

$$\tau = \underbrace{M(q)\ddot{q}_{\text{soll}} + C(q, \dot{q})\dot{q}_{\text{soll}} + g(q)}_{\text{Vorsteuerung (Modell)}} + \underbrace{K_p e + K_d \dot{e}}_{\text{Feedback}} \quad (7.3)$$

Das ist eine **Zwei-Freiheitsgrade-Struktur**: Die Vorsteuerung bestimmt das Führungsverhalten (Bahnfolge), das Feedback nur noch das Störverhalten (Restfehler). Beide lassen sich getrennt auslegen.

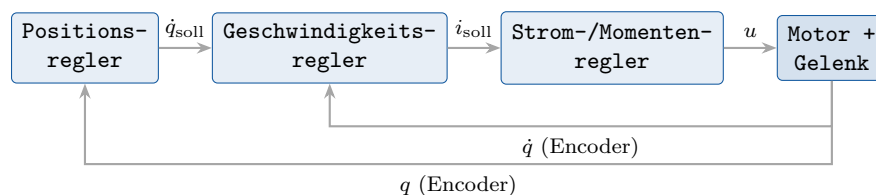
Wann einsetzen

Sobald die Bahn *im Voraus bekannt* ist und schnell/präzise gefolgt werden soll – Schweißnähte, Pick-and-Place mit kurzen Taktzeiten, der Schlagarm des Tischtennisroboters. Die Vorsteuerung eliminiert den *systematischen* Schleppfehler, ohne die Feedback-Verstärkung hochziehen zu müssen.

Unterschied zum Feedback: Vorsteuerung ist *vorausschauend* und kennt keinen Messfehler – sie kann daher nicht instabil werden, aber auch keine unvorhergesehene Störung ausgleichen. Sie ist nur so gut wie das Modell. Darum nie allein, sondern stets *kombiniert* mit Feedback.

7.3 Kaskadenregelung

Statt einer einzigen Schleife schachtelt die Kaskade mehrere ineinander – jede mit eigener, messbarer Zwischengröße. Beim Servoantrieb sind das drei Ebenen:



Die **innere** Schleife (Strom) ist am schnellsten getaktet ($\sim \text{kHz} - 10 \text{ kHz}$), die **äußere** (Position) am langsamsten. Faustregel: jede innere Schleife mindestens $3-5\times$ schneller als die nächstäußere.

Wann einsetzen

Praktisch in jedem industriellen Antrieb – Servoverstärker und FOC-Treiber sind intern bereits kaskadiert. Sinnvoll, sobald eine schnell messbare Zwischengröße (Strom, Drehzahl) existiert und die Stellgröße begrenzt werden muss.

Unterschied zu den anderen: Die Kaskade ist eine *strukturelle* Anordnung mehrerer Feedback-Schleifen, kein eigenständiges Regelgesetz. Ihr Vorteil: Störungen, die in einer inneren Schleife angreifen (z. B. Versorgungsspannungsschwankungen), werden dort sofort abgefangen, bevor sie die äußere erreichen; außerdem lässt sich jede Ebene einzeln in Betrieb nehmen und sättigen (Anti-Windup, Strombegrenzung). Querkopplungen zwischen den Gelenken behandelt sie aber nicht – dafür braucht es Entkopplung (s. u.).

7.4 Feedback-Entkopplung (Computed Torque)

Bei schnellen Bewegungen sind die Gelenke über M und C stark *gekoppelt*: ein bewegtes Gelenk übt Reaktionsmomente auf die anderen aus. Die Feedback-Entkopplung (auch **inverse Dynamik**-

oder **Computed-Torque**-Regelung) hebt das per exakter Modellrechnung auf. Man setzt eine Hilfsgröße $\mathbf{a}$ an und invertiert die Dynamik:

$$\boldsymbol{\tau} = \mathbf{M}(\mathbf{q}) \mathbf{a} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}), \quad \mathbf{a} = \ddot{\mathbf{q}}_{\text{soll}} + \mathbf{K}_d \dot{\mathbf{e}} + \mathbf{K}_p \mathbf{e} \quad (7.4)$$

Eingesetzt in (7.1) (ohne Störung) bleibt das exakt **lineare, entkoppelte** Fehlersystem

$$\ddot{\mathbf{e}} + \mathbf{K}_d \dot{\mathbf{e}} + \mathbf{K}_p \mathbf{e} = \mathbf{0}, \quad (7.5)$$

also für *jedes* Gelenk ein unabhängiges, frei platzierbares PD-Verhalten – unabhängig von Stellung und Geschwindigkeit.

Wann einsetzen

Schnelle, dynamische Bewegungen mit leichter Struktur und geringer Getriebeuntersetzung, bei denen die Kopplung spürbar wird – Leichtbauarme, Direktantriebe, der hochbeschleunigte Schlagarm. Voraussetzung: ein hinreichend genaues Dynamikmodell und Momenten-/Stromregelbare Antriebe.

Unterschied zur Vorsteuerung: Beide nutzen das Modell. Die Vorsteuerung (7.3) wertet es entlang der *Soll*-Größen aus (Steuerung, offene Wirkung); die Feedback-Entkopplung (7.4) wertet $\mathbf{M}$, $\mathbf{C}$ entlang der *Ist*-Größen aus und linearisiert die Strecke *innerhalb* der geschlossenen Schleife. Dadurch ist das resultierende Fehlverhalten arbeitspunktunabhängig konstant – der Preis ist die Modellauswertung in Echtzeit und die Empfindlichkeit gegen Modellfehler.

7.5 Regelung im Arbeitsraum (Operational Space Control)

Bisher wurde im Gelenkraum geregelt; die Aufgabe ist aber meist im **kartesischen Raum** definiert (Pose des Endeffektors $\mathbf{x}$). Statt erst per inverser Kinematik nach $\mathbf{q}_{\text{soll}}$ umzurechnen, formuliert man Regler und Dynamik direkt in $\mathbf{x}$ – über die Jacobi-Matrix $\mathbf{J}$ mit $\dot{\mathbf{x}} = \mathbf{J} \dot{\mathbf{q}}$. Das entkoppelte Gesetz im Arbeitsraum lautet

$$\boldsymbol{\tau} = \mathbf{J}^\top [\boldsymbol{\Lambda}(\mathbf{q}) \mathbf{a}_x + \boldsymbol{\mu}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{p}(\mathbf{q})], \quad \mathbf{a}_x = \ddot{\mathbf{x}}_{\text{soll}} + \mathbf{K}_d \dot{\mathbf{e}}_x + \mathbf{K}_p \mathbf{e}_x \quad (7.6)$$

mit der Arbeitsraum-Massenmatrix $\boldsymbol{\Lambda} = (\mathbf{J} \mathbf{M}^{-1} \mathbf{J}^\top)^{-1}$. Der Fehler $\mathbf{e}_x$ wird direkt im kartesischen Raum gebildet (Position als Differenz, Orientierung als Vektorteil des Fehlerquaternions, vgl. Kapitel 6).

Wann einsetzen

Wenn die *Aufgabe* im kartesischen Raum lebt: Kraft-/Impedanzregelung beim Kontakt (Polieren, Stecken, Montage), Bahnverfolgung am Werkstück, nullraumoptimierte redundante Arme. Auch dann, wenn die inverse Kinematik nahe Singularitäten unzuverlässig wird.

Unterschied zur Gelenkraumregelung: Identische Mathematik, anderer Raum. Im Arbeitsraum legst du Steifigkeit und Dämpfung *richtungsabhängig* fest (z. B. nachgiebig in Stoßrichtung, steif quer dazu) – ideal für Kontaktaufgaben. Nachteil: $\mathbf{J}$ wird nahe Singularitäten schlecht konditioniert, und der Nullraum redundanter Roboter muss separat behandelt werden.

7.6 Zustandsbeobachter

Computed-Torque und Vorsteuerung brauchen $\dot{\mathbf{q}}$ (oft auch Beschleunigungen oder die Störung) – gemessen wird aber meist nur die Position $\mathbf{q}$. Numerisches Differenzieren verstärkt Rauschen. Der **Zustandsbeobachter** schätzt die nicht gemessenen Zustände aus Modell und Eingang, korrigiert über den Messfehler:

$$\dot{\hat{\mathbf{x}}} = \mathbf{f}(\hat{\mathbf{x}}, \mathbf{u}) + \mathbf{L}(\mathbf{y} - \hat{\mathbf{y}}) \quad (7.7)$$

Die Beobacherverstärkung $\mathbf{L}$ legt fest, wie schnell die Schätzung dem wahren Zustand folgt (Luenberger für lineare, EKF für nichtlineare Systeme – siehe Kapitel 2).

Wann einsetzen

Wenn Zustände für das Regelgesetz gebraucht werden, die kein Sensor direkt liefert: Gelenkgeschwindigkeit aus reinem Positionsenncoder, Verspannung in einem elastischen Gelenk, oder – besonders wichtig – die *Störung* selbst (s. u.).

Unterschied zum Kalman-Filter: Der Beobachter ist das deterministische Konzept; der Kalman-Filter ist der Beobachter, dessen $\mathbf{L}$ statistisch optimal aus den Rausch-Kovarianzen bestimmt wird. Funktional dasselbe Bauteil im Regelkreis.

7.7 Störgrößenkompensation

Reibung, unbekannte Nutzlast und Kontaktkräfte stecken in $\tau_{\text{stör}}$ aus (7.1). Drei Strategien – gestaffelt nach dem, was über die Störung bekannt ist:

7.7.1 Statische vs. dynamische Störgrößenausschaltung

Lässt sich die Störung *messen* oder aus den Eingängen *berechnen* (z. B. ein bekanntes Lastmoment, das von der nachgeführten Last abhängt), schaltet man ihren Effekt vorausberechnet auf – analog zur Vorsteuerung, nur für die Störung statt für die Führung. Die **dynamische** Variante berücksichtigt dabei das *zeitliche* Übertragungsverhalten zwischen Störangriff und Ausgang (nicht nur den stationären Wert), sodass die Kompensation auch während des Einschwingens passt.

Wann einsetzen

Wenn die Störung *messbar* ist (Kraftsensor, bekannte, mitgeführte Last) und schnell genug wirkt, dass reines Feedback zu spät käme. Die statische Form genügt bei langsamen Störungen; die dynamische, wenn Störung und Strecke vergleichbar schnell sind.

7.7.2 Störentkopplung (Disturbance Decoupling)

Hier ist das Ziel strenger: die Reglerstruktur so wählen, dass die Störung den *geregelten Ausgang* **prinzipiell nicht mehr beeinflusst** – die Übertragungsfunktion von $\tau_{\text{stör}}$ auf $\mathbf{y}$ wird zu null. Das gelingt nur, wenn der Störangriff aus dem messbaren/stellbaren Raum heraus kompensierbar ist (die geometrische Entkopplungsbedingung). Die Störung darf intern noch wirken, erscheint aber nicht am Ausgang.

Wann einsetzen

Wenn eine bestimmte Störrichtung den Ausgang *strukturell* nicht erreichen darf und das System die Entkopplungsbedingung erfüllt. Schärfer als die bloße Ausschaltung, dafür an strukturelle Voraussetzungen gebunden.

Unterschied: Ausschaltung *kompensiert* eine konkrete Störung betragsmäßig; Entkopplung *eliminiert* ihren Einfluss auf den Ausgang per Struktur – unabhängig vom Störverlauf.

7.7.3 Dynamische Vorsteuerung – mit oder ohne Zustandsregelung

Ist die Störung *nicht* messbar, schätzt sie ein Beobachter (oft als **Disturbance Observer**, DOB, der $\hat{\tau}_{\text{stör}}$ aus dem Vergleich von Modell und Istverhalten rekonstruiert) und speist die Schätzung gegenphasig auf:

$$\tau = \tau_{\text{Regler}} - \hat{\tau}_{\text{stör}}. \quad (7.8)$$

Das ist eine *dynamische Vorsteuerung der Störung* aus dem Beobachter.

Zwei Ausbaustufen:

- **Ohne Zustandsregelung:** reine Aufschaltung der geschätzten Störung auf einen bestehenden (z. B. PD-) Regler. Einfach nachzurüsten, sehr wirksam gegen Reibung und langsame Lastwechsel.
- **Mit Zustandsregelung:** die Störung wird als *erweiterter Zustand* ins Modell aufgenommen (Störmodell), der Beobachter schätzt sie mit, und ein Zustandsregler $\mathbf{u} = -\mathbf{K}\hat{\mathbf{x}}$ verarbeitet sie konsistent mit den übrigen Zuständen. Das erlaubt frei platzierbare Dynamik des Gesamtsystems inklusive Störunterdrückung.

Voraussetzung und Grenze

Jede modellbasierte Störschätzung lebt vom Modell: Ist $\mathbf{M}, \mathbf{C}, \mathbf{g}$ falsch, interpretiert der DOB den Modellfehler als Störung und „kompensiert“ ihn ebenfalls – gewollt bei Robustheit, ungewollt, wenn die Bandbreite zu hoch gewählt ist und Messrauschen mit aufgeschaltet wird.

Unterschied zur messenden Ausschaltung: Statt die Störung zu *messen*, wird sie *geschätzt* – anwendbar auch dort, wo kein Sensor sie erfasst (Reibung!). Die Zustandsregelungs-Variante unterscheidet sich von der reinen Aufschaltung dadurch, dass Stör- und Regeldynamik *gemeinsam* ausgelegt werden, statt eine fertige Schleife nur zu ergänzen.

7.8 Überblick: Wann was?

Verfahren	Einsatz / Auslöser	Abgrenzung
Feedback (PD/PID)	Immer; langsame Bewegung, hohe Unterersetzung	Reaktiv, gleicht Unbekanntes aus, aber „hinterher“
Vorsteuerung	Bahn vorab bekannt, schnelle Folge nötig	Vorausschauend, modellbasiert, ohne Feedback nicht störfest
Kaskade	Schnelle Zwischengröße messbar (Strom, Drehzahl)	Struktur aus Schleifen, fängt innere Störungen früh ab
Feedback-Entkopplung	Schnelle Bewegung, spürbare Gelenkkopplung	Linearisiert Strecke <i>im</i> Kreis, arbeitspunktunabhängig
Arbeitsraum-Regelung	Aufgabe kartesisch, Kontakt/Kraft	Wie Entkopplung, aber in $\mathbf{x}$; richtungsabhängige Steifigkeit
Zustandsbeobachter	Zustand/Störung nicht direkt messbar	Liefert $\dot{\mathbf{q}}$, Störung etc.; KF = optimaler Beobachter
Störausschaltung	Störung messbar	Kompensiert Betrag (statisch/dynamisch)
Störentkopplung	Störrichtung darf Ausgang nicht erreichen	Eliminiert Einfluss strukturell, nicht nur betragsmäßig
Dyn. Vorst. (DOB)	Störung nicht messbar (Reibung, Last)	Schätzt statt misst; mit Zustandsregelung gemeinsam ausgelegt

Praxis-Fazit: Die Verfahren sind *Schichten*, keine Alternativen. Ein realer Roboterregler kombiniert sie: kaskadierte Antriebsregelung als Basis, Feedback-Entkopplung oder Arbeitsraumregelung für die Kopplung, Vorsteuerung für die bekannte Bahn, ein Beobachter für fehlende Zustände und ein Disturbance Observer für Reibung und Last. Jede Schicht nimmt dem darunterliegenden Feedback Arbeit ab – und nur das Feedback bleibt am Ende für das Unvorhergesehene zuständig.

Kapitel 8

Schrittmotoren und Drehmomentschätzung

8.1 NEMA-17-Grundlagen

Ein NEMA-17 ist ein bipolarer Hybridschrittmotor mit typisch $1,8^\circ/\text{Schritt} = 200$ Vollschritten/Umdrehung. Per **Microstepping** (z. B. 1/16, 1/256 beim TMC) wird sehr feinaufgelöst und vibrationsarm gefahren. Phasenströme typisch 0,4–2 A, Haltemoment je nach Baulänge $\approx 0,2$ –0,6 Nm. Im **offenen Regelkreis** kennt der Motor seine Last nicht – bei Überlast verliert er Schritte unbemerkt.

8.2 Indirekte Drehmomentschätzung

Statt eines teuren Drehmomentsensors wird das Lastmoment aus den *elektrischen* Größen geschätzt. Physikalischer Hintergrund: Bei einem belasteten Schrittmotor wächst der **Lastwinkel** (die Verschiebung zwischen Statorfeld und Rotor). Dieser zeigt sich in Phasenströmen und Gegen-EMK. Zwei praktische Umsetzungen:

1. **StallGuard (Trinamic):** Der Treiber misst intern die Gegen-EMK und liefert einen lastproportionalen Zahlenwert (SG_RESULT) über UART. Das ist der bevorzugte Ansatz – ein Proxy fürs Drehmoment ohne Zusatzhardware.
2. **Externe Strom-/Spannungsmessung:** Ein Sensormodul (z. B. INA226) misst Busspannung und -strom pro Treiber. Zusammen mit dem bekannten Microstep-Zustand lässt sich die mechanische Leistung und damit das Moment abschätzen.

Beide Wege sind weniger präzise als ein echter Sensor, aber günstig und praktikabel – siehe Teil II für die konkrete Hardware.

Kapitel 9

Zykloidgetriebe

Ein Zykloidgetriebe erzeugt auf kompaktem Bauraum eine hohe Übersetzung bei geringem Spiel (Backlash) und hoher Stoßfestigkeit – ideal als “letzte Stufe” vor dem Robotergelenk. Eine exzentrisch gelagerte Zykloidscheibe mit n Nocken wälzt in einem Ring mit N Bolzen ab. Die Übersetzung ist

$$i = \frac{N}{N - n}. \quad (9.1)$$

Mit einer Nockendifferenz von eins ($N - n = 1$) folgt $i = N$: 11 Ringbolzen und eine 10-Nocken-Scheibe ergeben 11:1.

Für die Simulation ist die innere Mechanik irrelevant – das Getriebe wird als *ein* angetriebenes Gelenk mit Übersetzung modelliert (siehe Teil IV, `ros2_control-Transmission`). Wichtig für die reale Auslegung: das übertragbare Ausgangsmoment, die Steifigkeit und das Restspiel, die du später in die Gelenklimits einträgst.

Kapitel 10

Maschinelles Lernen in ROS 2-Projekten

Kapitel 14 hat gezeigt, wie weit man in der Bildverarbeitung *ohne* Training kommt. Dieses Kapitel ergänzt die andere Seite: **wann** sich maschinelles Lernen (ML) in einem ROS 2-Roboter lohnt, **wie** man Trainings- und Inferenzcode sauber von den ROS-Knoten trennt und **womit** – scikit-learn, SciPy, TensorFlow, PyTorch – die gängigen Verfahren (Regression, Clustering, Bayes, Autoencoder, ANN, CNN, PINN) konkret angebunden werden. Der Anspruch ist nicht ein ML-Lehrbuch, sondern eine *Entscheidungs- und Integrationshilfe* für genau diese Roboterprojekte.

10.1 Wann ML – und wann nicht

ML ist kein Selbstzweck. Die teuerste Fehlentscheidung ist, ein neuronales Netz auf ein Problem zu werfen, das ein Kalman-Filter (Kap. 14 hinweg zu Kap. 2) oder eine geschlossene Formel exakter, schneller und nachvollziehbarer löst. Die nüchterne Faustregel:

Faustregel: klassisch vor lernend

Gibt es ein *physikalisches Modell* oder eine *geometrische Beziehung* (Kinematik, Projektion, Filtergleichung), nutze sie. Greife zu ML erst, wenn (a) das Modell unbekannt oder zu komplex ist, (b) genügend *repräsentative Daten* vorliegen und (c) du den Inferenzaufwand im Echtzeitbudget unterbringst.

Konkrete Anwendungsfälle in den Projekten dieses Buchs:

Aufgabe	Klassisch reicht	ML lohnt sich, weil ...
Ball-/Objekt-erkennung Flugbahn des Balls	Hintergrundsubtraktion, Farbschwelle, ORB ballistisches Modell + EKF	wechselnde Beleuchtung, Verdeckung, viele Objektklassen → CNN -Detektor Magnus-Effekt/Spin schwer zu modellieren → Regression/PINN als Korrektur
Inverse Kinematik	analytisch/numerisch (Kap. 5)	redundante/elastische Mechanik, schnelle Näherung → ANN
Punktwolken-Segmentierung	RANSAC-Ebene, Schwellwert	unstrukturierte Szene, mehrere Cluster → Clustering (DBSCAN)
Zustands/Anomalieüberwachung	feste Schwellen	“normal” ist mehrdimensional und schwer zu fixieren → Autoencoder

Sensorkalibrierung

Least-Squares (SciPy)

nichtlineare, unbekannte Kennlinie mit
Unsicherheit → **Gauß-Prozess**

10.2 Architektur: Training offline, Inferenz im Knoten

Die wichtigste Designentscheidung ist die **Trennung von Training und Inferenz**. Training ist datenintensiv, läuft batchweise auf einem Entwickler-PC (gern mit GPU) und produziert eine *Artefaktdatei* (Gewichte). Der ROS 2-Knoten lädt dieses Artefakt *einmal* beim Hochfahren und führt im Callback nur noch günstige Vorwärts-Inferenz aus.

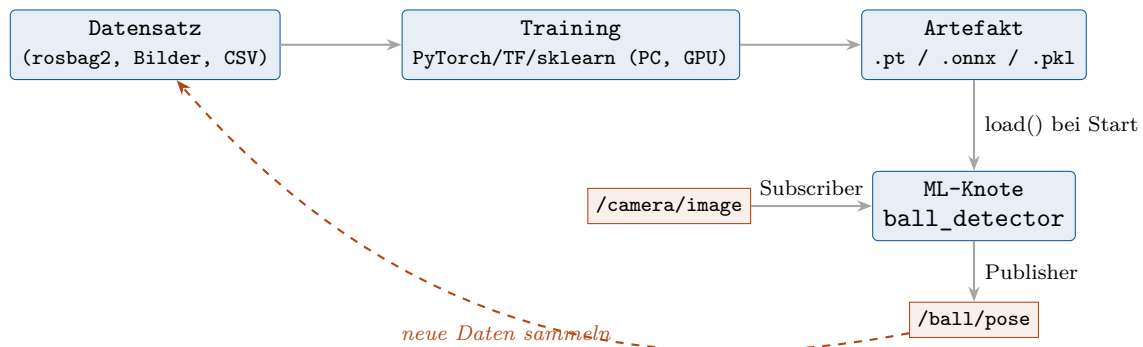


Abbildung 10.1: Offline-Training erzeugt ein Artefakt; der ROS 2-Knoten lädt es einmalig und betreibt nur noch Inferenz. Gesammelte Echtzeiten fließen ins nächste Training zurück.

Inferenz nie im Spin-Thread blockieren

Ein `model(x)`-Aufruf kann zehn bis hunderte Millisekunden dauern. Läuft er direkt im Subscriber-Callback des Standard-Executors, blockiert er alle anderen Callbacks. Lege Inferenz in eine **eigene Callback-Group** (`MutuallyExclusiveCallbackGroup`) mit `MultiThreadedExecutor` oder in einen separaten Worker-Thread und veröffentliche das Ergebnis asynchron (vgl. Kap. 14 zum Echtzeit-Scheduling).

```

import rclpy
from rclpy.node import Node
from rclpy.callback_groups import MutuallyExclusiveCallbackGroup
from sensor_msgs.msg import Image
from geometry_msgs.msg import PointStamped
from cv_bridge import CvBridge

class BallDetector(Node):
    def __init__(self):
        super().__init__('ball_detector')
        # Modell EINMALIG laden (Pfad als ROS-Parameter):
        self.declare_parameter('model_path', 'model.onnx')
        path = self.get_parameter('model_path').value
        self.model = self.load_model(path) # framework-spezifisch, s.u.
        self.bridge = CvBridge()
        cb = MutuallyExclusiveCallbackGroup()
        self.sub = self.create_subscription(
            Image, '/camera/image', self.on_image, 10, callback_group=cb)
        self.pub = self.create_publisher(PointStamped, '/ball/pose', 10)

    def on_image(self, msg):
        img = self.bridge.imgmsg_to_cv2(msg, 'rgb8')
        u, v, conf = self.infer(self.model, img) # reine Vorwaertsinferenz
        if conf < 0.5:
            return
        out = PointStamped()
        out.header = msg.header # gleiche Zeit/Frame!
        out.point.x, out.point.y = float(u), float(v)
        self.pub.publish(out)

```

Listing 10.1: Grundgerüst eines Inferenz-Knotens (framework-agnostisch)

Drei Regeln, die in jedem ML-Knoten gelten sollten:

- **Modellpfad als Parameter**, nie hartkodiert – so wechselt man Gewichte ohne Neukompilieren.
- **Header übernehmen** (stamp, frame_id): nur so passen Detektionen zeitlich zu /tf und lassen sich an einen EKF füttern.
- **Konfidenz mitschätzen** und schwache Detektionen verwerfen statt zu raten.

10.3 Die vier Bibliotheken und ihre Rolle

Bibliothek	Stärke	Rolle im ROS 2-Projekt
SciPy	numerische Mathematik	Optimierung (least_squares, curve_fit), Filter (signal), Interpolation, lineare Algebra – die Basis unter allem anderen, oft schon ausreichend.
scikit-learn	klassisches ML, “flach”	Regression, Clustering, PCA, SVM, Gauß-Prozesse, Vorverarbeitung (StandardScaler, Pipeline). CPU, Modelle als .pkl/joblib. Erste Wahl bei tabellarischen/kleinen Daten.
PyTorch	Deep Learning, Forschung	ANN, CNN, Autoencoder, PINN. Dynamischer Graph, torch.autograd (wichtig für PINN), Export nach .pt oder ONNX.

TensorFlow	Deep Learning, Deployment	wie PyTorch; stark in <code>tf.keras</code> und Edge-Deployment (TF-Lite auf Raspberry Pi/Jetson).
-------------------	---------------------------	--

Installation & Workspace

Die ML-Pakete gehören *nicht* in den ROS-Sourcetree. Üblich: ein Python-venv oder conda-Env, in dem sowohl die ROS-Python-API (über `rclpy`) als auch `torch/tensorflow` sichtbar sind. In `package.xml` deklariert man die Abhängigkeiten via `<exec_depend>python3-scikit-learn</exec_depend>` bzw. `rosdep`; große Frameworks (Torch/TF) installiert man per `pip` im Env und hält sie aus `rosdep` heraus. **ONNX Runtime** ist der pragmatische gemeinsame Nenner: in PyTorch/TF trainieren, nach `.onnx` exportieren, im Knoten nur `onnxruntime` laden – so muss der Roboter weder Torch noch TF mitschleppen.

10.4 Klassisches ML mit SciPy & scikit-learn

10.4.1 Regression – Kennlinien und Vorhersage

Regression schätzt eine kontinuierliche Größe. Zwei typische Einsätze: nichtlineare *Sensorkalibrierung* mit SciPy und eine lernende *Trajektorienkorrektur* mit scikit-learn.

```
import numpy as np
from scipy.optimize import curve_fit

# Modell: Distanz d aus Rohwert r eines IR-Sensors (1/r-Kennlinie)
def model(r, a, b, c):
    return a / (r + b) + c

popt, pcov = curve_fit(model, r_raw, d_true, p0=[1.0, 0.0, 0.0])
# popo -> in YAML speichern, im Knoten als feste Parameter laden:
d_pred = model(r_live, *popt)
```

Listing 10.2: Nichtlineare Kalibrierkurve mit SciPy `curve_fit`

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import Ridge
import joblib

# X: [Spin, Anstellwinkel, v0] -> y: Auftreffpunkt auf dem Tisch
reg = make_pipeline(StandardScaler(),
                    PolynomialFeatures(degree=2),
                    Ridge(alpha=1.0))
reg.fit(X_train, y_train)
joblib.dump(reg, 'landing_model.pkl') # Artefakt fuer den Knoten
```

Listing 10.3: Polynomiale Regression (sklearn-Pipeline)

10.4.2 Clustering – Punktwolken und Objekte gruppieren

Clustering ist *unüberwacht*: keine Labels nötig. DBSCAN ist für Robotik oft besser als KMeans, weil es die Clusterzahl nicht vorab braucht und Rauschen explizit als Ausreißer markiert – ideal, um aus einer Tiefenpunktwolke Objekte zu trennen.

```

from sklearn.cluster import DBSCAN
import numpy as np

# pts: Nx3-Array aus sensor_msgs/PointCloud2
labels = DBSCAN(eps=0.05, min_samples=20).fit_predict(pts)
for k in set(labels):
    if k == -1:          # -1 = Rauschen
        continue
    cluster = pts[labels == k]
    centroid = cluster.mean(axis=0)  # -> als Objektkandidat publizieren

```

Listing 10.4: DBSCAN-Segmentierung einer Punktwolke

10.4.3 Bayes & Gauß-Prozesse – Schätzen mit Unsicherheit

Der bayessche Blick liefert nicht nur einen Wert, sondern eine *Verteilung*. Genau das passt zu Robotik, wo der EKF (Kap. 2) bereits bayessch denkt. Ein **Gauß-Prozess** (GP) ist die elegante Form der Regression *mit* Konfidenzband: dort, wo wenig Daten liegen, wächst die Unsicherheit – ein direkt nutzbares Signal, um vorsichtig zu fahren.

```

from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, WhiteKernel

kernel = RBF(length_scale=1.0) + WhiteKernel(noise_level=1e-2)
gp = GaussianProcessRegressor(kernel=kernel, normalize_y=True)
gp.fit(X_train, y_train)

mu, sigma = gp.predict(X_query, return_std=True)
# sigma -> als Messrausch-Kovarianz direkt an den EKF weiterreichen

```

Listing 10.5: Gauß-Prozess-Regression mit Unsicherheit (sklearn)

Der **naive Bayes**-Klassifikator (`sklearn.naive_bayes`) bleibt die schnellste Baseline für diskrete Klassifikation (z. B. Zustand “Greifer voll/leer” aus wenigen Merkmalen) und eignet sich als Vergleichsmaßstab, bevor man zu neuronalen Netzen greift.

10.5 Neuronale Netze mit PyTorch

10.5.1 ANN (MLP) – gelernte inverse Kinematik

Ein mehrschichtiges Perzeptron (Multi-Layer Perceptron, MLP) approximiert beliebige glatte Funktionen. Sinnvoll, wenn die analytische inverse Kinematik mehrdeutig oder die reale Mechanik elastisch ist: das Netz liefert in konstanter Zeit eine gute Startnäherung, die ein numerischer Schritt verfeinert.

```

import torch, torch.nn as nn

class IKNet(nn.Module):
    def __init__(self, n_in=3, n_out=4):  # (x,y,z) -> 4 Gelenkwinkel
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_in, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_out))
    def forward(self, x):
        return self.net(x)

model = IKNet()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()
for xb, yb in loader:  # Training (offline, PC)
    opt.zero_grad()
    loss = loss_fn(model(xb), yb)
    loss.backward(); opt.step()
torch.save(model.state_dict(), 'ik_net.pt') # Artefakt

```

Listing 10.6: MLP fuer inverse Kinematik (PyTorch)

10.5.2 CNN – Objekt- und Ball-Erkennung

Faltungsnetze (Convolutional Neural Networks) sind der Standard, sobald aus *Bildern* gelernt wird. Praktisch trainiert man selten von Null, sondern nutzt **Transfer Learning**: ein vortrainiertes Backbone (z. B. ein schlankes YOLO/MobileNet) wird auf die eigenen wenigen Objektklassen feinjustiert. Im Knoten zählt nur die Inferenzlatenz – daher Export nach ONNX und Ausführung über `onnxruntime` (CPU/GPU/TensorRT).

```

import onnxruntime as ort
import numpy as np

class CNNDetector:
    def __init__(self, path):
        self.ssess = ort.InferenceSession(
            path, providers=['CUDAExecutionProvider', 'CPUExecutionProvider'])
        self.inp = self.ssess.get_inputs()[0].name

    def infer(self, img_rgb):  # img: HxWx3, uint8
        x = img_rgb.astype(np.float32).transpose(2, 0, 1)[None] / 255.0
        boxes, scores, classes = self.ssess.run(None, {self.inp: x})
        return boxes, scores, classes

```

Listing 10.7: CNN-Inferenz im Knoten via ONNX Runtime

10.5.3 Autoencoder – Anomalie- und Zustandsüberwachung

Ein Autoencoder lernt, seine Eingabe komprimiert zu rekonstruieren. Trainiert *nur auf Normalbetrieb*, steigt der Rekonstruktionsfehler bei Anomalien – ein label-freier Wächter für Vibrationen, Stromaufnahme oder Gelenkmoment, der Defekte meldet, bevor feste Schwellen anschlagen.

```

import torch, torch.nn as nn

class AE(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(d, 16), nn.ReLU(), nn.Linear(16, 4))
        self.dec = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, d))
    def forward(self, x):
        return self.dec(self.enc(x))

# Im Knoten: Fehler gegen Schwelle aus dem Training (z.B. 99%-Quantil)
def is_anomaly(model, x, thresh):
    with torch.no_grad():
        err = ((model(x) - x) ** 2).mean().item()
    return err > thresh, err

```

Listing 10.8: Autoencoder zur Anomalieerkennung (PyTorch)

10.5.4 PINN – physik-informierte Netze

Ein **Physics-Informed Neural Network** (PINN) verbindet beide Welten: Die Verlustfunktion bestraft nicht nur die Datenabweichung, sondern auch die Verletzung einer bekannten *Differentialgleichung*. Für die Ballflugbahn mit Luftwiderstand und Magnus-Effekt heißt das: Das Netz $\mathbf{x}(t)$ wird so trainiert, dass es zu den Messpunkten passt *und* die Bewegungsgleichung $m\ddot{\mathbf{x}} = \mathbf{F}_g + \mathbf{F}_{\text{drag}} + \mathbf{F}_{\text{magnus}}$ erfüllt. Vorteil: physikalisch plausible Vorhersagen auch mit wenigen Daten. Der Schlüssel ist `torch.autograd.grad`, das die Zeitableitungen $\dot{\mathbf{x}}, \ddot{\mathbf{x}}$ exakt aus dem Netz liefert.

```

import torch

def physics_residual(net, t, m, g):
    t = t.requires_grad_(True)
    x = net(t) # Position(t), z.B. Hoehe
    v = torch.autograd.grad(x, t, torch.ones_like(x), create_graph=True)[0]
    a = torch.autograd.grad(v, t, torch.ones_like(v), create_graph=True)[0]
    # ODE-Residuum: m*a + drag(v) - (-m*g) = 0 -> hier vereinfacht ohne drag
    return m * a + m * g

# Gesamtverlust = Daten-MSE + lambda * mean(residual^2)
loss = mse(net(t_data), x_data) + lam * (physics_residual(net, t_col, m, g)**2).mean()

```

Listing 10.9: PINN-Residuum fuer eine Bewegungsgleichung (PyTorch)

10.6 Verfahren-Wegweiser

Verfahren	Typ	Werkzeug	Wofür im Roboter
Least-Squares / <code>curve_fit</code>	Fit	SciPy	Kalibrierung, Modellabgleich
Lineare/Ridge- Regression	überwacht	scikit-learn	einfache Vorhersage, Baseline
Gauß-Prozess	überwacht, bayessch	scikit-learn	Regression <i>mit</i> Unsicherheit für den EKF
Naive Bayes / SVM	überwacht	scikit-learn	schnelle Klassifikation weniger Merkmale

KMeans	/	unüberwacht	scikit-learn	Punktwolken- /Objektsegmentierung
DBSCAN				
PCA		unüberwacht	scikit-learn	Dimensionsreduktion, Merkmal- sanalyse
MLP (ANN)		überwacht, tief	PyTorch/TF	gelernte IK, nichtlineare Rege- lung
CNN		überwacht, tief	PyTorch/TF ONNX	+ Bild-/Objekt-/Ballerkennung
Autoencoder		unüberwacht, tief	PyTorch/TF	Anomalie- /Zustandsüberwachung
PINN		hybrid	PyTorch	Trajektorie mit Physik- Nebenbedingung

Kernbotschaft

Maschinelles Lernen ist in ROS 2 kein Bruch, sondern ein weiterer Knoten: *offline lernen, ein Artefakt exportieren, im Lifecycle-Knoten laden, Inferenz aus dem kritischen Pfad heraushalten und Ergebnisse mit korrektem Header publizieren*. Beginne mit SciPy/scikit-learn; greife zu PyTorch/TF erst, wenn Bilder, hohe Dimensionen oder Physik-Nebenbedingungen es erzwingen.

Teil II

Stereokamera in ROS 2 – Praxis mit Code

Kapitel 11

Kamera anbinden, Frame splitten

Die synchronisierte ELP-Kamera liefert ein side-by-side-Bild. Erst den Treiber starten, dann links/rechts trennen.

```
ros2 run v4l2_camera v4l2_camera_node \
  --ros-args -p video_device:=/dev/video0 \
  -p image_size:="[2560,720]"
ros2 topic list                # /image_raw sollte erscheinen
ros2 run rqt_image_view rqt_image_view  # Bild ansehen
```

Listing 11.1: Treiber starten und Topics prüfen

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge

class StereoSplitter(Node):
    def __init__(self):
        super().__init__('stereo_splitter')
        self.bridge = CvBridge()
        self.pub_l = self.create_publisher(Image, '/left/image_raw', 10)
        self.pub_r = self.create_publisher(Image, '/right/image_raw', 10)
        self.create_subscription(Image, '/image_raw', self.cb, 10)

    def cb(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
        w = frame.shape[1] // 2
        left, right = frame[:, :w], frame[:, w:]
        hdr = msg.header # keep timestamp so stereo stays synchronized
        lm = self.bridge.cv2_to_imgmsg(left, 'bgr8'); lm.header = hdr
        rm = self.bridge.cv2_to_imgmsg(right, 'bgr8'); rm.header = hdr
        self.pub_l.publish(lm)
        self.pub_r.publish(rm)

def main():
    rclpy.init()
    rclpy.spin(StereoSplitter())
    rclpy.shutdown()
```

Listing 11.2: Knoten: side-by-side Frame in /left und /right splitten

Kapitel 12

Kalibrieren und Tiefe erzeugen

12.1 Warum kalibrieren? Das Kameramodell

Die Kalibrierung bestimmt die **Intrinsik** (Brennweiten f_x, f_y , Hauptpunkt c_x, c_y , zusammengefasst in $\mathbf{K}$), die **Verzeichnungskoeffizienten** (radial k_1, k_2, k_3 und tangential p_1, p_2) und bei Stereo zusätzlich die **Extrinsik** – die starre Transformation $(\mathbf{R}, \mathbf{t})$ zwischen linker und rechter Linse. Erst diese Parameter machen aus Pixeln metrische Strahlen: Sie sind die Voraussetzung für Rektifizierung, Disparität und damit jede Tiefenmessung (vgl. Kapitel 4).

Das Verfahren (Zhang-Methode, wie in OpenCV implementiert [5]): Ein *Schachbrettmuster* bekannter Geometrie wird in vielen Lagen aufgenommen. Da die 3D-Koordinaten der Ecken im Brett-System bekannt sind und ihre 2D-Pixelpositionen detektiert werden (`cv2.findChessboardCorners`, Abb. 12.1), entsteht ein großes System von 2D-3D-Korrespondenzen. Daraus löst ein Optimierer (Minimierung des Reprojektionsfehlers über alle Bilder und Ecken) Intrinsik und Verzeichnung. Faustregeln:

- **Viele Ansichten** (15–30), das Brett in unterschiedlichen Abständen, Winkeln und Bildregionen – gekippte Ansichten sind nötig, um die Brennweite von der Distanz zu trennen.
- **Reprojektionsfehler** als Gütemaß: deutlich unter 1 px ist gut.
- Bei Stereo wird nach den beiden Einzelkalibrierungen die Extrinsik geschätzt; schon kleine Fehler darin skalieren systematisch in die Objektgenauigkeit (Maßstab, absolute Position) hinein [6].

12.2 Kalibrierung und Tiefenpipeline in ROS 2

```
# Kalibrieren: erzeugt left.yaml / right.yaml
#               mit Intrinsik + Extrinsik
ros2 run camera_calibration cameracalibrator \
  --size 8x6 --square 0.025 \
  --ros-args -r left:=/left/image_raw \
             -r right:=/right/image_raw

# Rektifizierung + Disparitaet -> Tiefe / Punktwolke
ros2 launch stereo_image_proc stereo_image_proc.launch.py
# liefert /disparity und /points2
```

Listing 12.1: Stereokalibrierung (Schachbrett) und Tiefenpipeline

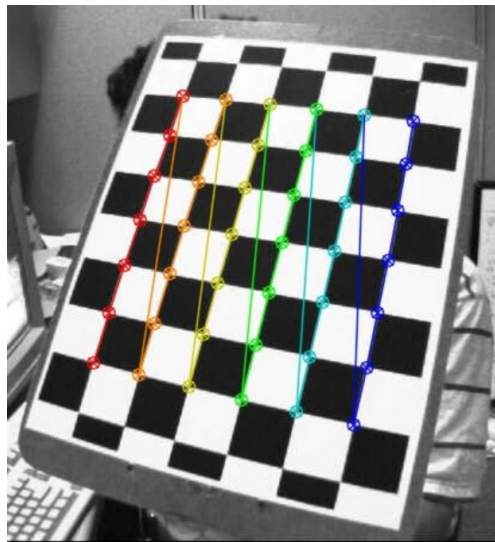


Abbildung 12.1: Detektiertes Schachbrettmuster während der Kalibrierung: Die farbig markierten, in fester Reihenfolge verbundenen Innenecken liefern die 2D-Bildpunkte, deren 3D-Lage im Brett-System bekannt ist. Aus vielen solchen Ansichten schätzt der Optimierer Intrinsik, Verzeichnung und (bei Stereo) die Extrinsik. Nach [5].

Kapitel 13

Kalman-Filter-Knoten (Ballverfolgung)

```
import numpy as np, rclpy
from rclpy.node import Node
from geometry_msgs.msg import PointStamped

class BallTracker(Node):
    def __init__(self):
        super().__init__('ball_tracker')
        dt = 1/60.0
        self.x = np.zeros((6,1))          # [x y z vx vy vz]
        self.P = np.eye(6) * 1.0
        self.F = np.eye(6); self.F[:3,3:] = np.eye(3)*dt
        self.g = np.array([[0],[0],[-9.81]])
        self.Bu = np.vstack([0.5*dt**2*self.g, dt*self.g])
        self.H = np.zeros((3,6)); self.H[:, :3] = np.eye(3)
        self.Q = np.eye(6)*0.01
        self.R = np.eye(3)*0.0025          # ~5 cm Messrauschen
        self.sub = self.create_subscription(
            PointStamped, '/ball/position', self.update, 10)
        self.timer = self.create_timer(dt, self.predict)

    def predict(self):
        self.x = self.F @ self.x + self.Bu
        self.P = self.F @ self.P @ self.F.T + self.Q

    def update(self, msg):
        z = np.array([[msg.point.x],[msg.point.y],[msg.point.z]])
        S = self.H @ self.P @ self.H.T + self.R
        K = self.P @ self.H.T @ np.linalg.inv(S)    # Kalman-Gain
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(6) - K @ self.H) @ self.P

    def predict_intercept(self, z_plane):
        # reine Praediktion bis zur Schlagebene (kein Messupdate)
        x = self.x.copy()
        for _ in range(120):
            x = self.F @ x + self.Bu
            if x[2,0] <= z_plane:
                return x[:3].flatten()
        return None
```

Listing 13.1: EKF/KF-Knoten: 3D-Position rein, geglättete Schätzung + Vorhersage raus

Kapitel 14

Merkmalerkennung ohne KI-Training

14.1 Detektoren im Vergleich: SIFT, SURF, KAZE, AKAZE, ORB, BRISK

Bevor wir Code schreiben, lohnt der nüchterne Vergleich der klassischen Detektor-Deskriptor-Paare. Sie durchlaufen alle dieselben Stufen – Merkmalsdetektion und -beschreibung, Matching, Ausreißerverwerfung (Lowe-Ratio + RANSAC) und Schätzung der Transformation – unterscheiden sich aber stark in Genauigkeit und Geschwindigkeit [7]:

- **SIFT** und **KAZE/AKAZE** liefern die *genauesten*, invariantesten Matches (Skala, Rotation, Blickwinkel), sind aber rechenintensiver.
- **ORB** und **BRISK** verwenden *binäre* Deskriptoren mit Hamming-Matching und sind dadurch am *schnellsten* und sparsamsten – ideal für Echtzeit auf schwacher Hardware, zum Preis etwas geringerer Robustheit. ORB findet zudem typischerweise die meisten Merkmale pro Bild.

Faustregel für dieses Projekt: ORB als Standardwahl für Echtzeit-Matching und visuelle Odometrie, SIFT/AKAZE, wenn es bei der CAD-Posenschätzung auf maximale Genauigkeit ankommt. Der folgende Code zeigt die ORB-Variante exemplarisch.

```
import cv2

orb = cv2.ORB_create(nfeatures=1500)
# Referenzansicht des Bauteils (z.B. aus CAD-Render) einmalig beschreiben:
kp_ref, des_ref = orb.detectAndCompute(ref_gray, None)
matcher = cv2.BFMatcher(cv2.NORM_HAMMING)

def detect(frame_gray):
    kp, des = orb.detectAndCompute(frame_gray, None)
    if des is None:
        return []
    raw = matcher.knnMatch(des_ref, des, k=2)
    good = [m for m, n in raw if m.distance < 0.75 * n.distance] # Lowe
    return kp, good

# 6D-Pose ueber 2D-3D-Korrespondenzen (CAD-Punkte <-> Bildpunkte):
# ok, rvec, tvec = cv2.solvePnP(Ransac(obj_pts_3d, img_pts_2d,
#                                     K, distCoeffs))
```

Listing 14.1: ORB-Erkennung und Matching (klassisch, kein Training nötig)

Für **Bewegungserkennung** (Projekt 3) genügt Hintergrundsubtraktion:

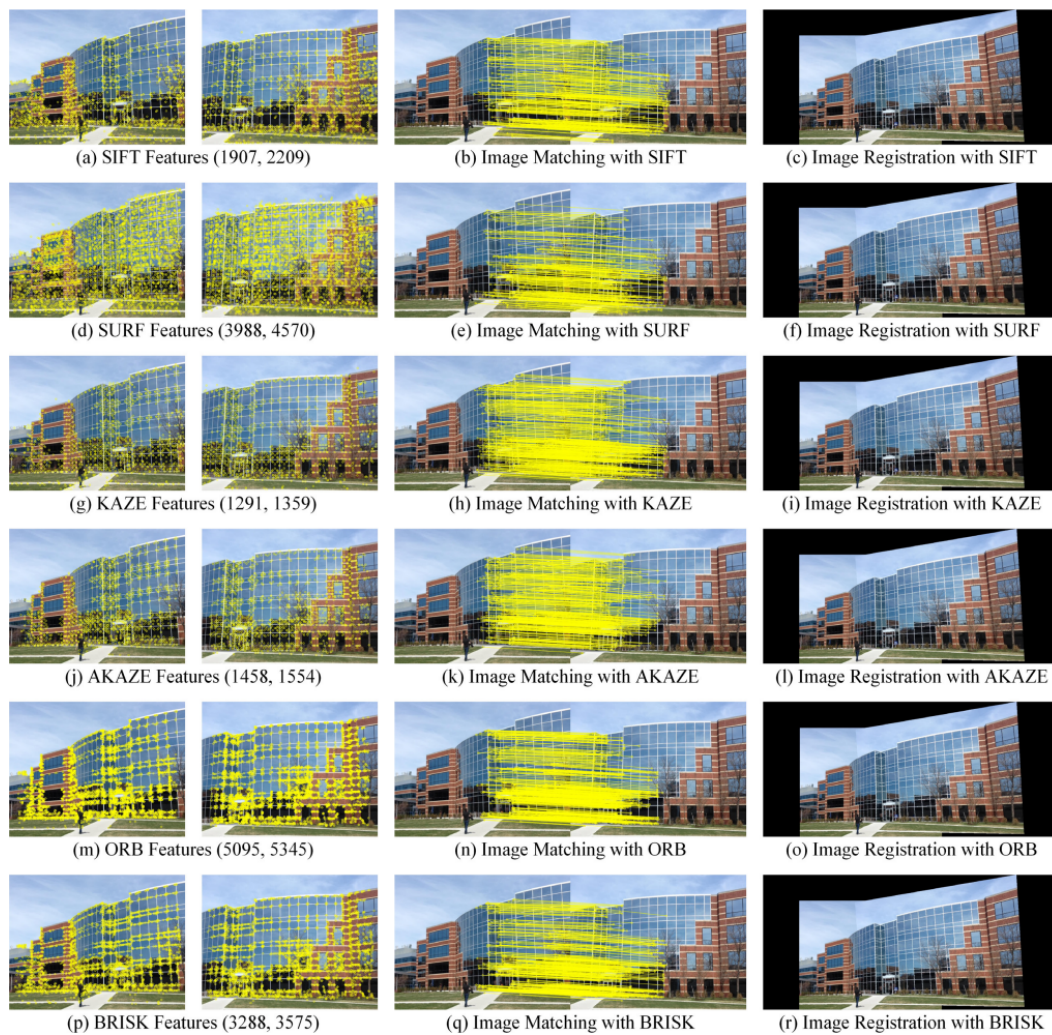


Fig. 4. Feature-detection, matching, and mosaicing with SIFT, SURF, KAZE, AKAZE, ORB, and BRISK.

Abbildung 14.1: Merkmalsdetektion, Matching und Bildregistrierung derselben Szene mit SIFT, SURF, KAZE, AKAZE, ORB und BRISK (Zeilen). Die Detektoren unterscheiden sich deutlich in Anzahl und Verteilung der gefundenen Merkmale sowie in der Matching-Qualität; SIFT/AKAZE gelten als am genauesten, ORB/BRISK als am effizientesten. Nach [7].

```
bg = cv2.createBackgroundSubtractorMOG2(detectShadows=False)
mask = bg.apply(frame)                                # bewegte Pixel
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
movers = [c for c in contours if cv2.contourArea(c) > 500]
```

Listing 14.2: Bewegungserkennung ohne Merkmale

Kapitel 15

Echtzeit-Stereoverarbeitung: Scheduling, Frame-Dropping und Zustandsautomat

Der Balltracker ist **zeitkritisch**: Bei 60 fps hast du pro Frame nur $\approx 16,7$ ms, um zu splitten, zu rektifizieren, den Ball in beiden Bildern zu finden, zu triangulieren und den Kalman-Filter zu aktualisieren. Dauert ein Frame länger, läuft die Verarbeitung der Realität hinterher. Dieses Kapitel zeigt, wie man damit umgeht: Frames gezielt verwerfen, die Prädiktion von der Messung entkoppeln und die Ablauflogik als Zustandsautomat strukturieren.

15.1 Das Zeitbudget eines Frames

Die Pipeline besteht aus Stufen mit je eigenem Latenzanteil:

Stufe	typ. Latenz	Anmerkung
Belichtung + USB-Transfer	2–8 ms	von der Kamera vorgegeben
Split + Rektifizierung	1–3 ms	einmalige Remap-LUT
Balldetektion (sparse, beide Bilder)	2–10 ms	der teure, schwankende Teil
Triangulation + KF-Update	< 1 ms	billig

Es gibt **zwei verschiedene Fehlermodi**, die man nicht verwechseln darf:

Latenz (zu alt).

Ein einzelner Frame ist verarbeitet, aber das Ergebnis kommt zu spät – die Schätzung beschreibt einen vergangenen Ballort.

Durchsatz (Stau).

Frames kommen schneller, als du sie verarbeitest. Ohne Gegenmaßnahme wächst die Warteschlange unbegrenzt, und die Latenz steigt mit jeder Sekunde weiter an (*Bufferbloat*).

Die Kernregel für den Kalman-Filter

Ein **veralteter** Messwert ist schlechter als *gar kein* Messwert. Der Filter kann über reine Prädiktion problemlos einige Frames überbrücken (vgl. Kalman-Knoten). Also: lieber den alten Frame **verwerfen** und auf den nächsten aktuellen warten, als einen veralteten Messwert ins Update zu geben. **Niemals Frames aufstauen.**

15.2 Frame-Dropping: DDS macht die Arbeit

Der wichtigste Hebel ist die **QoS**: Mit `KEEP_LAST`, Tiefe 1 und `BEST_EFFORT` verwirft DDS selbst alte Bilder – während dein Callback rechnet, überschreibt jeder neue Frame den wartenden, und du bekommst nach dem Callback stets den *neuesten* statt des nächst-ältesten. Genau das ist das `sensor_data`-QoS-Profil.

```
from rclpy.qos import qos_profile_sensor_data
# entspricht: BEST_EFFORT + KEEP_LAST + depth=1
self.create_subscription(
    Image, '/left/image_raw', self.on_frame, qos_profile_sensor_data,
    callback_group=self.cb_detect)
```

Listing 15.1: QoS für Bild-Topics: nur der neueste Frame zählt

Warum nicht RELIABLE? `RELIABLE` lässt DDS verlorene Frames neu senden – bei einem Hochfrequenz-Bildstrom erzeugt das genau den Rückstau, den du vermeiden willst. `RELIABLE` gehört auf das *Ergebnis* (die 3D-Ballposition), `BEST_EFFORT` auf den Bildstrom.

Als zweite Verteidigungslinie verwirfst du im Callback explizit zu alte Frames anhand des Zeitstempels – das fängt auch den Fall ab, dass DDS dir doch einen leicht veralteten Frame zustellt:

```
def on_frame(self, msg):
    now = self.get_clock().now()
    stamp = rclpy.time.Time.from_msg(msg.header.stamp)
    age_s = (now - stamp).nanoseconds * 1e-9
    if age_s > self.max_age_s:          # z. B. 0.020 s = ein Frame bei 60 fps
        self.dropped += 1
        return                          # verwerfen statt verspätet verarbeiten
    self.process(msg)                  # detektieren -> triangulieren -> KF-Update
```

Listing 15.2: Stale-Gating: Frame anhand seines Alters verwerfen

15.3 Scheduling: Executor und Callback-Gruppen

Standardmäßig serialisiert der `SingleThreadedExecutor` alle Callbacks: ein langsamer Detektions-Callback **blockiert dann auch den Prädiktions-Timer**. Für zeitkritischen Code trennst du die Anliegen über einen `MultiThreadedExecutor` und **Callback-Gruppen**:

- `MutuallyExclusiveCallbackGroup`: Callbacks dieser Gruppe laufen nie gleichzeitig – richtig für den CPU-gebundenen Detektor (eine Instanz zur Zeit, zusammen mit Tiefe 1 ergibt das automatische Frame-Dropping).
- Ein *eigener* Timer-Callback in einer *eigenen* Gruppe für die Prädiktion – so läuft sie in festem Takt weiter, selbst während die Detektion noch an einem Frame rechnet.


```

from rclpy.executors import MultiThreadedExecutor
from rclpy.callback_groups import MutuallyExclusiveCallbackGroup

class Perception(Node):
    def __init__(self):
        super().__init__('perception')
        self.cb_detect = MutuallyExclusiveCallbackGroup() # teuer, schwankend
        self.cb_predict = MutuallyExclusiveCallbackGroup() # billig, deterministisch
        self.create_subscription(Image, '/left/image_raw', self.on_frame,
                                qos_profile_sensor_data, callback_group=self.cb_detect)
        self.create_timer(1/120.0, self.predict, callback_group=self.cb_predict)

def main():
    rclpy.init()
    node = Perception()
    ex = MultiThreadedExecutor(num_threads=3) # detect | predict | reserve
    ex.add_node(node); ex.spin()

```

Listing 15.3: Prädiktion und Detektion in getrennten Gruppen, parallel ausgeführt

Prädiktion schneller als die Kamera. Der Predict-Timer darf ruhig mit 120 Hz laufen, obwohl die Kamera nur 60 fps liefert – Prädiktion ist billig und liefert zwischen den Messungen eine glatte, aktuelle Schätzung. Die Messung (Update) kommt ereignisgesteuert, wann immer ein *frischer* Frame durchkommt. Das ist genau die Predict/Update-Aufteilung aus dem Kalman-Knoten, nur sauber auf zwei Threads gelegt.

Veraltete und vertauschte Messungen (Out-of-Order)

Kommt eine Messung mit einem Zeitstempel *vor* dem aktuellen Filterstand an (z.B. weil ein langsamer Detektions-Thread sie spät abliefert), sind zwei Wege sauber: (a) sie schlicht **verwerfen** (einfach, meist ausreichend), oder (b) ein **Out-of-Sequence-Update** – Filter auf den Mess-Zeitpunkt zurückrollen, einarbeiten, wieder vorrollen. Für den Tischtennisball reicht (a). Hilfreich, wenn linkes/rechtes Bild synchron sein müssen: `message_filters.ApproximateTimeSynchronizer`.

15.4 Ablauflogik als Zustandsautomat

Die Frage “was tue ich pro Frame” hängt vom **Zustand** ab: Solange kein Ball da ist, suchst du; sobald genug Punkte einer Flugbahn vorliegen, rechnest du den Auftreffpunkt; ist der Schlag ausgelöst, fährst du ihn ab. Diese Logik gehört nicht verstreut in den Bild-Callback, sondern in einen expliziten **Zustandsautomaten**, der die Wahrnehmungs-Rate von der Entscheidungs-Rate entkoppelt.

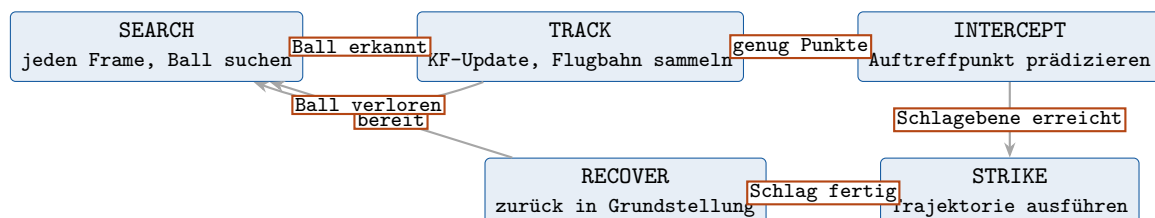


Abbildung 15.1: Zustandsautomat des Balltrackers. Die Stereo-Detektion treibt die Übergänge; die Prädiktion läuft zustandsübergreifend im eigenen Timer weiter.

Umsetzung in ROS 2 – drei Stufen nach Komplexität:

Eigenes Enum im Knoten.

Für eine überschaubare Logik wie diese das Pragmatischste: ein Zustandsfeld plus `match/if`.

Kein Framework, voll unter Kontrolle.

YASMIN.

ROS 2-native Zustandsmaschinen-Bibliothek (Python/C++) mit Introspektion – sinnvoll, wenn die Zustände wachsen.

Behavior Trees.

BehaviorTree.CPP (wie in Nav2) oder `py_trees_ros` – für komplexe, wiederverwendbare Verhaltensbäume mit Fallbacks; für den reinen Balltracker meist überdimensioniert.

```
from enum import Enum, auto

class S(Enum):
    SEARCH = auto(); TRACK = auto(); INTERCEPT = auto(); STRIKE = auto(); RECOVER = auto()

class Brain(Node):
    def __init__(self, kf):
        super().__init__('brain')
        self.kf = kf; self.state = S.SEARCH; self.hits = 0
        # eigener Takt: Entscheidung entkoppelt von der Frame-Rate
        self.create_timer(1/200.0, self.tick)

    def on_measurement(self, ok):          # vom Detektor pro frischem Frame
        self.hits = self.hits + 1 if ok else 0

    def tick(self):
        if self.state is S.SEARCH and self.hits > 0:
            self.state = S.TRACK
        elif self.state is S.TRACK:
            if self.hits == 0:                self.state = S.SEARCH    # Ball verloren
            elif self.hits >= self.min_track: self.state = S.INTERCEPT
        elif self.state is S.INTERCEPT:
            p = self.kf.predict_intercept(self.z_plane)    # reine Praediktion
            if p is not None:
                self.send_goal(p); self.state = S.STRIKE
        elif self.state is S.STRIKE and self.motion_done():
            self.state = S.RECOVER
        elif self.state is S.RECOVER and self.at_home():
            self.hits = 0; self.state = S.SEARCH
```

Listing 15.4: Zustandsautomat als Enum – Entscheidung getrennt vom Bild-Callback

Das Architektur-Prinzip in einem Satz

Drei entkoppelte Takte statt einer verschachtelten Schleife: **Detektion** (ereignisgesteuert, verwirft alte Frames per QoS), **Prädiktion** (schneller Timer, läuft immer), **Entscheidung** (eigener Timer, fährt den Zustandsautomaten) – über einen `MultiThreadedExecutor` mit getrennten Callback-Gruppen, damit kein langsamer Frame die anderen beiden blockiert.

Teil III

Hardware-Auswahl

Kapitel 16

Komponenten im günstigen Hobby-Bereich

Leitprinzip

Fokus liegt nicht auf Elektrotechnik: Wo möglich Module und Plug-and-Play, niedrige Kosten. Die Architektur trennt sauber eine **Echtzeit-Motorebene** (Mikrocontroller, nah an den Treibern) von einer **High-Level-Ebene** (PC/Pi mit vollem ROS 2 für Trajektorie, Vision, Kalman).

16.1 Schrittmotor und Treiber

Komponente	Empfehlung	Begründung
Motor	NEMA-17 (z. B. 17HS19)	Standard, günstig, ausreichend Moment mit Getriebe
Treiber	TMC2209 (UART)	StallGuard4 liefert Lastwert über UART = indirekte Drehmomentschätzung; leise, Stepstick-Format, $\approx 5\text{--}10\text{€}$
Treiber-Alt. Strom/Spannung	TMC2130/5160 (SPI) INA226 -Modul (I ² C)	höherer Strom / SPI statt UART misst Busspannung + Strom direkt, plug-and-play, $\approx 2\text{€}$ pro Kanal
Closed-Loop (opt.)	MKS SERVO42 / BTT S42B	integrierter Encoder + Treiber, falls echter Regelkreis gewünscht

Konkret für den Regelungsansatz: Der TMC2209 im UART-Modus gibt `SG_RESULT` (Last-proxy) ohne Zusatzhardware. Willst du *zusätzlich* Busspannung und -strom explizit messen, setzt du je Achse ein INA226-Modul auf die Treiberversorgung. Beide hängen am selben I²C/UART-Bus des Mikrocontrollers.

16.2 Recheneinheiten

ESP32 (Motorebene)

Dual-Core, viele PWM/GPIO, FreeRTOS, harte Echtzeit für Schrittgenerierung und das Auslesen von TMC (UART) und INA226 (I²C). Entscheidend: **micro-ROS** bindet den ESP32 als vollwertigen ROS 2-Knoten in den Graphen ein.

Raspberry Pi (High-Level)

Pi 4/5 mit Ubuntu kann den vollen ROS 2-Stack tragen (Trajektorienplanung, Kalman). Der **Pi**

Zero ist dafür zu schwach – für Vision ungeeignet; nutze ihn höchstens als micro-ROS-Brücke. Vision und schweres Rechnen gehören auf den externen PC.

Empfohlene Arbeitsteilung

Externer PC (Ubuntu + ROS 2): Trajektorienplanung, Stereo-Vision, Kalman-Filter, Visualisierung. $\longleftrightarrow$ (WLAN/USB) $\longleftrightarrow$ **ESP32** (micro-ROS): empfängt Sollwerte, generiert Schritte, liest StallGuard/INA226, publiziert Gelenkzustände. Die Stereokamera hängt per USB am PC.

16.3 Stereokamera

Das ELP-Modell ist eine synchronisierte Doppelobjektiv-USB-Kamera (UVC-konform). Sie meldet sich unter Linux als V4L2-Gerät (`/dev/video*`) und liefert in der Regel ein *nebeneinanderliegendes* (side-by-side) Bild über eine USB-Verbindung – die Synchronisierung beider Sensoren ist der Vorteil gegenüber zwei Einzelkameras (wichtig für bewegte Objekte wie den Ball). In ROS 2:

- Treiber: `v4l2_camera` oder `usb_cam`.
- Side-by-side-Frame in linkes/rechtes Bild splitten (kleiner eigener Knoten, siehe Teil VI).
- Eigene Stereokalibrierung ist Pflicht – das Modul ist nicht vorkalibriert.

Kapitel 17

Optimierungspotenzial für ein eigenes PCB

Diese Punkte sind *Erweiterung*, nicht Voraussetzung – mit Modulen kommst du zunächst aus. Für ein späteres eigenes Board:

- **Treiber integrieren** statt Stepstick-Module: TMC2209 direkt bestücken – spart Bauhöhe, bessere Thermik über Kupferflächen und Thermovias unter dem Chip.
- **Strom-/Spannungsmessung onboard:** INA-Sensor mit dimensioniertem Shunt je Kanal nahe am Treiber, kurze Kelvin-Anbindung des Shunts.
- **Power-Integrity:** Bulk-Elko plus keramische Stützkondensatoren direkt an jedem Treiber; getrennte Power-/Logik-Masse mit Sternpunkt (Single-Point-Ground).
- **Signalführung:** STEP/DIR bzw. UART als kurze, von der Leistung getrennte Leitungen; bei gemischten Pegeln (3,3 V/5 V) Levelshifter vorsehen.
- **Schutz:** ESD-Dioden an USB- und Encoder-Leitungen; Verpolschutz; Freilauf-/EMV-Betrachtung der Motorleitungen.
- **Konnektivität:** JST-Stecker für Motoren/Encoder, Schraubklemme für die Versorgung; optional CAN-Transceiver für saubere Mehrachs-Verkettung statt vieler UARTs.
- **Onboard-Encoder-Interface** für späteren Closed-Loop-Betrieb (FOC).
- **Thermik:** Thermopad/Kühlfläche unter den Treibern, definierter Luftstrom.

Teil IV

Software-Setup: VM, Linux, ROS 2

Kapitel 18

Betriebssystem und Distribution

18.1 Distributionswahl (Stand Juni 2026)

ROS 2-Releases erscheinen jährlich; relevant sind die LTS-Versionen. Für ein langfristiges Lernprojekt:

Distribution	Ubuntu	Support bis	Eignung
Jazzy Jalisco	24.04	2029 (LTS)	Empfehlung: reif, viele Tutorials, von Isaac Sim unterstützt
Humble Hawksbill	22.04	2027 (LTS)	meiste Tutorials, etwas älter
Kilted Kaiju	24.04	~2026	non-LTS, kurzer Support – meiden
Lyrical Luth	24.04	2031 (LTS)	brandneu (Mai 2026), Ökosystem noch nicht nachgezogen

Empfehlung

Ubuntu 24.04 LTS + ROS 2 Jazzy Jalisco. LTS bis 2029, ausgereifte Tutorials, von Gazebo Harmonic und NVIDIA Isaac Sim unterstützt. Lyrical Luth ist erst ~10 Tage alt – für den Einstieg zu frisch.

18.2 VM, Dual-Boot oder WSL2?

Wichtige Einschränkung bei der VM

Eine VM (VirtualBox/VMware) ist für die *ROS 2-Grundlagen* in Ordnung, aber **Gazebo Harmonic und Isaac Sim brauchen GPU-Beschleunigung**, die in VMs schlecht durchgereicht wird – 3D-Simulation läuft dort zäh. Optionen, nach Eignung:

1. **Natives Ubuntu (Dual-Boot)** – beste Leistung, empfohlen sobald du simulierst.
2. **WSL2 unter Windows** – inzwischen mit GPU-Zugriff (WSLg), guter Kompromiss.
3. **VM** – nur für CLI/Tutorials; für schwere Simulation ungeeignet.

Kapitel 19

ROS 2 installieren – Schritt für Schritt

```
# 1) Locale auf UTF-8
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8

# 2) Universe-Repo + Schluesel + ROS2-Quelle
sudo apt install -y software-properties-common curl
sudo add-apt-repository universe
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) \
  signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] \
  http://packages.ros.org/ros2/ubuntu \
  $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
sudo apt update
```

Listing 19.1: Locale + ROS 2-Apt-Quelle (Ubuntu 24.04)

```
# 3) ROS2 Desktop (inkl. RViz2, Demos)
sudo apt install -y ros-jazzy-desktop

# 4) Build-Werkzeuge
sudo apt install -y ros-dev-tools \
  python3-colcon-common-extensions python3-rosdep
sudo rosdep init && rosdep update

# 5) In jeder Shell verfuegbar machen
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc && source ~/.bashrc

# 6) Funktionstest (zwei Terminals)
ros2 run demo_nodes_cpp talker      # Terminal A
ros2 run demo_nodes_py listener    # Terminal B
```

Listing 19.2: ROS 2 Jazzy + Entwicklungswerkzeuge

19.1 Pakete, die du für die Projekte brauchst

Paket (apt: ros-jazzy-...)	Zweck
ros-gz (Gazebo Harmonic Bridge)	Simulation ↔ ROS 2
ros2-control, ros2-controllers	Hardware-Abstraktion, joint_trajectory_controller
gz-ros2-control	ros2_control-Plugin für Gazebo
moveit	Bewegungsplanung, IK, Greifen (Projekt 4)
navigation2, nav2-bringup	Autonome Navigation (Projekt 3)
slam-toolbox	SLAM/Kartierung (Projekt 3)
robot-localization	EKF/UKF-Sensorfusion (Odometrie)
image-pipeline	camera_calibration, stereo_image_proc
v4l2-camera bzw. usb-cam	Treiber für die USB-Stereokamera
vision-opencv (cv_bridge)	OpenCV ↔ ROS 2-Bildnachrichten
xacro, robot-state-publisher	URDF/Xacro, TF-Veröffentlichung
rqt, rqt-graph, rqt-image-view	Introspektion/Visualisierung

Installation gebündelt, z. B. `sudo apt install ros-jazzy-ros-gz ros-jazzy-ros2-control ros-jazzy-moveit ...`. Für den ESP32 zusätzlich der **micro-ROS Agent** (separat aus dem micro-ROS-Workspace gebaut).

19.2 Die Umgebung dauerhaft verfügbar machen (source)

ROS 2 legt sich nicht wie ein gewöhnliches Programm global in den Suchpfad. Stattdessen wird die Arbeitsumgebung pro Terminal mit `source` aktiviert: Das Skript `setup.bash` setzt Umgebungsvariablen – `PATH`, `LD_LIBRARY_PATH`, `AMENT_PREFIX_PATH`, `PYTHONPATH`, `GZ_SIM_RESOURCE_PATH` ... – *nur* für die aktuelle Shell. Erst dadurch finden `ros2`, `rviz2`, `gz` und die Pakete einander. Das ist auch der Grund, warum `gz` “nicht gefunden” wird, solange ROS 2 im Terminal nicht gesourct ist.

19.2.1 Underlay und Overlay

Es werden zwei Ebenen übereinandergelegt:

- **Underlay** – die systemweite ROS 2-Installation (`/opt/ros/jazzy/setup.bash`).
- **Overlay** – der eigene Workspace (`~/ros2_ws/install/setup.bash`), der das Underlay ergänzt bzw. überlagert.

Reihenfolge zählt: erst das Underlay, dann das Overlay sourcen.

19.2.2 Automatisch bei jedem Terminal – via `~/.bashrc`

Trägt man die `source`-Zeilen in `~/.bashrc` ein, führt *jede neue interaktive Shell* sie beim Start automatisch aus. Nach dem Booten genügt also: Terminal öffnen – `ros2`, `gz` und `rviz2` stehen sofort bereit.

```
source /opt/ros/jazzy/setup.bash           # Underlay (ROS 2)
source ~/ros2_ws/install/setup.bash       # Overlay (eigener Workspace)
# GPU-Rendering fuer Gazebo-Kamerasensoren (Single-NVIDIA-GPU, siehe Sim-Kapitel)
export __EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json
```

Listing 19.3: Feste Einträge am Ende von `~/.bashrc`

Einmal pro Terminal – nicht pro Befehl

`source` wirkt auf die gesamte Terminal-Sitzung. Man **sourct** **nicht** nach jedem Programm- oder Simulationsschritt erneut – die Umgebung bleibt für alle Befehle in diesem Terminal gesetzt. Über `~/.bashrc` geschieht selbst dieses eine Mal automatisch. Ein `source ~/.bashrc` ist nur nötig, um *Änderungen* an der Datei in ein bereits offenes Terminal zu übernehmen.

19.2.3 Was sich *nicht* automatisch sourcen lässt (und warum)

Kontext	Auto über <code>.bashrc</code> ?	Hinweis
Interaktives Terminal	ja	jede neue Shell ist fertig konfiguriert
Skripte via <code>cron/systemd</code>	nein	lesen <code>.bashrc</code> nicht – im Skript explizit <code>source</code>
Manche IDE-Run-Configs	teils	Umgebung ggf. in der Startkonfiguration setzen
Python-venv (MuJoCo, Isaac Sim)	bewusst nein	pro Aufgabe <code>source ~/venvs/.../bin/activate</code> , danach <code>deactivate</code>

venv und ROS 2 nicht vermischen

Die Python-Umgebungen für MuJoCo und Isaac Sim werden *nicht* in `.bashrc` aktiviert: Sie würden jede Shell mit einem fremden Python-Interpreter belegen und können mit dem System-Python von ROS 2 kollidieren. Isaac Sim und MuJoCo also gezielt im jeweiligen `venv` starten – ROS-Knoten dagegen in einer Shell, in der *nur* das ROS-Setup gesourct ist.

Teil V

Simulation und der Weg vom CAD-Modell zur Realität

Kapitel 20

Simulationswerkzeuge im ROS 2-Ökosystem

Gazebo Classic ist seit **Januar 2025 EOL**; “modernes Gazebo” (früher Ignition) ist der Nachfolger:

Werkzeug	Charakter	Eignung
Gazebo Harmonic	Standard-Simulator für Jazzy, Bridge <code>ros_gz</code>	Allrounder: Physik, Sensoren (Kamera, Tiefe, IMU, LiDAR), <code>gz_ros2_control</code> . Erste Wahl.
NVIDIA Isaac Sim	photorealistisch, GPU	Vision/Pose-Schätzung mit realistischen Bildern; braucht starke GPU; nur LTS-Distros
Webots MuJoCo	einfach, didaktisch schnelle, präzise Dynamik	schneller Einstieg, gute ROS 2-Anbindung kontaktreiche Regelung, RL; weniger Sensor-Realismus
PyBullet	leichtgewichtig	schnelles Prototyping von Kinematik/Dynamik

RViz2 ist kein Simulator, sondern Visualisierung – du nutzt es parallel.

20.1 Was lässt sich vorab simulieren?

Sehr viel – die meiste *Logik* der vier Projekte ist simulierbar:

- **Kinematik/Dynamik** des Stepper-Roboters: Gelenkbewegung, Trajektorienabfahrt, Kollisionen, Massen/Trägheiten.
- **Sensoren:** simulierte Stereokamera (gerenderte Bilder – du kannst deine Merkmalerkennung darauf testen), Tiefenbild, IMU, LiDAR, Odometrie.
- **Tischtennisball:** Projekttilflug inkl. Gravitation und Aufprall als Starrkörperphysik; dein Kalman-Filter kann auf den simulierten 3D-Positionen laufen.
- **Navigation:** Nav2 + simulierte Odometrie/LiDAR in einer Welt.
- **Greifen:** MoveIt 2-Planung in simulierter Szene mit platzierten Bauteilen.

Kannst du die Projekte vollständig in Simulation umsetzen?

Die Steuerungs-, Planungs- und Wahrnehmungs-Logik: ja. Du kannst Trajektorienregelung, Kalman-Schätzung, Navigation und die Vision-Pipeline komplett in Gazebo entwickeln und testen. **Was die Simulation nicht abbildet:** reales Kameraräuschen/Kalibrierfehler, Schrittverluste und Resonanzen des realen Steppers, exakte Kontaktdynamik (Ballspin/-absprung sind Näherungen), reale Latenzen und Zeitverhalten. Diese Effekte treten erst auf der echten Hardware zutage.

20.2 MuJoCo und Isaac Sim im Detail

Zwei Werkzeuge der obigen Tabelle verdienen eine genauere Betrachtung, weil sie in der modernen Robotik – besonders im Lernen von Regelungen (*Reinforcement Learning*, RL) und in der Wahrnehmung – eine Sonderstellung einnehmen. Sie lösen Probleme, an denen Gazebo als Allzweck-Simulator an seine Grenzen stößt: **MuJoCo** bei der schnellen, präzisen Kontaktdynamik, **Isaac Sim** beim photorealistischen Rendern und beim massiv parallelen GPU-Training.

20.2.1 MuJoCo – Multi-Joint dynamics with Contact

MuJoCo (*Multi-Joint dynamics with Contact*) ist eine quelloffene Physik-Engine, ursprünglich 2012 von Emanuel Todorov vorgestellt [24] und seit 2021 von Google DeepMind entwickelt und unter Apache-2.0 frei verfügbar [25]. Ihr Markenzeichen ist ein **stabiler, weicher Kontaktlöser** (konvexe Optimierung pro Zeitschritt), der kontaktreiche Vorgänge – Greifen, Laufen, Anschlagen – numerisch stabil und schnell rechnet.

- **Stärken:** sehr schnelle und genaue Dynamik (oft schneller als Echtzeit, hunderte bis tausende Schritte/s), differenzierbare Physik, analytisch saubere Gelenk-/Sehnenmodelle, exzellent für Regelungs- und RL-Forschung. Mit **MJX** läuft MuJoCo über JAX auf der GPU und simuliert tausende Welten parallel.
- **Schwächen:** *kein* photorealistisches Rendering (einfacher OpenGL-Viewer), weniger fertige Sensor-Plugins als Gazebo, Modelle in eigenem MJCF-XML (URDF-Import möglich, aber oft mit Nacharbeit).
- **Robotik-Relevanz:** De-facto-Standard im RL-Benchmarking (*Gymnasium/dm_control*), Lokomotion vierbeiniger Roboter, geschickte Manipulation, Modellprädiktive Regelung (MPC). Wenn die *Kontaktphysik* und die *Trainingsgeschwindigkeit* zählen, ist MuJoCo erste Wahl.

20.2.2 NVIDIA Isaac Sim – photorealistisch und GPU-parallel

Isaac Sim ist NVIDIAS Robotik-Simulator auf Basis der **Omniverse**-Plattform und des USD-Szenenformats [22]. Die Physik liefert **PhysX 5** (GPU-beschleunigt), das Rendering nutzt **RTX-Raytracing** – also physikalisch plausibles Licht, Schatten und Materialien.

- **Stärken:** *photorealistische* Kamerabilder, exzellente Erzeugung synthetischer Trainingsdaten (*Synthetic Data*, inkl. automatischer Labels: Segmentierung, Bounding-Boxes, Tiefe), *Domain Randomization* für robuste Vision-Modelle, GPU-parallele Physik für großskaliges RL über **Isaac Lab** [26].
- **Schwächen:** braucht eine starke NVIDIA-RTX-GPU (VRAM-hungrig), hohe Einstiegshürde, große Installation; weniger “leichtgewichtig” als Gazebo oder MuJoCo für schnelle Iteration.
- **Robotik-Relevanz:** überall dort, wo das *Aussehen* der Szene entscheidend ist – Training von Objekterkennung/Pose-Schätzung, Sim-to-Real für kamerabasierte Politiken, digitale Zwillinge ganzer Fertigungszellen.

20.3 Lassen sie sich integriert einsetzen?

Ja – aber meist arbeitsteilig, nicht als ein gemeinsamer Simulationskern. Beide sprechen ROS 2, sodass sie sich in dieselbe Knoten- und Topic-Architektur einfügen wie Gazebo. Der gemeinsame Nenner ist wieder `ros2_control` (vgl. das folgende Kapitel zur Sim-Real-Trennung): die High-Level-Logik bleibt identisch, nur die unterste Schicht – der Simulator hinter dem Hardware-Interface – wird getauscht.

Werkzeug	ROS 2-Anbindung	Typische Rolle im Workflow
Gazebo Harmonic	<code>ros_gz-Bridge</code> , <code>gz_ros2_control</code>	Allround-Integrationstest, Sensorik, Nav2/MoveIt2
MuJoCo	<code>mujoco_ros2_control</code> , eigene Bridges	Regler-/RL-Training, Kontaktdynamik, MPC
Isaac Sim	natives ROS 2-Bridge-Extension	Vision-Trainingsdaten, photorealistische Pose-Schätzung, RL via Isaac Lab

Der heute verbreitete **kombinierte Workflow** sieht so aus:

1. **Politik/Regler lernen** – massiv parallel in **Isaac Lab** (auf Isaac Sim) oder in **MuJoCo/MJX** auf der GPU. Hier zählen Geschwindigkeit und Kontaktphysik, nicht das Aussehen.
2. **Wahrnehmung trainieren** – photorealistische, automatisch gelabelte Bilder aus **Isaac Sim** für die Objekt-/Pose-Erkennung (Domain Randomization gegen den Sim-to-Real-Gap).
3. **Integrieren und gegentesten** – die fertigen Knoten im **Gazebo**-Allroundmodell oder direkt am realen Roboter über `ros2_control` verifizieren.

Einordnung für dieses Projekt

Für den Stepper- und Tischtennisroboter ist **Gazebo Harmonic die praktische Basis**: Kinematik, Sensorik, Kontakt Ball/Schläger, `ros2_control`. **MuJoCo** lohnt sich, falls die *Schlag-/Kontaktdynamik* oder eine *gelernte Schlagpolitik* (RL) genauer untersucht werden soll – es rechnet kontaktreiche Vorgänge schneller und stabiler. **Isaac Sim** wird relevant, wenn die *Ballerkennung* mit photorealistischen, gelabelten Bildern trainiert werden soll, statt sie nur an realen Kamerabildern abzustimmen. Beide setzen voraus: MuJoCo profitiert von einer GPU (MJX), Isaac Sim *erfordert* eine starke NVIDIA-RTX-GPU – das ist die wichtigste Hürde gegenüber dem leichten Gazebo-Setup.

20.4 Praktische Einrichtung auf dem Dual-Boot-System

Die bisher beschriebenen Werkzeuge werden nun konkret auf dem Arbeitsrechner dieses Projekts installiert: einem **Dual-Boot mit Ubuntu 24.04.4 LTS**, ausgestattet mit einer **NVIDIA GeForce RTX 3060 (12 GB)**. Der NVIDIA-Treiber inkl. CUDA ist bereits aktiv (verifizierbar mit `nvidia-smi`), ebenso `git`, `VSCode` und `python3`. Es fehlen noch `pip`, `Docker`, `ROS 2`, `Gazebo`, `colcon` und `roscdep`. Die folgende Reihenfolge baut das vollständige Entwicklungs- und Simulationssystem auf.

Warum die Reihenfolge zählt

ROS 2 muss *vor* den Simulations- und Projektpaketen liegen, weil diese `apt`-Pakete (`ros-jazzy-*`) auf die ROS 2-Apt-Quelle verweisen. Gazebo Harmonic wird auf Jazzy nicht separat installiert, sondern kommt als Abhängigkeit von `ros-jazzy-ros-gz` mit. Der `colcon` Workspace wird erst angelegt, nachdem ROS 2 in der Shell verfügbar ist.

20.4.1 Schritt 0 – System aktualisieren und Basiswerkzeuge

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y build-essential cmake git wget curl \
  gnupg lsb-release ca-certificates \
  python3-pip python3-venv python3-dev
```

Listing 20.1: Systemaktualisierung und Build-Grundausrüstung

20.4.2 Schritt 1 – ROS 2 Jazzy installieren

Die ROS 2-Kerninstallation (Apt-Quelle, `ros-jazzy-desktop`, `ros-dev-tools`, `colcon`, `rosdep`) ist in Kapitel 19 ausführlich beschrieben. Kurzfassung als Erinnerung – gebündelt:

```
sudo apt install -y ros-jazzy-desktop ros-dev-tools \
  python3-colcon-common-extensions python3-rosdep
sudo rosdep init && rosdep update
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc && source ~/.bashrc
```

Listing 20.2: ROS 2-Kern (Details siehe Kapitel “ROS 2 installieren”)

20.4.3 Schritt 2 – Simulations- und Projektpakete (inkl. Gazebo Harmonic)

```
sudo apt install -y \
  ros-jazzy-ros-gz ros-jazzy-gz-ros2-control \
  ros-jazzy-ros2-control ros-jazzy-ros2-controllers \
  ros-jazzy-moveit \
  ros-jazzy-navigation2 ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox \
  ros-jazzy-robot-localization \
  ros-jazzy-image-pipeline ros-jazzy-v4l2-camera ros-jazzy-vision-opencv \
  ros-jazzy-xacro ros-jazzy-robot-state-publisher \
  ros-jazzy-joint-state-publisher-gui \
  ros-jazzy-rqt ros-jazzy-rqt-graph ros-jazzy-rqt-image-view

# Gazebo Harmonic kam als Abhängigkeit mit -- prüfen:
gz sim --version
```

Listing 20.3: Gazebo Harmonic, `ros2_control`, MoveIt2, Nav2, Vision-Pipeline

gz: command not found? ROS 2 sourcen

Auf Jazzy ist Gazebo Harmonic *vendored*: es gibt kein systemweites `/usr/bin/gz`. Das `gz`-Binary liegt unter `/opt/ros/jazzy/opt/gz_tools_vendor/bin/` und gelangt erst durch `source /opt/ros/jazzy/setup.bash` in den `PATH`. Wer `ros-gz` in einem bereits geöffneten Terminal installiert, muss dieses Terminal neu starten (oder `source ~/.bashrc` ausführen), sonst meldet die Shell `gz: command not found`, obwohl das Paket installiert ist.

20.4.4 Schritt 3 – colcon-Workspace anlegen

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
colcon build --symlink-install
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc && source ~/.bashrc
# Später, sobald in src/ Pakete liegen, deren Abhängigkeiten auflösen:
# rosdep install --from-paths src --ignore-src -r -y
```

Listing 20.4: Eigener Arbeits-Workspace für die Projekt-Pakete

20.4.5 Schritt 4 – MuJoCo in einer Python-Umgebung

Ubuntu 24.04 ist “externally managed” (PEP 668)

Ein systemweites pip install schlägt unter Ubuntu 24.04 fehl. Für eigenständige Simulations-/RL-Skripte (MuJoCo, JAX) eine **virtuelle Umgebung** (venv) nutzen. Diese venv *nicht* gleichzeitig mit dem ROS2-Setup aktivieren, wenn ROS-Knoten laufen sollen – sie ist für ROS-unabhängige Skripte gedacht.

```
python3 -m venv ~/venvs/robotics
source ~/venvs/robotics/bin/activate
pip install --upgrade pip
pip install mujoco numpy scipy matplotlib
# GPU-beschleunigtes RL (nutzt die RTX 3060):
pip install "jax[cuda12]" mujoco-mjx
python -c "import mujoco; print(mujoco.__version__)"
deactivate
```

Listing 20.5: MuJoCo und der GPU-Pfad MJX

20.4.6 Schritt 5 – NVIDIA Isaac Sim (optional)

Die RTX 3060 mit 12 GB VRAM erfüllt die Mindestanforderung für Isaac Sim; für große, photorealistische Szenen bleibt der Speicher aber knapp. Isaac Sim lohnt sich hier vor allem zur Erzeugung gelabelter Trainingsbilder für die Ballerkennung.

```
nvidia-smi # Treiber/CUDA verifizieren (hier bereits aktiv)
python3 -m venv ~/venvs/isaac
source ~/venvs/isaac/bin/activate
pip install --upgrade pip
pip install isaacsim --extra-index-url https://pypi.nvidia.com
# Hinweis: die von NVIDIA unterstützte Python-Version laut Doku beachten.
# Alternative: Omniverse-/Isaac-Launcher (grafischer Installer von NVIDIA).
```

Listing 20.6: Isaac Sim über den pip-Workflow

20.4.7 Schritt 6 – VSCode-Erweiterungen

```
code --install-extension ms-iot.vscode-ros      # ROS-Integration
code --install-extension ms-python.python      # Python
code --install-extension ms-vscode.cpptools    # C/C++
code --install-extension ms-vscode.cmake-tools # CMake/colcon
code --install-extension redhat.vscode-xml     # URDF/Xacro
code --install-extension redhat.vscode-yaml    # Launch-/Param-Dateien
```

Listing 20.7: Erweiterungen für ROS 2-Entwicklung

20.4.8 Schritt 7 – micro-ROS-Agent für den ESP32

Der **micro-ROS-Agent** verbindet den ESP32 (TMC2209-Ansteuerung) mit dem ROS 2-Graphen. Der einfachste Weg ist das offizielle Docker-Image:

```
sudo apt install -y docker.io
sudo usermod -aG docker $USER      # danach einmal ab- und wieder anmelden
# ESP32 am USB (/dev/ttyUSB0), 115200 Baud:
docker run -it --rm --net=host --device=/dev/ttyUSB0 \
  microros/micro-ros-agent:jazzy serial --dev /dev/ttyUSB0 -b 115200
```

Listing 20.8: micro-ROS-Agent via Docker (serielle Brücke)

20.4.9 Schritt 8 – GPU-Rendering (EGL) für Gazebo-Kamerasensoren

Startet Gazebo auf diesem System, erscheinen trotz funktionierender 3D-Ansicht oft Warnungen wie:

```
libEGL warning: egl: failed to create dri2 screen
libEGL warning: pci id for fd 117: 10de:2504, driver (null)
```

Listing 20.9: EGL-Warnungen beim Gazebo-Start (Ausgabe, kein Befehl)

Ursache: Es sind zwei EGL-Treiber registriert – `10_nvidia.json` und `50_mesa.json` (in `/usr/share/egl_vendor.d/`). Bei nur *einer* NVIDIA-GPU (hier die RTX 3060, PCI-ID `10de:2504`) probiert die `glvnd`-Schicht zuerst den Mesa-Treiber auf der NVIDIA-Karte, der dort keinen DRI-Treiber findet (`driver (null)`), und fällt erst danach auf den NVIDIA-Treiber zurück – deshalb erscheint das Bild trotz der Warnung.

Warum das wichtig ist: Die GUI rendert ohnehin über NVIDIA. Kritisch ist das *Off-Screen-Rendering der simulierten Kamerasensoren*, das Gazebo über EGL ausführt – die Grundlage der Stereokamera-Projekte. Fällt EGL auf die Mesa-Software zurück, wird die Sensorbild-Erzeugung langsam oder fehlerhaft.

Behebung: `glvnd` auf den NVIDIA-Treiber festlegen. Da nur *eine* NVIDIA-GPU verbaut ist, ist das gefahrlos – dauerhaft in `~/.bashrc`:

```
export __EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/egl_vendor.d/10_nvidia.json
```

Listing 20.10: NVIDIA-EGL erzwingen

Nur bei Single-NVIDIA-GPU

Diese Zeile zwingt *alle* EGL-Anwendungen auf den NVIDIA-Treiber. Auf einem Rechner ohne NVIDIA-GPU (oder headless ohne GPU) muss sie wieder entfernt werden. Auf Hybrid-Laptops (Intel + NVIDIA) stattdessen das Render-Offload nutzen: `__NV_PRIME_RENDER_OFFLOAD=1 __GLX_VENDOR_LIBRARY_NAME=nvidia gz sim ...`

Schnelltest des Gesamtsystems

Nach Schritt 3 lässt sich die Installation verifizieren: `ros2 run demo_nodes_cpp talker` in einem Terminal, `ros2 run demo_nodes_py listener` in einem zweiten – die Nachrichten müssen ankommen. `gz sim shapes.sdf` startet Gazebo Harmonic mit einer Testwelt; `rviz2` öffnet die Visualisierung. Damit stehen *Kommunikation*, *Simulation* und *Visualisierung*.

Kapitel 21

Wo sich Simulation und reale Implementierung trennen

Das ist die architektonisch wichtigste Einsicht des ganzen Dokuments. Der Trick von **ros2_control**: deine High-Level-Knoten (Trajektorienplaner, Vision, Kalman, Nav2) bleiben *identisch*. Was sich ändert, ist nur die unterste Schicht – das **Hardware-Interface-Plugin**.

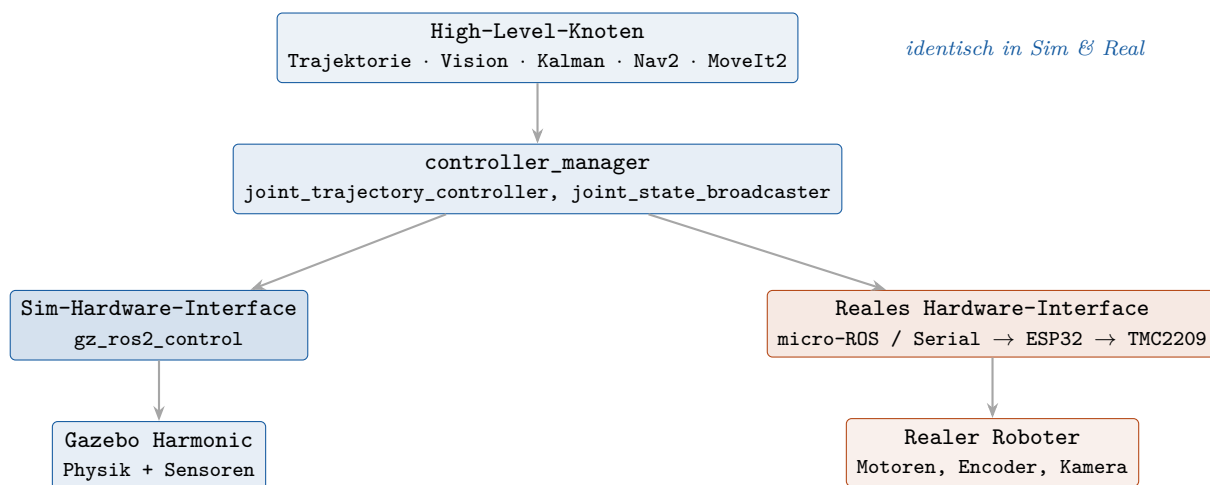


Abbildung 21.1: Sim-zu-Real: nur die unterste Schicht (Hardware-Interface) wird getauscht. Alles darüber bleibt gleich.

Konkret: In der Simulation lädt die URDF das Plugin `gz_ros2_control/GazeboSimSystem`. Für die echte Maschine schreibst du ein eigenes `hardware_interface::SystemInterface`, das Sollwerte an den ESP32 schickt (micro-ROS oder serielle Brücke) und Gelenkzustände (Encoder/StallGuard) zurückliest. **Dieselben Controller, dieselben Topics, dieselbe Trajektorie** – du tauschst eine Plugin-Zeile und die Hardware. Genauso bei der Kamera: simulierte Bildquelle vs. `v4l2_camera` – die nachgelagerte Vision-Pipeline merkt keinen Unterschied.

21.1 Der vollständige Weg: von der frischen Ubuntu-Installation zum Roboter

Die in Kapitel 19 und im vorigen Kapitel beschriebene Einrichtung und die hier gezeigte Sim-Real-Architektur greifen zu *einem* durchgängigen Arbeitsablauf ineinander. Auf dem Dual-Boot-System (Ubuntu 24.04.4 LTS, RTX 3060) folgt die Entwicklung dieser Reihenfolge – jede Phase baut auf der vorigen auf, und der Übergang von Simulation zu realer Hardware ist genau der oben gezeigte Plugin-Tausch.

1. **System aufsetzen.** Frisches Ubuntu 24.04, NVIDIA-Treiber aktiv, ROS 2 Jazzy, Gazebo Harmonic, `ros2_control`, Vision-/Nav2-Pakete, `colcon-Workspace` (Schritte 0–3 des vorigen Kapitels). Verifikation: `talker/listener` und eine Gazebo-Testwelt laufen.
2. **Roboter modellieren.** URDF/Xacro mit Links, Gelenken, Trägheiten und Sensoren erstellen (folgendes Kapitel zu CAD → URDF). In dieser Datei steht bereits das `ros2_control`-Tag – zunächst mit dem *Simulations*-Plugin `gz_ros2_control/GazeboSimSystem`.
3. **In Simulation entwickeln.** Roboter in Gazebo Harmonic laden, `controller_manager` mit `joint_trajectory_controller` und `joint_state_broadcaster` starten. High-Level-Logik – Trajektorienplaner, Kalman-Filter, Vision-Pipeline, Nav2/MoveIt2 – vollständig gegen die simulierte Maschine entwickeln und testen.
4. **Regelung/Wahrnehmung verfeinern (optional).** Schlag-/Kontaktdynamik oder eine gelernte Politik in `MuJoCo/MJX` trainieren, photorealistische, gelabelte Trainingsbilder für die Ballerkennung in **Isaac Sim** erzeugen. Die fertigen Knoten bleiben unverändert.
5. **Auf reale Hardware umschalten.** In der URDF das Hardware-Interface von `gz_ros2_control` auf das eigene `hardware_interface::SystemInterface` (micro-ROS/Serial → ESP32 → TMC2209) umstellen; reale Kamera über `v4l2_camera` statt simulierter Bildquelle. **Controller, Topics, Trajektorien und alle High-Level-Knoten bleiben identisch** – siehe das Schichtdiagramm oben.
6. **Gegentesten.** Dieselben Testfälle, die in Simulation liefen, auf der echten Maschine fahren. Jetzt – und erst jetzt – treten die in Kapitel 19 bzw. im Simulationskapitel benannten realen Effekte zutage: Kamerarauschen, Schrittverluste, Kontakt- und Latenzverhalten.

Was an dieser Reihenfolge entscheidend ist

Die Trennlinie zwischen Simulation und Realität verläuft *nicht* quer durch den Code, sondern sauber an *einer* Schicht – dem Hardware-Interface. Die gesamte oben aufgebaute Software-Umgebung (ROS 2, Gazebo, MuJoCo, Isaac Sim) dient dazu, möglichst viel *vor* dem Hardware-Tausch abzusichern. Was in Simulation funktioniert, läuft nach dem Plugin-Wechsel ohne Änderung an der Logik auf dem realen Roboter – abzüglich der physikalischen Effekte, die nur die Hardware zeigt.

Kapitel 22

Vorabschritte: vom CAD-Modell zur Simulation

Bevor du simulierst, brauchst du ein **URDF** (Unified Robot Description Format) – die XML-Beschreibung deines Roboters aus *Links* (starre Körper) und *Joints* (Gelenke dazwischen).

22.1 Getrennte STL-Exporte pro Gelenk

Wenn zwischen Teil A und Teil B eine Gelenkverbindung besteht, müssen beide *separat* als Mesh (STL/DAE) exportiert werden – denn jedes ist ein eigener Link mit eigenem Koordinatensystem, und das Gelenk dazwischen wird in der URDF definiert, nicht im CAD-Mesh. Vorgehen:

1. Im CAD die Gelenkachsen und je Link ein Referenz-Frame festlegen (idealerweise Mesh-Ursprung = Gelenkpunkt – erspart später viel Frame-Frust).
2. **Jeden Link einzeln** als STL exportieren (Teil A, Teil B, ...), bevorzugt in seinem eigenen Frame.
3. Masse + Trägheitstensor je Link aus dem CAD übernehmen (für korrekte Dynamik).
4. URDF schreiben: Links mit `visual/collision/inertial`; Joints mit `parent/child, origin` (Lage des Gelenks), `axis` und `limit`.

Automatisierung – nicht von Hand schreiben

Nutze einen CAD-zu-URDF-Exporter statt das XML manuell zu tippen: **sw2urdf** (Solid-Works), **onshape-to-robot** (OnShape) oder Fusion-360-Plugins. Du wählst Gelenkachsen und Referenz-Frames interaktiv; das Tool exportiert die Meshes *und* eine konsistente URDF mit korrekten Frames und Trägheiten.

```

<link name="teilA">
  <visual>
    <geometry>
      <mesh filename="package://my_robot/meshes/teilA.stl"/>
    </geometry>
  </visual>
  <collision>
    <geometry>
      <mesh filename="package://my_robot/meshes/teilA.stl"/>
    </geometry>
  </collision>
  <inertial>
    <mass value="0.45"/>
    <inertia ixx="0.001" iyy="0.001" izz="0.0008"
      ixy="0" ixz="0" iyz="0"/>
  </inertial>
</link>

<joint name="gelenk_AB" type="revolute">
  <parent link="teilA"/>
  <child link="teilB"/>
  <origin xyz="0 0 0.08" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-1.57" upper="1.57" effort="12.0" velocity="3.0"/>
</joint>

<link name="teilB"> <!-- visual/collision/inertial wie oben --> </link>

```

Listing 22.1: URDF-Auszug: zwei Links mit Drehgelenk

22.2 Gelenktypen

- *revolute* – Drehgelenk mit Anschlag (Limits). Der Normalfall für Roboterarme.
- *continuous* – Drehgelenk ohne Anschlag (Räder).
- *prismatic* – Linearführung.
- *fixed* – starre Verbindung (z. B. Kamera an Basis).

22.3 Getriebeübersetzung in `ros2_control`

Motor + Zykloidgetriebe werden als *ein* angetriebenes Gelenk abstrahiert. Die Untersetzung steckt in der `ros2_control-Transmission`:

```

<transmission name="trans_gelenk_AB">
  <type>transmission_interface/SimpleTransmission</type>
  <joint name="gelenk_AB">
    <hardwareInterface>
      hardware_interface/PositionJointInterface
    </hardwareInterface>
  </joint>
  <actuator name="motor_AB">
    <mechanicalReduction>40</mechanicalReduction> <!-- 40:1 -->
  </actuator>
</transmission>

```

Listing 22.2: Getriebeübersetzung als SimpleTransmission

Die Physik-Engine sieht nur das Ausgangsgelenk. Die Übersetzung bildet das Verhältnis Motorwinkel $\leftrightarrow$ Gelenkwinkel ab; die *nach* dem Getriebe verfügbaren Werte trägst du als `effort/velocity`-

Limits am Joint ein (hohes Moment, niedrige Drehzahl). Reales Restspiel und Reibung kannst du optional über die Joint-Dynamik (`damping`, `friction`) annähern.

Teil VI

Einstiegsprojekte: Erste Schritte mit ROS 2, RViz und Gazebo

Kapitel 23

Projekt 0.A – Planarer 2-Gelenk-Arm: Nodes, Topics, Services, Actions, IK

Ziel: ein erstes, vollständig selbst implementiertes ROS 2-System, das alle Grundbausteine berührt – **Nodes, Topics, Services, Actions, URDF** und **inverse Kinematik** – und das wir in **RViz** visualisieren und in **Gazebo Harmonic** simulieren. Wir übernehmen nur die *Geometrie* eines bekannten Lernroboters (den planaren 2-Gelenk-Arm im Stil des “RRBot” aus `ros2_control_demos`) und schreiben die gesamte Logik selbst. Die Sim-Real-Architektur aus Kapitel 21 gilt unverändert.

Warum gerade dieser Arm?

Der Arm besteht aus zwei Drehgelenken (*revolute*) und zwei Stäben und bewegt sich in einer Ebene. Seine Geometrie ist trivial – zwei Zylinder, ganz ohne CAD-Mesh – aber er deckt alle Lernziele ab. Wichtig: die **inverse Kinematik eines 2-Glieder-Arms ist von Hand nachprüfbar** (es gibt sogar eine geschlossene Lösung über den Kosinussatz). Wir nutzen den Arm aber, um das *allgemeine* IK-Verfahren zu üben, das auch bei komplexen Robotern greift: Vorwärtskinematik aus **Denavit-Hartenberg**-Parametern plus **numerische** Lösung über die Jacobi-Matrix. Die geschlossene Lösung dient nur als Referenz zum Gegenprüfen. Vertiefung in Kapitel 6.

23.1 ROS 2-Bausteine

Typ	Konkret (selbst implementiert, sofern nicht anders vermerkt)
Nodes	<code>arm_commander</code> (Action-/Service-Client) · <code>reach_action_server</code> (IK + Bahnerzeugung) · <code>robot_state_publisher</code> (Standard) · <code>controller_manager</code> + <code>joint_trajectory_controller</code> + <code>joint_state_broadcaster</code> (Standard) · Hardware-Interface (Sim: <code>gz_ros2_control</code>)
Topics	<code>/joint_states</code> · <code>/tf</code> , <code>/tf_static</code> · <code>/robot_description</code> · <code>/arm_controller/joint_trajectory</code>
Services	<code>/arm/set_named_pose</code> (eigener srv: <code>home/extended/folded</code>) · <code>/controller_manager/switch_controller</code> (Standard)
Actions	<code>/arm/reach_point</code> (eigene action: Ziel x, y ; Feedback: Restdistanz; Result: Erfolg) · intern <code>control_msgs/FollowJointTrajectory</code>
Parameter	Gliedlängen l_1, l_2 · Gelenklimits · benannte Posen

23.2 rqt_graph (Zielbild)

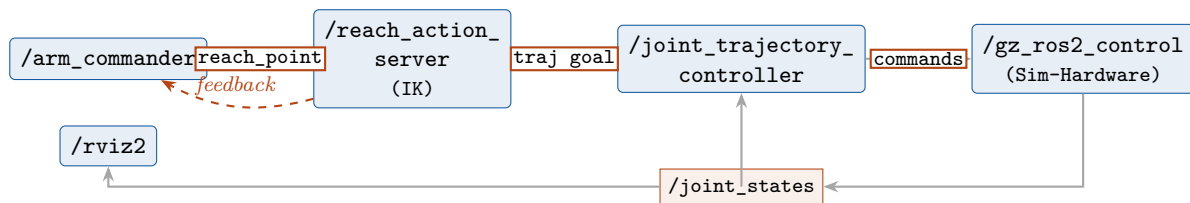


Abbildung 23.1: Projekt 0.A: Der Befehlsknoten schickt ein kartesisches Ziel als Action-Goal; der Action-Server rechnet die IK, erzeugt eine Trajektorie und übergibt sie dem Controller. /joint_states schließt die Schleife und speist RViz.

Abbildung 23.2 zeigt den *tatsächlichen* rqt_graph (`ros2 run rqt_graph rqt_graph`), aufgenommen während **Schritt 8**: der `reach_action_server` läuft, der Gazebo-Stack aus `gazebo.launch.py` ist aktiv, und es wird ein Ziel über `/arm/reach_point` gesendet.

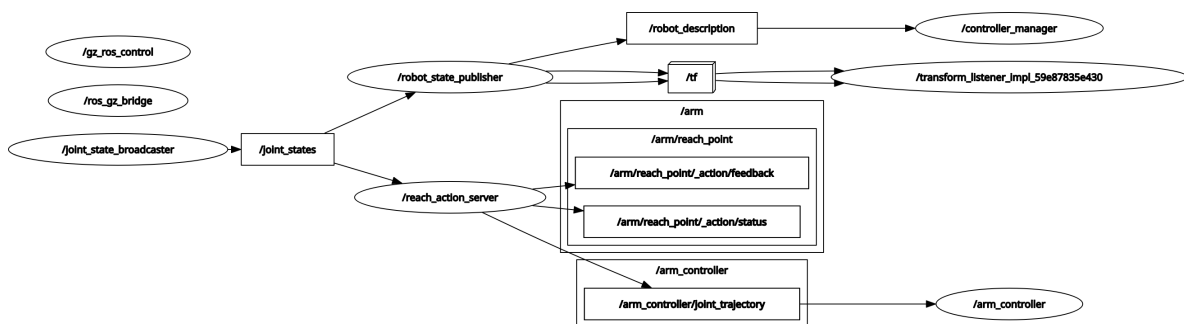


Abbildung 23.2: Tatsächlicher rqt_graph aus Schritt 8 (laufender `reach_action_server` + Gazebo-Stack, ein Ziel wird angefahren). Ovale sind Knoten, Rechtecke Topics. Vgl. das idealisierte Zielbild oben – hier tritt anstelle des `arm_commander` (Schritt 5) der `reach_action_server` auf, dazu die sonst verborgene Action-, Zeit- und TF-Infrastruktur.

So liest sich der Graph:

- **Action-Schnittstelle.** Der Goal-Sender – ein transienter `ros2 action send_goal`-Knoten (`/_ros2cli_...`) – ist über die versteckten Action-Topics `/arm/reach_point/_action/*` (Goal, Feedback, Status, Result) mit `/reach_action_server` verbunden. Genau diese Kante trägt das `-feedback` aus Schritt 8.
- **Trajektorie.** `/reach_action_server` publiziert auf `/arm_controller/joint_trajectory`; Abonnent ist der `/controller_manager`, in dem `arm_controller` und `joint_state_broadcaster` laufen.
- **Rückkopplung.** Der `joint_state_broadcaster` veröffentlicht `/joint_states`. Daran hängen *zwei* Knoten: `/reach_action_server` (rechnet daraus per Vorwärtskinematik die Restdistanz fürs Feedback) und `/robot_state_publisher`.
- **TF und Anzeige.** `/robot_state_publisher` wandelt `/joint_states` in `/tf` (+ `/tf_static`) um – die Quelle, aus der `/rviz2` die Glieder positioniert.
- **Zeit.** Der `ros_gz_bridge`-Knoten (`parameter_bridge`) speist `/clock` aus Gazebo; daran hängen alle `use_sim_time`-Knoten – fehlt diese Kante, steht das ganze System ohne Uhr da (vgl. Checkliste).
- **Simulation.** Der Gazebo-Knoten und das Topic `/robot_description` (von `robot_state_publisher`, beim Spawnen gelesen) vervollständigen das Bild.

Zum Gegenprüfen: Verläuft eine erwartete Kante nicht wie hier beschrieben – etwa `/joint_states` erreicht den `robot_state_publisher` nicht –, ist genau das die Stelle für die Fehlersuche. `rqt_graph` ist damit das schnellste Architektur-Diagnosewerkzeug.

23.3 Schritt für Schritt

23.3.1 Schritt 1 – Pakete anlegen und die Struktur verstehen

Die Projektlogik kommt in ein Python-Paket, die eigenen Schnittstellen (`srv/action`) in ein separates `ament_cmake`-Paket – nur dort generiert ROS 2 aus `.srv/.action` echte Nachrichtentypen.

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python p0a_arm \
  --license Apache-2.0 \
  --dependencies rclpy sensor_msgs trajectory_msgs control_msgs

ros2 pkg create --build-type ament_cmake p0a_arm_interfaces \
  --license Apache-2.0 --dependencies rosidl_default_generators

mkdir -p p0a_arm/{urdf,config,launch} p0a_arm_interfaces/{srv,action}
```

Listing 23.1: Pakete anlegen (im Workspace-src)

Was die Optionen bedeuten.

- `-build-type ament_python` – das Paket wird über `setup.py` gebaut, die Knoten sind Python-Skripte. Alternative `ament_cmake` (C++ oder Interface-Generierung).
- `-dependencies ...` – trägt die genannten Pakete als `<depend>` in die `package.xml` ein. `rclpy` ist die Python-Client-Bibliothek von ROS 2 (Knoten, Publisher, Timer ...), dazu die verwendeten Nachrichtenpakete. *Deklaration*, keine Installation.
- `-license Apache-2.0` – vermeidet die TODO-Warnung und schreibt eine gültige Lizenz in `package.xml`.

Zielstruktur des Python-Pakets.

```
p0a_arm/
|- package.xml           # Metadaten + Abhaengigkeiten (<depend>)
|- setup.py              # Build/Install: Knoten (entry_points) + Daten (
|- setup.cfg             # Ablageort der ausfuehrbaren Skripte
|- resource/p0a_arm      # Marker fuer den ament-Index
|- p0a_arm/              # Python-Modul (gleicher Name) -> hier die Knoten
| | - __init__.py
| | - arm_commander.py
| | - reach_action_server.py
| | - ik.py
|- urdf/                 arm.urdf.xacro
|- config/               controllers.yaml arm.rviz
`- launch/               display.launch.py gazebo.launch.py
```

Listing 23.2: Verzeichnisbaum von `p0a_arm` (Endstand Projekt)

Die `package.xml` ist die Visitenkarte des Pakets: Name, Version, Lizenz und die `<depend>`-Zeilen, die `roscpp` und `colcon` auswerten. Die `setup.py` bestimmt, *was installiert wird* – dazu in Schritt 3 mehr: `urdf/`, `launch/` und `config/` müssen dort explizit eingetragen werden, sonst landen sie nicht im `install/-`Baum.

```
src/
|- p0a_arm/              # neu (ament_python: rclpy-Knoten)
`- p0a_arm_interfaces/   # neu (ament_cmake: srv/action)
```

Listing 23.3: `ros2_ws`-Delta nach Schritt 1 (# neu / # angepasst)

23.3.2 Schritt 2 – Geometrie als URDF/Xacro

Wir schreiben das Modell selbst – das *ist* das URDF-Lernziel. Datei: `p0a_arm/urdf/arm.urdf.xacro`. Beide Gelenke drehen um z , die Glieder liegen entlang $+x$ ($l_1 = 1,0$, $l_2 = 0,8$, passend zur DH-Tabelle aus Schritt 7).

Aufbau einer URDF. Eine URDF beschreibt einen Baum aus **Links** (starre Körper) und **Joints** (Verbindungen):

- `<robot name=...>` – Wurzelement, allgemeine Metadaten. Der Robotername ist frei; entscheidend sind die später referenzierten *Link*- und *Joint*-Namen.
- `<joint>` – verbindet einen `<parent>`- mit einem `<child>`-Link, mit `<origin>` (Lage relativ zum Parent), `<axis>` (Achse) und `<limit>` (Grenzen). `type: revolute` (begrenzt drehbar), `continuous`, `prismatic` (schiebend), `fixed` (starr).
- `<link>` – ein Körper mit `<visual>` (Darstellung), `<collision>` (Kollisionskörper) und `<inertial>` (Masse + Trägheitstensor; für Gazebo nötig, für RViz optional).
- `<ros2_control>/<gazebo>` – keine Geometrie, sondern die Anbindung an die Simulation (Schritt 4).

Müssen Joints und Links in Reihenfolge stehen?

Nein. URDF wird *nicht* als Reihenfolge von oben nach unten gelesen. Der kinematische Baum entsteht allein aus den `<parent>/<child>`-Verweisen der Joints. Die Abfolge `joint1`, `link1`, `joint2`, `link2` ... ist nur *lesbare* Konvention – man dürfte alle Links zuerst und dann alle Joints notieren. Pflicht ist nur: genau *ein* Wurzel-Link (ohne Parent), ein zusammenhängender, zyklensfreier Baum, und jeder referenzierte Link existiert.

Die vollständige, lauffähige Datei (Glieder als Boxen, mit Inertials und Collisions):

```

<?xml version="1.0"?>
<!-- Planarer 2-Gelenk-Arm: Gelenke um z, Glieder entlang +x. l1=1.0, l2=0.8 -->
<robot name="rrarm">

  <link name="world"/>
  <joint name="fix" type="fixed">
    <parent link="world"/><child link="base"/>
  </joint>

  <link name="base">
    <visual><origin xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.05"/></geometry>
      <material name="grey"><color rgba="0.5 0.5 0.5 1"/></material></visual>
    <collision><origin xyz="0 0 0.05"/>
      <geometry><cylinder length="0.1" radius="0.05"/></geometry></collision>
    <inertial><mass value="1.0"/>
      <inertia ixx="0.01" ixy="0" ixz="0" iyy="0.01" iyz="0" izz="0.01"/></inertial>
  </link>

  <joint name="joint1" type="revolute">
    <parent link="base"/><child link="link1"/>
    <origin xyz="0 0 0.1"/><axis xyz="0 0 1"/>
    <limit lower="-3.14" upper="3.14" effort="50" velocity="2.0"/>
  </joint>
  <link name="link1">
    <visual><origin xyz="0.5 0 0"/>
      <geometry><box size="1.0 0.06 0.06"/></geometry>
      <material name="blue"><color rgba="0.2 0.4 0.8 1"/></material></visual>
    <collision><origin xyz="0.5 0 0"/>
      <geometry><box size="1.0 0.06 0.06"/></geometry></collision>
    <inertial><origin xyz="0.5 0 0"/><mass value="1.0"/>
      <inertia ixx="0.001" ixy="0" ixz="0" iyy="0.084" iyz="0" izz="0.084"/></inertial>
  </link>

  <joint name="joint2" type="revolute">
    <parent link="link1"/><child link="link2"/>
    <origin xyz="1.0 0 0"/><axis xyz="0 0 1"/>
    <limit lower="-3.14" upper="3.14" effort="50" velocity="2.0"/>
  </joint>
  <link name="link2">
    <visual><origin xyz="0.4 0 0"/>
      <geometry><box size="0.8 0.05 0.05"/></geometry>
      <material name="orange"><color rgba="0.9 0.5 0.1 1"/></material></visual>
    <collision><origin xyz="0.4 0 0"/>
      <geometry><box size="0.8 0.05 0.05"/></geometry></collision>
    <inertial><origin xyz="0.4 0 0"/><mass value="0.8"/>
      <inertia ixx="0.0008" ixy="0" ixz="0" iyy="0.043" iyz="0" izz="0.043"/></inertial>
  </link>

  <!-- Anbindung an die Simulation (Schritt 4) -->
  <ros2_control name="GazeboSystem" type="system">
    <hardware><plugin>gz_ros2_control/GazeboSimSystem</plugin></hardware>
    <joint name="joint1"><command_interface name="position"/>
      <state_interface name="position"/><state_interface name="velocity"/></joint>
    <joint name="joint2"><command_interface name="position"/>
      <state_interface name="position"/><state_interface name="velocity"/></joint>
  </ros2_control>
</robot>

```

Listing 23.4: urdf/arm.urdf.xacro – vollständig

Der <gazebo>-Plugin-Block kommt erst in Schritt 4

Der Block, der die Controller lädt, verweist mit `$(find p0a_arm)` auf das *installierte* Paket:

```
<gazebo>
  <plugin filename="gz_ros2_control-system"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>$(find p0a_arm)/config/controllers.yaml</parameters>
  </plugin>
</gazebo>
```

`$(find ...)` lässt sich erst auflösen, *nachdem* das Paket gebaut und gesourct ist. Schon für den RViz-Test (Schritt 3) hineingenommen, scheitert `xacro` mit “package not found”. Daher erst in Schritt 4 ergänzen.

```
src/p0a_arm/
`- urdf/arm.urdf.xacro # neu (Robotermodell: links, joints, ros2_control)
```

Listing 23.5: `ros2_ws-Delta` nach Schritt 2 (# neu / # angepasst)

23.3.3 Schritt 3 – In RViz darstellen (über eine Launch-Datei)

Den großen URDF-String *nicht* als Kommandozeilen-Parameter übergeben – `rcl` kann das mehrzeilige XML nicht parsen (Couldn’t parse parameter override rule). Der saubere Weg ist eine **Launch-Datei**, die `robot_state_publisher` den `robot_description`-Parameter als echten Wert übergibt.

Ressourcen installierbar machen. Damit `ros2 launch` die Dateien findet, müssen `urdf/`, `launch/` und `config/` in der `setup.py` als `data_files` eingetragen werden:

```
import os
from glob import glob
# ... in data_files=[ ... ] zusätzlich:
(os.path.join('share', package_name, 'urdf'), glob('urdf/*')),
(os.path.join('share', package_name, 'launch'), glob('launch/*.launch.py')),
(os.path.join('share', package_name, 'config'), glob('config/*')),
```

Listing 23.6: Ergänzung in `setup.py`

```

from launch import LaunchDescription
from launch.substitutions import Command, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from launch_ros.parameter_descriptions import ParameterValue

def generate_launch_description():
    pkg = FindPackageShare('p0a_arm')
    xacro_file = PathJoinSubstitution([pkg, 'urdf', 'arm.urdf.xacro'])
    rviz_config = PathJoinSubstitution([pkg, 'config', 'arm.rviz'])
    robot_description = ParameterValue(Command(['xacro ', xacro_file]),
                                       value_type=str)

    return LaunchDescription([
        Node(package='robot_state_publisher', executable='robot_state_publisher',
            parameters=[{'robot_description': robot_description}]),
        Node(package='joint_state_publisher_gui',
            executable='joint_state_publisher_gui'),
        Node(package='rviz2', executable='rviz2',
            arguments=['-d', rviz_config]),
    ])

```

Listing 23.7: launch/display.launch.py

RViz-Ansicht vorkonfigurieren (automatisch). Frisch gestartet steht RViz auf *Fixed Frame* map (Fehler “Frame [map] does not exist”) und zeigt kein Modell – es ist noch kein RobotModel-Display angelegt. Damit Modell und TF-Achsen sofort erscheinen, legt die mitgelieferte config/arm.rviz drei Dinge fest:

```

Visualization Manager:
  Global Options:
    Fixed Frame: world          # statt "map" -> kein Frame-Fehler mehr
  Displays:
    - Class: rviz_default_plugins/RobotModel
      Description Source: Topic
      Description Topic:
        Value: /robot_description
      Durability Policy: Transient Local  # WICHTIG: rsp latcht die Beschreibung
    - Class: rviz_default_plugins/TF      # zeigt die Gelenk-Frames

```

Listing 23.8: config/arm.rviz – die entscheidenden Einträge

Der manuelle Weg (ohne Config-Datei). In *Global Options* das *Fixed Frame* auf world setzen. Dann unten links *Add* → RobotModel → OK und in dessen Eigenschaften *Description Topic* auf /robot_description stellen – der Arm erscheint. Optional *Add* → TF für die Gelenk-Frames. Zum Schluss *File* → *Save Config As* nach config/arm.rviz, damit der -d-Start sie künftig automatisch lädt. **Häufige Falle:** ohne *Transient Local* sieht das RobotModel die einmalig gelatchte Beschreibung nicht – das Modell bleibt unsichtbar.

```

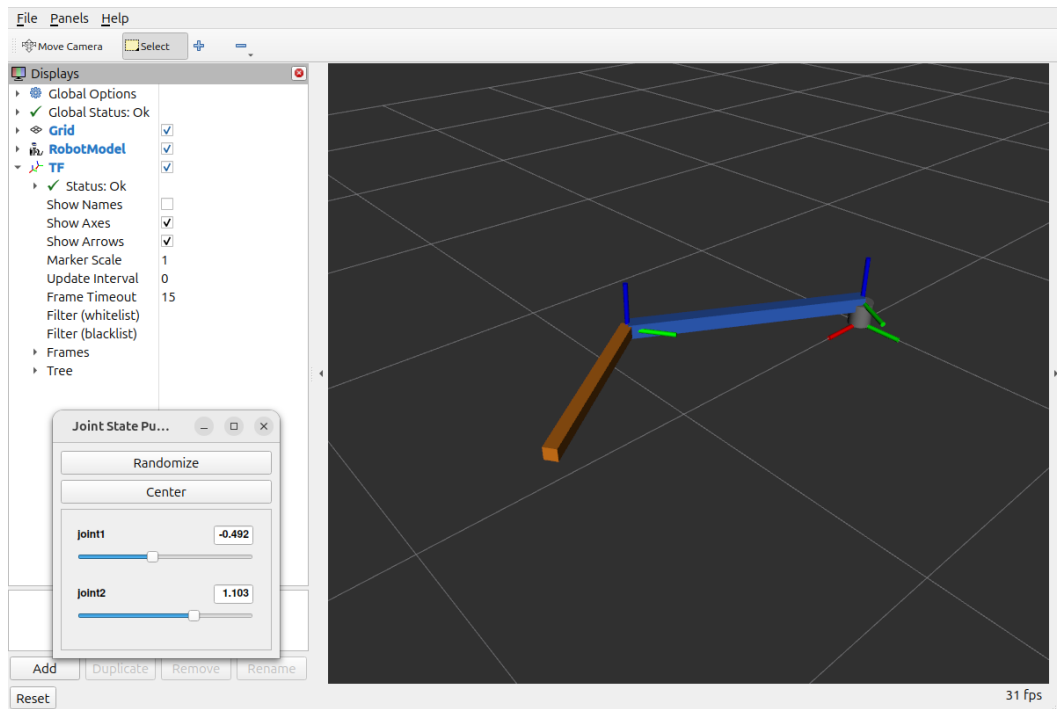
cd ~/ros2_ws
colcon build --packages-select p0a_arm
source install/setup.bash          # neue Terminals tun das via .bashrc selbst
ros2 launch p0a_arm display.launch.py  # laedt arm.rviz automatisch (-d)

```

Listing 23.9: Bauen, sourcen, starten

```
src/p0a_arm/
|- setup.py           # angepasst: data_files (urdf, launch, config)
|- launch/display.launch.py # neu
~- config/arm.rviz    # neu
```

Listing 23.10: ros2_ws-Delta nach Schritt 3 (# neu / # angepasst)

Abbildung 23.3: RViz mit dem Arm-Modell und den TF-Achsen. Über den `joint_state_publisher_gui` lassen sich die Gelenke bewegen – dreht sich der Arm plausibel, sind URDF und TF korrekt.

23.3.4 Schritt 4 – In Gazebo simulieren (`ros2_control`)

Jetzt mit Physik. **Zuerst** den in Schritt 2 gezeigten `<gazebo>`-Block in `arm.urdf.xacro` aktivieren (Auskommentierung aufheben): Da das Paket nun gebaut und gesourct ist, löst `$(find p0a_arm)` auf den installierten config-Pfad auf. Die Controller beschreibt eine YAML, die der vom Plugin gestartete `controller_manager` lädt:

```
controller_manager:
  ros__parameters:
    update_rate: 100
    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster
    arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController
arm_controller:
  ros__parameters:
    joints: [joint1, joint2]
    command_interfaces: [position]
    state_interfaces: [position, velocity]
```

Listing 23.11: config/controllers.yaml

Nach jeder Änderung an `urdf/` oder `config/` neu bauen und sourcen, sonst läuft die alte installierte Fassung: `colcon build --packages-select p0a_arm && source install/setup.bash`.

Empfohlen: alles in einer Launch-Datei. Vier Knoten von Hand in mehreren Terminals zu starten ist fehleranfällig (Gazebo blockiert sein Terminal, `robot_state_publisher` muss laufen, die Controller dürfen erst nach dem Spawn kommen). Eine Launch-Datei erledigt das in *einem* Befehl und sequenziert die Schritte über Event-Handler:

```
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription, RegisterEventHandler
from launch.event_handlers import OnProcessExit
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import Command, PathJoinSubstitution
from launch_ros.actions import Node
from launch_ros.substitutions import FindPackageShare
from launch_ros.parameter_descriptions import ParameterValue

def generate_launch_description():
    pkg = FindPackageShare('p0a_arm')
    ros_gz_sim = FindPackageShare('ros_gz_sim')
    xacro_file = PathJoinSubstitution([pkg, 'urdf', 'arm.urdf.xacro'])
    robot_description = ParameterValue(Command(['xacro ', xacro_file]),
                                       value_type=str)

    rsp = Node(package='robot_state_publisher', executable='robot_state_publisher',
               parameters=[{'robot_description': robot_description,
                           'use_sim_time': True}]) # TF-Stamps = Gazebo-Zeit

    gz = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            PathJoinSubstitution([ros_gz_sim, 'launch', 'gz_sim.launch.py']),
            launch_arguments={'gz_args': '-r empty.sdf'}.items()) # -r: sofort laufen
    # /clock von Gazebo nach ROS bruecken ('[' = gz -> ROS) -- Zeitquelle fuer
    # alle use_sim_time-Knoten (controller_manager, rsp, RViz)
    clock_bridge = Node(package='ros_gz_bridge', executable='parameter_bridge',
                       arguments=['/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock'])
    spawn = Node(package='ros_gz_sim', executable='create',
                 arguments=['-topic', 'robot_description', '-name', 'rrarm'])
    jsb = Node(package='controller_manager', executable='spawner',
               arguments=['joint_state_broadcaster'])
    arm = Node(package='controller_manager', executable='spawner',
               arguments=['arm_controller'])
    return LaunchDescription([
        rsp, gz, clock_bridge, spawn,
        RegisterEventHandler(OnProcessExit(target_action=spawn, on_exit=[jsb])),
        RegisterEventHandler(OnProcessExit(target_action=jsb, on_exit=[arm])),
    ])
```

Listing 23.12: launch/gazebo.launch.py

Damit genügen **zwei Terminals**: eines startet alles, das zweite prüft die Controller und schickt eine Trajektorie.

```
cd ~/ros2_ws
colcon build --packages-select p0a_arm
source install/setup.bash
ros2 launch p0a_arm gazebo.launch.py
```

Listing 23.13: Terminal 1 – bauen, sourcen, alles starten

```
source ~/ros2_ws/install/setup.bash
ros2 control list_controllers          # beide sollten "active" sein
ros2 topic pub --once /arm_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory \
  '{joint_names: [joint1, joint2],
  points: [{positions: [0.8, -0.6], time_from_start: {sec: 2}}]}'
```

Listing 23.14: Terminal 2 – Controller prüfen und Arm kommandieren

Was dabei unter der Haube passiert. Die Launch-Datei ersetzt diese vier Einzelschritte – nützlich zum Verständnis, von Hand aber nicht nötig:

```
ros2 launch ros_gz_sim gz_sim.launch.py gz_args:"-r empty.sdf"
ros2 run ros_gz_sim create -topic robot_description -name rrarm
ros2 control load_controller --set-state active joint_state_broadcaster
ros2 control load_controller --set-state active arm_controller
```

Listing 23.15: Äquivalente Einzelbefehle

Was die Befehle und Flags tun.

- `ros_gz_sim gz_sim.launch.py gz_args:=empty.sdf` – startet Gazebo Harmonic mit einer leeren Welt.
- `ros_gz_sim create -topic robot_description` – spawnt das Modell aus dem (vom `robot_state_publisher`) veröffentlichten `robot_description`-Topic in die laufende Welt.
- `ros2 control load_controller -set-state active <name>` – lädt einen Controller *und* schaltet ihn in einem Schritt aktiv. Ohne `-set-state active` bliebe er nur konfiguriert (inaktiv) und würde keine Befehle umsetzen.
- `controller_manager spawner <name>` – die Launch-Variante davon: lädt, konfiguriert und aktiviert einen Controller in einem Schritt.
- **Reihenfolge:** erst `joint_state_broadcaster` (publiziert `/joint_states`), dann `arm_controller` (nimmt Trajektorien an).

```
src/p0a_arm/
|- urdf/arm.urdf.xacro          # angepasst: <gazebo>-Plugin-Block aktiviert
|- config/controllers.yaml      # neu
`- launch/gazebo.launch.py     # neu (Gazebo + rsp + spawn + Controller)
```

Listing 23.16: `ros2_ws-Delta` nach Schritt 4 (# neu / # angepasst)

23.3.5 Schritt 5 – Topics: kommandieren und auslesen

Erst direkt von der Kommandozeile, dann aus einem eigenen Knoten:

```
ros2 topic echo /joint_states          # Ist-Zustand mitlesen
ros2 topic pub --once /arm_controller/joint_trajectory \
  trajectory_msgs/msg/JointTrajectory \
  '{joint_names: [joint1, joint2],
  points: [{positions: [0.5, -0.8], time_from_start: {sec: 2}}]}'
```

Listing 23.17: Gelenke per Topic ansprechen

Als Knoten verpackt fährt `arm_commander` abwechselnd zwei Posen an, sodass eine sichtbare Bewegung entsteht.

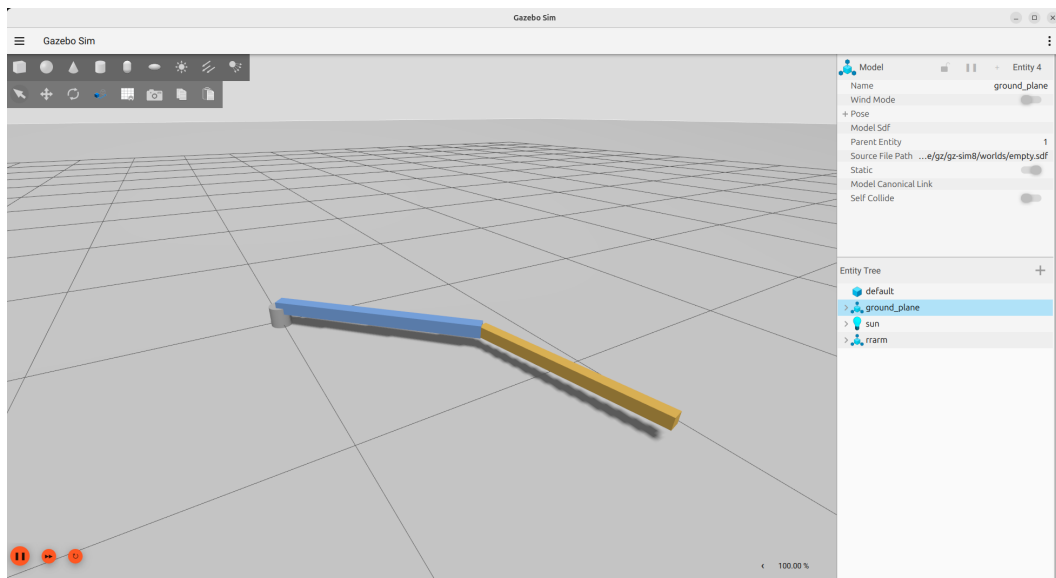


Abbildung 23.4: Der Arm in Gazebo Harmonic. Hier wirkt Schwerkraft: ohne aktiven Controller sinkt der Arm – mit aktivem `arm_controller` hält er die per Trajektorie kommandierte Stellung.

```
import rclpy
from rclpy.node import Node
from builtin_interfaces.msg import Duration
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint

class ArmCommander(Node):
    def __init__(self):
        super().__init__('arm_commander')
        self.pub = self.create_publisher(
            JointTrajectory, '/arm_controller/joint_trajectory', 10)
        self.poses = [[0.8, -0.6], [-0.8, 0.6]]  # zwei Zielposen
        self.idx = 0
        self.create_timer(3.0, self.tick)  # alle 3 s neues Ziel

    def tick(self):
        target = self.poses[self.idx]
        self.idx = (self.idx + 1) % len(self.poses)
        msg = JointTrajectory()
        msg.joint_names = ['joint1', 'joint2']
        p = JointTrajectoryPoint()
        p.positions = target
        p.time_from_start = Duration(sec=2)  # 2 s bis zum Ziel
        msg.points = [p]
        self.pub.publish(msg)

def main():
    rclpy.init()
    rclpy.spin(ArmCommander())
    rclpy.shutdown()
```

Listing 23.18: `p0a_arm/arm_commander.py`

Den Knoten in `setup.py` als Skript registrieren (sonst kennt `ros2 run` ihn nicht):

```
entry_points={
    'console_scripts': [
        'arm_commander = p0a_arm.arm_commander:main',
    ],
},
```

Listing 23.19: p0a_arm/setup.py – entry_points

Ausführen (drei Terminals). Damit sich etwas bewegt, muss der Gazebo-Stack aus Schritt 4 laufen – er stellt den `arm_controller`, der die Trajektorien ausführt.

```
cd ~/ros2_ws
colcon build --packages-select p0a_arm
source install/setup.bash
ros2 launch p0a_arm gazebo.launch.py
```

Listing 23.20: Terminal 1 – Gazebo-Stack (wie Schritt 4)

```
source ~/ros2_ws/install/setup.bash
ros2 run p0a_arm arm_commander
```

Listing 23.21: Terminal 2 – den Knoten starten

```
source ~/ros2_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix --share p0a_arm)/config/arm.rviz" \
    --ros-args -p use_sim_time:=true # mit Gazebo: sonst "No transform"
```

Listing 23.22: Terminal 3 (optional) – RViz eigenständig

RViz neben Gazebo – nicht über display.launch.py

Der Arm bewegt sich bereits sichtbar in **Gazebo**. Will man ihn *zusätzlich* in RViz sehen, RViz **eigenständig** starten (Terminal 3) – *nicht* über `display.launch.py`: dessen `joint_state_publisher_gui` würde mit dem `Gazebo-joint_state_broadcaster` um `/joint_states` konkurrieren. RViz zeigt die Bewegung dann über `/joint_states` → TF.

```
src/p0a_arm/
|- setup.py # angepasst: entry_points -> arm_commander
~- p0a_arm/arm_commander.py # neu
```

Listing 23.23: ros2_ws-Delta nach Schritt 5 (# neu / # angepasst)

23.3.6 Schritt 6 – Service: in eine benannte Pose fahren

Ein Service ist die richtige Wahl für einen kurzen Befehl mit klarer Antwort.

```
string name # "home" | "extended" | "folded"
---
bool ok
string message
```

Listing 23.24: srv/SetNamedPose.srv

Interface bauen. Eigene srv-Typen entstehen nur in einem ament_cmake-Paket. In p0a_arm_interfaces zwei Dinge ergänzen – die Generierung in CMakeLists.txt und die Laufzeit-Abhängigkeiten in package.xml:

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/SetNamedPose.srv"
)
ament_export_dependencies(rosidl_default_runtime)
```

Listing 23.25: p0a_arm_interfaces/CMakeLists.txt (Ergänzung)

```
<exec_depend>rosidl_default_runtime</exec_depend>
<!-- am Ende, direkt vor <export>: -->
<member_of_group>rosidl_interface_packages</member_of_group>
```

Listing 23.26: p0a_arm_interfaces/package.xml (Ergänzung)

Server-Knoten. Er bildet den Namen auf Gelenkwinkel ab, sendet die Trajektorie (wie in Schritt 5) und meldet Erfolg in der Antwort.

```

import rclpy
from rclpy.node import Node
from builtin_interfaces.msg import Duration
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from p0a_arm_interfaces.srv import SetNamedPose

class NamedPoseServer(Node):
    NAMED_POSES = {
        'extended': [0.0, 0.0],    # voll gestreckt
        'home':      [1.57, 0.0],  # 90-Grad-Ruhestellung
        'folded':    [0.0, 2.8],   # link2 zurueckgeklappt
    }

    def __init__(self):
        super().__init__('named_pose_server')
        self.pub = self.create_publisher(
            JointTrajectory, '/arm_controller/joint_trajectory', 10)
        self.create_service(SetNamedPose, '/arm/set_named_pose', self.on_request)

    def on_request(self, request, response):
        target = self.NAMED_POSES.get(request.name)
        if target is None:
            response.ok = False
            response.message = f"unbekannte Pose '{request.name}'"
            return response
        msg = JointTrajectory()
        msg.joint_names = ['joint1', 'joint2']
        p = JointTrajectoryPoint()
        p.positions = target
        p.time_from_start = Duration(sec=2)
        msg.points = [p]
        self.pub.publish(msg)
        response.ok = True
        response.message = f"fahre zu '{request.name}' -> {target}"
        return response

def main():
    rclpy.init()
    rclpy.spin(NamedPoseServer())
    rclpy.shutdown()

```

Listing 23.27: p0a_arm/named_pose_server.py

Callback nicht handle nennen

Die Callback-Methode heißt bewusst `on_request` und *nicht* `handle`: `rclpy.node.Node` besitzt bereits eine Eigenschaft `handle` (den internen `rcl`-Knoten-Handle). Eine eigene Methode `handle` überschreibt sie, und schon `super().__init__()` scheitert mit 'method' object does not support the context manager protocol. Gleiches gilt für andere Node-Member (`context`, `executor`, ...).

Im `p0a_arm`-Paket die Abhängigkeit (für den import) und den Einstiegspunkt eintragen:

```
<depend>p0a_arm_interfaces</depend>
```

Listing 23.28: p0a_arm/package.xml

```
'named_pose_server' = p0a_arm.named_pose_server:main',
```

Listing 23.29: p0a_arm/setup.py – entry_points

Bauen und ausführen. Das Interface-Paket muss *vor* p0a_arm gebaut werden; colcon ordnet das anhand der <depend>-Zeile automatisch.

```
cd ~/ros2_ws
colcon build --packages-select p0a_arm_interfaces p0a_arm
source install/setup.bash
ros2 launch p0a_arm gazebo.launch.py
```

Listing 23.30: Terminal 1 – bauen + Gazebo-Stack

```
source ~/ros2_ws/install/setup.bash
ros2 run p0a_arm named_pose_server
```

Listing 23.31: Terminal 2 – Server bereitstellen

```
source ~/ros2_ws/install/setup.bash
ros2 service call /arm/set_named_pose \
  p0a_arm_interfaces/srv/SetNamedPose "{name: folded}"
ros2 service call /arm/set_named_pose \
  p0a_arm_interfaces/srv/SetNamedPose "{name: extended}"
ros2 service call /arm/set_named_pose \
  p0a_arm_interfaces/srv/SetNamedPose "{name: home}"
```

Listing 23.32: Terminal 3 – Service aufrufen

```
src/
|- p0a_arm_interfaces/
|  |- srv/SetNamedPose.srv # neu
|  |- CMakeLists.txt      # angepasst: rosidl_generate_interfaces(srv)
|  `-- package.xml        # angepasst: rosidl_default_runtime + member_of_group
`-- p0a_arm/
    |- setup.py            # angepasst: entry_points -> named_pose_server
    |- package.xml         # angepasst: <depend>p0a_arm_interfaces</depend>
    `-- p0a_arm/named_pose_server.py # neu
```

Listing 23.33: ros2_ws-Delta nach Schritt 6 (# neu / # angepasst)

23.3.7 Schritt 7 – Inverse Kinematik allgemein: Denavit-Hartenberg + Jacobi

Für zwei Glieder gäbe es eine geschlossene Lösung (Kosinussatz) – doch die **lässt sich nicht auf komplexe Roboter übertragen**. Reale Arme (6DoF und mehr) löst man *numerisch*: Man beschreibt die Kinematik mit **Denavit-Hartenberg-Parametern** ($a_i, \alpha_i, d_i, \theta_i$), bildet die Vorwärtskinematik als Produkt der Gliedtransformationen und iteriert die IK über die **Jacobi-Matrix**. Genau das tun auch KDL und TRAC-IK hinter MoveIt 2. Wir schreiben es so, dass derselbe Code von 2 auf N Gelenke skaliert.

DH-Tabelle des Arms.

i	a_i	α_i	d_i	θ_i
1	l_1	0	0	θ_1 (Gelenk)
2	l_2	0	0	θ_2 (Gelenk)

Vorwärtskinematik. Jede Zeile liefert eine homogene Transformation; das Produkt ergibt die Endeffektor-Lage:

$$A_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad T_n^0 = \prod_{i=1}^n A_i, \quad \mathbf{p}(\boldsymbol{\theta}) = T_n^0[0:3, 3].$$

Numerische IK (Damped Least Squares). Mit dem Fehler $\mathbf{e} = \mathbf{p}^* - \mathbf{p}(\boldsymbol{\theta})$ und der Jacobi-Matrix $J = \partial \mathbf{p} / \partial \boldsymbol{\theta}$ iteriert man bis $\|\mathbf{e}\|$ klein ist:

$$\Delta \boldsymbol{\theta} = J^\top (J J^\top + \lambda^2 I)^{-1} \mathbf{e}, \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \Delta \boldsymbol{\theta}.$$

Die Dämpfung λ hält die Lösung auch nahe Singularitäten stabil.

```
import numpy as np

def dh(a, alpha, d, theta):
    ct, st, ca, sa = np.cos(theta), np.sin(theta), np.cos(alpha), np.sin(alpha)
    return np.array([[ct, -st*ca, st*sa, a*ct],
                    [st, ct*ca, -ct*sa, a*st],
                    [0, sa, ca, d],
                    [0, 0, 0, 1]])

# DH-Tabelle: (a, alpha, d) je Gelenk; theta ist die Gelenkvariable
DH = [(1.0, 0.0, 0.0), (0.8, 0.0, 0.0)] # l1, l2

def fk(q):
    T = np.eye(4)
    for (a, al, d), th in zip(DH, q):
        T = T @ dh(a, al, d, th)
    return T[:3, 3] # Endeffektor-Position (x, y, z)

def jacobian(q, eps=1e-6):
    p0 = fk(q); J = np.zeros((3, len(q)))
    for i in range(len(q)):
        dq = np.array(q, float); dq[i] += eps
        J[:, i] = (fk(dq) - p0) / eps
    return J

def ik_dls(target, q0, lam=0.1, iters=200, tol=1e-4):
    q = np.array(q0, float)
    for _ in range(iters):
        e = np.asarray(target, float) - fk(q)
        if np.linalg.norm(e) < tol:
            break
        J = jacobian(q)
        q += J.T @ np.linalg.solve(J @ J.T + lam**2*np.eye(3), e)
    return q, float(np.linalg.norm(e)) # Gelenkwinkel + Restfehler
```

Listing 23.34: ik.py – DH-Vorwärtskinematik + numerische IK (skaliert auf N Gelenke)

Numerische IK in der Praxis

Anders als die geschlossene Lösung hängt die numerische IK vom **Startwert** ab (sinnvoll: der aktuelle Gelenkzustand aus `/joint_states`), liefert *eine* von mehreren möglichen Lösungen und muss **Gelenklimits** nachgelagert prüfen. Die Dämpfung λ verhindert Ausreißer nahe Singularitäten – beim planaren Arm etwa, dass die z -Richtung gar nicht erreichbar ist (Zielebene $z = 0$). Die geschlossene 2-Glieder-Lösung eignet sich hervorragend als **Referenz**, um die numerische Implementierung gegenzuprüfen.


```
src/p0a_arm/
~ - p0a_arm/ik.py # neu (DH-Vorwaertskinematik + numerische Jacobi-IK)
```

Listing 23.35: ros2_ws-Delta nach Schritt 7 (# neu / # angepasst)

23.3.8 Schritt 8 – Action: kartesisches Ziel anfahren

Das Anfahren eines Punktes ist ein *länger laufender* Auftrag mit Fortschritt – also eine **Action**.

```
float64 x
float64 y
---
bool success
float64 final_error
---
float64 distance_to_goal
```

Listing 23.36: action/ReachPoint.action

Interface erweitern. Actions brauchen zusätzlich `action_msgs`. Die Action neben dem `srv` zur Generierung hinzufügen:

```
find_package(action_msgs REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/SetNamedPose.srv"
  "action/ReachPoint.action"
  DEPENDENCIES action_msgs
)
```

Listing 23.37: p0a_arm_interfaces/CMakeLists.txt (Ergänzung)

```
<depend>action_msgs</depend>
```

Listing 23.38: p0a_arm_interfaces/package.xml (Ergänzung)

Action-Server. Er löst die IK (Startwert: aktueller Gelenkzustand aus `/joint_states`), sendet die Trajektorie an den `arm_controller` und meldet die Restdistanz als Feedback, bis das Ziel erreicht ist.

```

import math, time
import rclpy
from rclpy.action import ActionServer
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
from rclpy.node import Node
from builtin_interfaces.msg import Duration
from sensor_msgs.msg import JointState
from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
from p0a_arm_interfaces.action import ReachPoint
from p0a_arm.ik import fk, ik_dls

JOINTS = ['joint1', 'joint2']

class ReachActionServer(Node):
    def __init__(self):
        super().__init__('reach_action_server')
        cb = ReentrantCallbackGroup()
        self.joint_pos = {}
        self.pub = self.create_publisher(
            JointTrajectory, '/arm_controller/joint_trajectory', 10)
        self.create_subscription(JointState, '/joint_states',
            self.on_joint_state, 10, callback_group=cb)
        ActionServer(self, ReachPoint, '/arm/reach_point',
            execute_callback=self.execute, callback_group=cb)

    def on_joint_state(self, msg):
        for n, p in zip(msg.name, msg.position):
            self.joint_pos[n] = p

    def current_q(self):
        return [self.joint_pos.get(j, 0.0) for j in JOINTS]

```

Listing 23.39: p0a_arm/reach_action_server.py (Kern)

... und die eigentliche Auftragsbearbeitung – IK lösen, Trajektorie senden, Feedback liefern – im execute-Callback:

```

def execute(self, goal_handle):
    target = [goal_handle.request.x, goal_handle.request.y, 0.0]
    q_goal, _ = ik_dls(target, self.current_q())      # IK loesen
    traj = JointTrajectory(joint_names=JOINTS)
    pt = JointTrajectoryPoint(positions=[float(v) for v in q_goal])
    pt.time_from_start = Duration(sec=2)
    traj.points = [pt]
    self.pub.publish(traj)                            # Trajektorie senden
    fb = ReachPoint.Feedback()
    deadline = time.time() + 6.0
    while time.time() < deadline:                      # Feedback-Schleife
        ee = fk(self.current_q())
        dist = math.hypot(target[0]-ee[0], target[1]-ee[1])
        fb.distance_to_goal = float(dist)
        goal_handle.publish_feedback(fb)
        if dist < 0.02:
            break
        time.sleep(0.1)
    ee = fk(self.current_q())
    res = ReachPoint.Result()
    res.final_error = float(math.hypot(target[0]-ee[0], target[1]-ee[1]))
    res.success = res.final_error < 0.05
    goal_handle.succeed()
    return res

def main():
    rclpy.init()
    ex = MultiThreadedExecutor()
    ex.add_node(ReachActionServer())
    ex.spin()

```

Listing 23.40: reach_action_server.py – execute + main

MultiThreadedExecutor + ReentrantCallbackGroup

Der `execute`-Callback blockiert in seiner Feedback-Schleife. Damit die `/joint_states`-Subscription *währenddessen* weiterläuft, liegen beide in einer `ReentrantCallbackGroup` unter einem `MultiThreadedExecutor`. Mit dem Standard-Executor bliebe der Gelenkzustand eingefroren – das Feedback käme nie voran.

Einstiegspunkt in `setup.py`:

```
'reach_action_server = p0a_arm.reach_action_server:main',
```

Listing 23.41: p0a_arm/setup.py – entry_points

Ausführen (vier Terminals). Gazebo-Stack starten, Action-Server bereitstellen, dann Ziele senden; RViz optional.

```

cd ~/ros2_ws
colcon build --packages-select p0a_arm_interfaces p0a_arm
source install/setup.bash
ros2 launch p0a_arm gazebo.launch.py

```

Listing 23.42: Terminal 1 – bauen + Gazebo-Stack

```
source ~/ros2_ws/install/setup.bash
ros2 run p0a_arm reach_action_server
```

Listing 23.43: Terminal 2 – Action-Server

```
source ~/ros2_ws/install/setup.bash
ros2 action send_goal /arm/reach_point \
  p0a_arm_interfaces/action/ReachPoint "{x: 1.2, y: 0.4}" --feedback
ros2 action send_goal /arm/reach_point \
  p0a_arm_interfaces/action/ReachPoint "{x: 0.6, y: 0.9}" --feedback
ros2 action send_goal /arm/reach_point \
  p0a_arm_interfaces/action/ReachPoint "{x: 1.5, y: -0.3}" --feedback
```

Listing 23.44: Terminal 3 – drei kartesische Ziele anfahren (mit Feedback)

```
source ~/ros2_ws/install/setup.bash
rviz2 -d "$(ros2 pkg prefix --share p0a_arm)/config/arm.rviz" \
  --ros-args -p use_sim_time:=true # mit Gazebo: sonst "No transform"
```

Listing 23.45: Terminal 4 (optional) – RViz eigenständig

Alle drei Ziele liegen im erreichbaren Ring ($0,2 \leq r \leq 1,8$ bei $l_1 = 1,0$, $l_2 = 0,8$); der Arm fährt sie nacheinander an, `-feedback` zeigt die schrumpfende Restdistanz, das Result `success + final_error`.

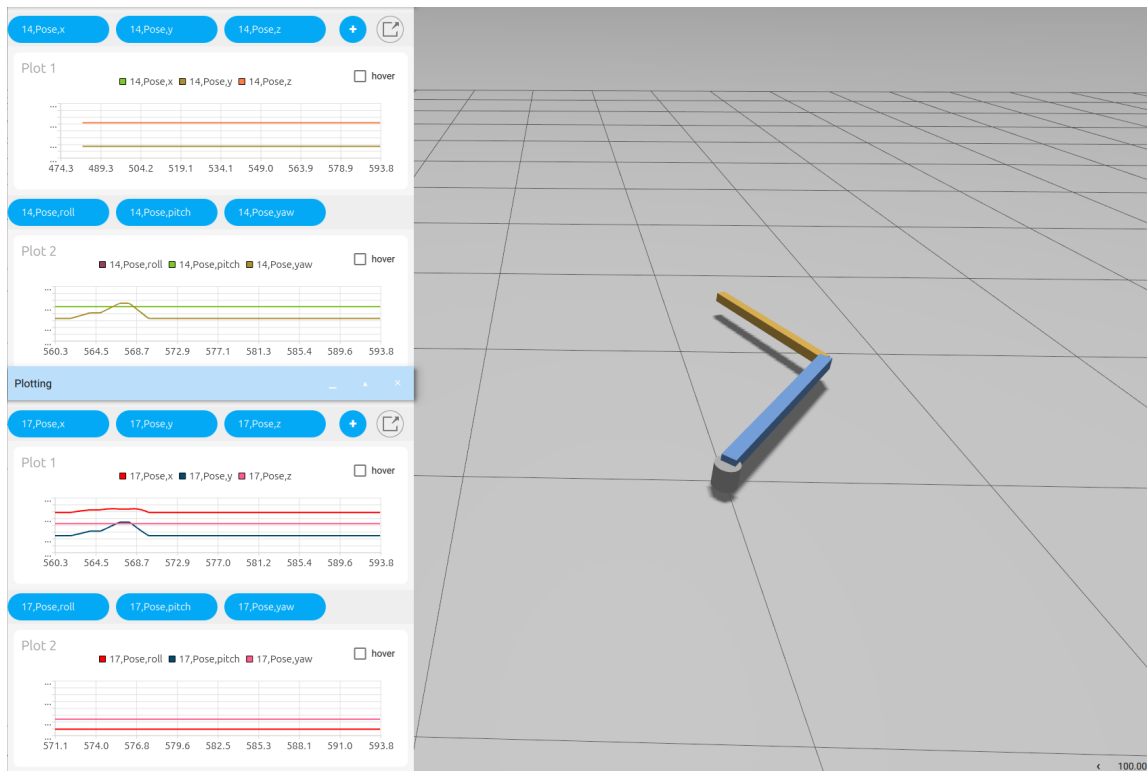


Abbildung 23.5: Projekt 0.A abgeschlossen: der Arm fährt das per Action gesendete kartesische Ziel an (numerische IK). Daneben die Verläufe von Position (x, y, z) und Orientierung (Roll/Pitch/Yaw) der beiden Glieder – die Bewegung ist in Gazebo, RViz und den Plots konsistent.

```
src/
|- p0a_arm_interfaces/
| |- action/ReachPoint.action # neu
| |- CMakeLists.txt           # angepasst: action + DEPENDENCIES action_msgs
| `-- package.xml             # angepasst: <depend>action_msgs</depend>
`-- p0a_arm/
    |- setup.py                # angepasst: entry_points -> reach_action_server
    `-- p0a_arm/reach_action_server.py # neu
```

Listing 23.46: ros2_ws-Delta nach Schritt 8 (# neu / # angepasst)

23.4 Implementierungs-Referenz: rclpy und Launch

Die *Konzepte* – was ein Topic, Service oder eine Action *ist* – stehen in den Grundlagenkapiteln. Hier geht es um die *Umsetzung*: welche rclpy-Funktionen man ruft und was sie tun. Projekte 0.A und 0.B legen dieses Handwerkszeug fest; die späteren Projekte setzen es voraus.

23.4.1 Das Knotengerüst

Jeder Python-Knoten erbt von `rclpy.node.Node`, wird nach `rclpy.init()` erzeugt und mit `rclpy.spin(node)` betrieben – `spin` ist der Ereignis-Loop, der *alle* Callbacks (Timer, Subscription, Service, Action) abarbeitet, bis Strg-C.

```
import rclpy
from rclpy.node import Node

class MyNode(Node):
    def __init__(self):
        super().__init__('my_node') # eindeutiger Knotenname im Graphen

def main():
    rclpy.init()                    # ROS-Kontext starten
    rclpy.spin(MyNode())            # blockiert, ruft Callbacks
    rclpy.shutdown()
```

Listing 23.47: Minimales Knotengerüst

Tutorial: [docs.ros.org – Publisher/Subscriber \(Python\)](https://docs.ros.org/en/Noetic-Release/Concepts/PublisherSubscriber/Python.html).

23.4.2 create_publisher – senden

`self.create_publisher(MsgType, '/topic', qos)` liefert ein Publisher-Objekt; `qos` ist meist die Queue-Tiefe (z. B. 10). Gesendet wird mit `publisher.publish(msg)`, wobei `msg` eine Instanz des Nachrichtentyps ist.

```
self.pub = self.create_publisher(JointTrajectory,
                                 '/arm_controller/joint_trajectory', 10)
self.pub.publish(msg) # msg: trajectory_msgs/JointTrajectory
```

23.4.3 create_subscription – empfangen

`self.create_subscription(MsgType, '/topic', callback, qos)`. Bei jeder eintreffenden Nachricht ruft ROS `callback(msg)` auf.

```
self.create_subscription(JointState, '/joint_states', self.on_joint_state, 10)

def on_joint_state(self, msg):    # msg: sensor_msgs/JointState
    for name, pos in zip(msg.name, msg.position):
        self.joint_pos[name] = pos
```

23.4.4 create_timer – periodisch handeln

`self.create_timer(periode_s, callback)` ruft `callback()` im festen Takt – so publiziert `arm_commander` alle 3 s ein neues Ziel.

23.4.5 create_service – auf Anfragen antworten

`self.create_service(SrvType, '/name', callback)`. Der Callback erhält `request` (Eingangsfelder) und ein bereits angelegtes `response`, das er füllt und zurückgibt.

```
self.create_service(SetNamedPose, '/arm/set_named_pose', self.on_request)

def on_request(self, request, response):    # NICHT 'handle' nennen (Kollision)!
    response.ok = True
    response.message = f"ok: {request.name}"
    return response
```

Tutorial: [docs.ros.org – Service/Client \(Python\)](https://docs.ros.org/en/Noetic/Service/Client/Python.html).

23.4.6 Interfaces definieren – srv und action

Eine `.srv`-Datei hat zwei Teile (Request / Response), getrennt durch `--`; eine `.action` drei (Goal / Result / Feedback):

```
string name          # Request (Client -> Server)
---
bool ok              # Response (Server -> Client)
string message
```

Listing 23.48: SetNamedPose.srv

```
float64 x            # Goal: was der Client sendet -> "{x: 1.2, y: 0.4}"
float64 y
---
bool success         # Result: Endergebnis
float64 final_error
---
float64 distance_to_goal # Feedback: laufender Zwischenstand
```

Listing 23.49: ReachPoint.action

ROS generiert daraus die Python-Typen `ReachPoint.Goal`, `ReachPoint.Result`, `ReachPoint.Feedback`. Das **Goal** sind genau die Felder vor dem ersten `--`. Tutorials: [Custom Interfaces](#), [Creating an Action](#).

23.4.7 ActionServer – langlaufende Aufträge umsetzen

`ActionServer(node, ActionType, '/name', execute_callback=..., callback_group=...)`. Der `execute_callback` bearbeitet ein Ziel und nutzt:

- `goal_handle.request` – die Goal-Felder (hier `.x`, `.y`).
- `goal_handle.publish_feedback(fb)` – Zwischenstand senden (`fb = ReachPoint.Feedback()`).
- `goal_handle.succeed()` bzw. `.abort()` – Ausgang markieren.
- `return result` – das `ReachPoint.Result()` an den Client.

```
from rclpy.action import ActionServer
ActionServer(self, ReachPoint, '/arm/reach_point',
              execute_callback=self.execute, callback_group=cb)

def execute(self, goal_handle):
    target = (goal_handle.request.x, goal_handle.request.y) # Goal lesen
    fb = ReachPoint.Feedback()
    fb.distance_to_goal = ...; goal_handle.publish_feedback(fb)
    goal_handle.succeed()
    res = ReachPoint.Result(); res.success = True
    return res
```

Listing 23.50: Aufbau eines Action-Servers (vgl. Schritt 8)

Tutorial: [docs.ros.org – Action Server/Client \(Python\)](https://docs.ros.org/en/Noetic-Release/Action-Server/Client-Python.html).

23.4.8 ReentrantCallbackGroup + MultiThreadedExecutor

Der `execute_callback` blockiert (Feedback-Schleife). Mit dem *Standard-Executor* läuft immer nur *ein* Callback – die `/joint_states`-Subscription käme währenddessen nie dran, der Gelenkzustand bliebe eingefroren. Abhilfe: beide Callbacks in eine **ReentrantCallbackGroup** legen (dürfen nebenläufig/wiedereintretend laufen) und mit einem **MultiThreadedExecutor** (mehrere Threads) spinnen.

```
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor

cb = ReentrantCallbackGroup()
self.create_subscription(JointState, '/joint_states', self.on_js, 10,
                        callback_group=cb)
# ... ActionServer(..., callback_group=cb)
ex = MultiThreadedExecutor(); ex.add_node(node); ex.spin()
```

Hintergrund: [docs.ros.org – About Executors](https://docs.ros.org/en/Noetic-Release/About-Executors.html).

23.4.9 Aufbau einer Launch-Datei

Eine Launch-Datei ist Python: `generate_launch_description()` gibt eine `LaunchDescription` mit einer Liste von Aktionen zurück.

- `Node(package=..., executable=..., name=..., parameters=[...], arguments=[...])` – startet einen Knoten. `parameters` setzt Knoten-Parameter (z. B. `{'use_sim_time': True}`); `arguments` sind Programm-Argumente (z. B. `['-d', 'rviz_config']`).
- **Substitutions** (erst beim Start aufgelöst): `FindPackageShare('pkg')` liefert den share-Pfad; `PathJoinSubstitution([...])` setzt Pfade zusammen; `Command(['xacro ', datei])` fügt die *Ausgabe* eines Programms ein (das erzeugte URDF); `ParameterValue(..., value_type=str)` typisiert den Wert.
- `IncludeLaunchDescription(PythonLaunchDescriptionSource(<datei>), launch_arguments={...}.items())` – eine *fremde* Launch-Datei einbinden (so starten wir Gazebo über `gz_sim.launch.py`).
- `RegisterEventHandler(OnProcessExit(target_action=A, on_exit=[B]))` – B erst starten, wenn A fertig ist: so laufen `Spawn` → `Broadcaster` → `Controller` in der richtigen Reihenfolge.

Tutorials: docs.ros.org – [Launch](#).

23.5 Häufige Fehler und Checkliste (Projekt 0.A)

Stolpersteine, die fast jeden treffen

- **Aus dem falschen Verzeichnis gestartet.** Pfade wie `src/p0a_arm/...` gelten ab `~/ros2_ws`, nicht ab `~/ros2_ws/src` – sonst entsteht `src/src/...`
- **URDF per `-p` an der Kommandozeile.** Das mehrzeilige XML bricht `rc1`; immer über die Launch-Datei übergeben.
- **`$(find p0a_arm)` vor dem Bauen.** Erst `colcon build + source`, sonst “package not found”.
- **`urdf//launch//config/` fehlen in `setup.py`.** Dann liegen sie nicht im `install/-`Baum und `ros2 launch` findet nichts – `data_files` ergänzen, neu bauen.
- **Terminal nicht neu gesourct.** Nach `apt install` oder `colcon build` ein neues Terminal öffnen oder `source ~/.bashrc`.
- **`gz / joint_state_publisher_gui` nicht gefunden.** Paket installieren (Installationskapitel) und ROS 2 sourcen.
- **Custom `srv/action` im Python-Paket.** Nicht möglich – Interfaces gehören ins `ament_cmake`-Paket `p0a_arm_interfaces`.
- **Methode wie ein Node-Member benannt** (`handle, context ...`). Überschreibt `rc1py-Interna` – Callbacks neutral benennen (`on_request`).
- **Link ohne `<inertial>` in Gazebo.** RViz verzeiht es, Gazebo lässt das Glied wegfallen oder wird instabil.
- **RViz zeigt nichts / “Frame [map] does not exist”.** *Fixed Frame* auf `world` stellen und ein `RobotModel-Display` (`/robot_description`, QoS *Transient Local*) hinzufügen – oder die fertige `config/arm.rviz` laden.
- **“No transform from [link1] to [world]”, Modell bewegt sich nicht (mit Gazebo).** Sim-Zeit-Problem. Zweierlei nötig: (1) `/clock` von Gazebo nach ROS brücken (`ros_gz_bridge`, siehe `gazebo.launch.py`) – fehlt sie, warnt der `controller_manager` “No clock received”; (2) `robot_state_publisher` und RViz mit `use_sim_time:=true` starten. Nur beides zusammen liefert konsistente TF-Zeitstempel.
- **`-build -type` mit Leerzeichen.** Es heißt `-build-type` (ein Token); sonst bricht `ros2 pkg create` mit “expected one argument”.
- **xacro: “module ‘xml’ has no attribute ‘parsers’”.** Irreführende Folgemeldung – tatsächlich wurde die Datei *nicht gefunden* (falscher Pfad bzw. falsches Arbeitsverzeichnis). Pfad prüfen, nicht die xacro-Meldung jagen.
- **`member_of_group` an falscher Stelle.** In der `Interface-package.xml` muss es *nach* den `depend`-Zeilen, direkt vor `<export>` stehen – sonst Schema-Fehler.
- **Action ohne `action_msgs`.** `rosidl_generate_interfaces` für `.action` braucht `find_package(action_msgs),` `DEPENDENCIES action_msgs` und `<depend>action_msgs</depend>`.
- **`display.launch.py` und Gazebo gleichzeitig.** Der darin enthaltene `joint_state_publisher_gui` konkurriert mit dem `Gazebo-joint_state_broadcaster` um `/joint_states` – neben Gazebo RViz eigenständig starten (`rviz2 -d ...`).

Schnelle Prüfbefehle

```
xacro src/p0a_arm/urdf/arm.urdf.xacro > /tmp/arm.urdf    # parst die URDF?
colcon build --packages-select p0a_arm                    # baut das Paket?
ros2 pkg prefix p0a_arm                                   # wird es gefunden?
ros2 node list ; ros2 topic list                           # laufen die Knoten?
rqt_graph                                                  # stimmt die Architektur?
```

23.6 Was du danach beherrschst

- **URDF/TF:** ein Robotermodell von Hand schreiben, in RViz prüfen.
- **Topics:** Soll-/Ist-Größen publizieren und mitlesen.
- **Services:** kurze Befehle mit Antwort (benannte Posen).
- **Actions:** länger laufende Aufträge mit Feedback (Ziel anfahren).
- **IK:** das allgemeine Verfahren – DH-Vorwärtskinematik plus numerische Jacobi-IK (Damped Least Squares) – selbst implementiert; direkt übertragbar auf 6-DoF-Roboter.
- **ros2_control + Gazebo:** dieselbe Controller-Schicht wie in den vier Hauptprojekten (Kapitel 21).

Kapitel 24

Projekt 0.B – Stereokamera, Objekterkennung und Kalman-Tracking

Ziel: Eine simulierte **Stereokamera** beobachtet einen **farbigen Ball**. Wir erkennen ihn im Bild (OpenCV), schätzen seine **3D-Position und Entfernung** durch eigene **Triangulation** und glätten bzw. prädictieren die Bahn mit einem selbst implementierten **Kalman-Filter**. Das ist die direkte Vorübung zu Projekt 2 (Tischtennisroboter, Kapitel 23 liefert den Bewegungsteil).

Was wir übernehmen – und was wir selbst bauen

Übernommen (Geometrie/Sensor): ein Kugel-Primitive als Ball und der Gazebo-Kamerasensor – kein CAD nötig. **Selbst implementiert:** die Farberkennung, die Stereo-Triangulation (Entfernung) und der Kalman-Filter. Theorie dazu in den Kapiteln 4 (Stereosehen) und 2 (Kalman/EKF) – hier setzen wir sie praktisch um.

24.1 ROS 2-Bausteine

Typ	Konkret (selbst implementiert, sofern nicht anders vermerkt)
Nodes	Gazebo-Stereokamera + ros_gz-Bildbrücke (Standard) · ball_detector (HSV-Erkennung L+R und Triangulation) · ball_tracker (Kalman + Vorhersage) · marker_viz (RViz-Marker)
Topics	/stereo/left/image, /stereo/right/image · /ball/point (gemessen, PointStamped) · /ball/estimate (KF) · /ball/prediction (nav_msgs/Path) · /ball/markers (MarkerArray) · /clock, /tf
Services	/tracker/reset (KF zurücksetzen, std_srvs/Trigger)
Parameter	HSV-Bereich (Ballfarbe) · Brennweite f , Basisbreite B , Hauptpunkt (c_x, c_y) · KF-Rauschen Q, R , Vorhersage-Horizont

24.2 rqt_graph (Zielbild)

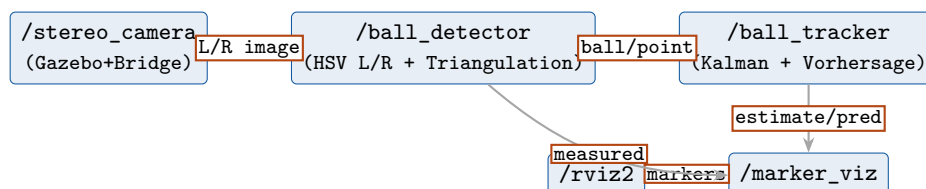


Abbildung 24.1: Projekt 0.B: Wahrnehmungskette von der Stereokamera über Erkennung und Triangulation bis zum Kalman-Tracker und der RViz-Darstellung.

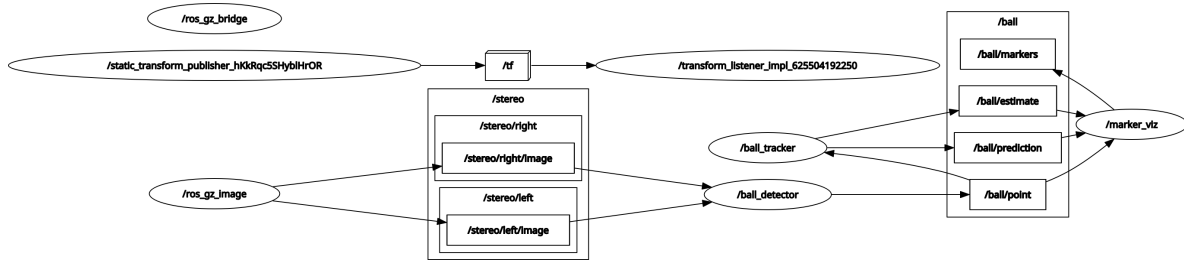


Abbildung 24.2: Tatsächlicher `rqt_graph` von Projekt 0.B: Die `ros_gz`-Bildbrücke speist `/stereo/left/right/image` in `ball_detector`, dieser publiziert `/ball/point`; `ball_tracker` erzeugt `/ball/estimate` und `/ball/prediction`, die `marker_viz` und (über `fov_viz`) RViz darstellen.

24.3 Schritt für Schritt

24.3.1 Schritt 0 – Paket anlegen (ein Workspace für alle Projekte)

Ein `colcon`-Workspace fasst beliebig viele Pakete; Projekt 0.B kommt deshalb als weiteres Paket in dasselbe `~/ros2_ws` neben die 0.A-Pakete – *kein* neuer Workspace nötig. **Lehre aus 0.A:** hier brauchen wir *keine* eigenen Interfaces; Standardnachrichten reichen (`geometry_msgs`, `nav_msgs/Path`, `visualization_msgs`, `std_srvs`). Das erspart das gesamte `rosidl/ament_cmake`-Drumherum.

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python p0b_stereo --license Apache-2.0 \
  --dependencies rclpy sensor_msgs geometry_msgs nav_msgs \
    visualization_msgs std_srvs cv_bridge message_filters
mkdir -p p0b_stereo/{worlds,launch,config,p0b_stereo}
```

Listing 24.1: Paket anlegen

```
src/p0b_stereo/ # neu -- ament_python, Standard-Messages (keine Interfaces!)
```

Listing 24.2: `ros2_ws`-Delta nach Schritt 0 (# neu)

24.3.2 Schritt 1 – Welt: Ball und Stereokamera

Zwei Kamerasensoren im Abstand der Basisbreite $B = 0,10$ m, Blick entlang $+x$, und ein orangefarbener Ball (dynamisch – er fällt und lässt sich in der GUI mit der Maus bewegen). Wichtig: `<optical_frame_id>` setzt den Rahmen, in dem wir später die 3D-Punkte veröffentlichen.

```

<model name="stereo_rig"><static>true</static><pose>0 0 1 0 0 0</pose>
  <link name="base">
    <visual name="body">      <!-- sichtbarer Koerper des Rigs in Gazebo -->
      <geometry><box><size>0.05 0.18 0.05</size></box></geometry></visual>
    <sensor name="left" type="camera">
      <pose>0 0.05 0 0 0 0</pose>      <!-- +B/2 -->
      <topic>/stereo/left/image</topic>
      <update_rate>30</update_rate><always_on>1</always_on>
      <camera><horizontal_fov>1.047</horizontal_fov>
        <image><width>640</width><height>480</height></image>
        <optical_frame_id>stereo_left_optical</optical_frame_id></camera>
    </sensor>
    <sensor name="right" type="camera">
      <pose>0 -0.05 0 0 0 0</pose>      <!-- -B/2 -->
      <topic>/stereo/right/image</topic>
      <update_rate>30</update_rate><always_on>1</always_on>
      <camera><horizontal_fov>1.047</horizontal_fov>
        <image><width>640</width><height>480</height></image>
        <optical_frame_id>stereo_right_optical</optical_frame_id></camera>
    </sensor>
  </link>
</model>

<model name="ball"><pose>3 0 1.5 0 0 0</pose>
  <link name="link">
    <inertial><mass>0.05</mass><inertia><ixx>7.2e-5</ixx><iyy>7.2e-5</iyy>
      <izz>7.2e-5</izz><ixy>0</ixy><ixz>0</ixz><iyz>0</iyz></inertia></inertial>
    <collision name="c"><geometry><sphere><radius>0.06</radius></sphere></geometry></collision>
    <visual name="v"><geometry><sphere><radius>0.06</radius></sphere></geometry>
      <material><ambient>1 0.25 0 1</ambient><diffuse>1 0.25 0 1</diffuse></material></visual>
    </link>
  </model>

```

Listing 24.3: worlds/stereo.sdf (Auszug – Ground/Sun/Plugins analog)

Die vollständige Datei (mit ground, sun und den Gazebo-system-Plugins inkl. Sensors/ogre2) liegt in worlds/stereo.sdf; gestartet wird sie über die Launch-Datei aus Schritt 6.

```
src/p0b_stereo/worlds/stereo.sdf    # neu (Stereo-Rig + Ball + Welt)
```

Listing 24.4: ros2_ws-Delta nach Schritt 1 (# neu)

SDF (0.B) vs. URDF/Xacro (0.A) – der Unterschied

URDF/Xacro (Projekt 0.A, arm.urdf.xacro) beschreibt *einen Roboter* als kinematischen Baum aus Links und Joints. Es ist ein *ROS-Format*: robot_state_publisher, ros2_control und RViz lesen es direkt; Xacro ist nur ein Makro-Präprozessor darüber. Sensoren und Physik müssen über <gazebo>-Erweiterungen angehängt werden.

SDF (Projekt 0.B, stereo.sdf) ist das *native Format von Gazebo* und beschreibt eine ganze *Welt*: Licht, Boden, mehrere Modelle, **Sensoren** (Kamera, IMU, LiDAR) und **System-Plugins** (Physik, Rendering) – alles erstklassig. ROS kennt SDF nicht direkt; die Verbindung entsteht über ros_gz-Brücken.

Faustregel: *einen Roboter* für ROS beschreiben → URDF/Xacro (wird beim Spawnen intern nach SDF gewandelt). *Eine Simulationswelt mit Sensoren* bauen → SDF. In 0.A war der Arm der Gegenstand (URDF, in Gazebo gespawnt); in 0.B ist die Kamera-Welt der Gegenstand (SDF), und die Wahrnehmungsknoten leben rein in ROS.

24.3.3 Schritt 2 – Bilder und Zeit nach ROS bruecken

Die Bilder bringt die `ros_gz_image`-Bildbrücke nach ROS; die Simulationszeit (`/clock`) braucht eine eigene Brücke – **exakt die Lehre aus 0.A**: ohne `/clock` stehen alle `use_sim_time`-Knoten ohne Uhr da.

```
ros2 run ros_gz_image image_bridge /stereo/left/image /stereo/right/image
ros2 run ros_gz_bridge parameter_bridge /clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock
```

Listing 24.5: Brücken (in der Launch-Datei aus Schritt 6 gebündelt)

24.3.4 Schritt 3 – Objekterkennung (OpenCV, HSV)

In beiden Bildern den Ball über seine Farbe finden und den Pixelmittelpunkt (u, v) bestimmen (Kern des `ball_detector`):

```
def detect(self, img_msg):
    img = self.bridge.imgmsg_to_cv2(img_msg, 'bgr8')
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    mask = cv2.inRange(hsv, self.lo, self.hi)      # self.lo/hi = HSV-Grenzen
    cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    if not cnts:
        return None
    (u, v), r = cv2.minEnclosingCircle(max(cnts, key=cv2.contourArea))
    return (float(u), float(v)) if r >= 3.0 else None
```

Listing 24.6: HSV-Erkennung (Methode `detect`)

24.3.5 Schritt 4 – Triangulation und der vollständige Detektor

Bei achsparalleler Stereoanordnung ist die Disparität $d = |u_L - u_R|$; mit Brennweite f (Pixel) und Basisbreite B :

$$Z = \frac{f B}{d}, \quad X = \frac{(u_L - c_x) Z}{f}, \quad Y = \frac{(v_L - c_y) Z}{f}.$$

f, c_x, c_y berechnen wir aus den bekannten Kameraparametern ($f = \frac{w/2}{\tan(\text{hfov}/2)}$) – so ist *keine* `camera_info`-Brücke nötig (eine Fehlerquelle weniger). Beide Bilder werden per `ApproximateTimeSynchronizer` zum selben Zeitpunkt zusammengeführt:

```

class BallDetector(Node):
    def __init__(self):
        super().__init__('ball_detector')
        w, h, hfov = 640, 480, 1.047
        self.f = (w/2.0) / np.tan(hfov/2.0)          # Brennweite [px]
        self.cx, self.cy, self.B = w/2.0, h/2.0, 0.10
        self.frame_id = 'stereo_left_optical'
        self.lo = np.array([3, 120, 70]); self.hi = np.array([25, 255, 255])
        self.bridge = CvBridge()
        self.pub = self.create_publisher(PointStamped, '/ball/point', 10)
        left = Subscriber(self, Image, '/stereo/left/image')
        right = Subscriber(self, Image, '/stereo/right/image')
        self.sync = ApproximateTimeSynchronizer([left, right], 10, 0.05)
        self.sync.registerCallback(self.on_pair)

    def on_pair(self, left_msg, right_msg):
        L, R = self.detect(left_msg), self.detect(right_msg)
        if L is None or R is None: return
        (uL, vL), (uR, _) = L, R
        d = abs(uL - uR)
        if d < 1e-3: return                      # Objekt zu weit / kein Match
        Z = self.f * self.B / d
        X = (uL - self.cx) * Z / self.f
        Y = (vL - self.cy) * Z / self.f
        msg = PointStamped()
        msg.header.stamp = left_msg.header.stamp
        msg.header.frame_id = self.frame_id
        msg.point.x, msg.point.y, msg.point.z = float(X), float(Y), float(Z)
        self.pub.publish(msg)

```

Listing 24.7: p0b_stereo/ball_detector.py (Kern)

Im echten Knoten sind HSV, B und Brennweite über `declare_parameter` einstellbar. Vertiefung: Kapitel 4, 12.

```

src/p0b_stereo/
|- setup.py          # angepasst: entry_points -> ball_detector
`- p0b_stereo/ball_detector.py # neu (Erkennung + Triangulation)

```

Listing 24.8: ros2_ws-Delta nach Schritt 4 (# neu / # angepasst)

24.3.6 Schritt 5 – Kalman-Tracker mit Vorhersage

Lineares Modell konstanter Geschwindigkeit, Zustand $\mathbf{x} = [X, Y, Z, \dot{X}, \dot{Y}, \dot{Z}]^\top$:

$$\hat{\mathbf{x}}^- = F\hat{\mathbf{x}}, \quad P^- = FPF^\top + Q; \quad K = P^-H^\top(HP^-H^\top + R)^{-1}, \quad \hat{\mathbf{x}} = \hat{\mathbf{x}}^- + K(\mathbf{z} - H\hat{\mathbf{x}}^-).$$

```
def on_point(self, msg):
    t = msg.header.stamp.sec + msg.header.stamp.nanosec*1e-9
    z = np.array([msg.point.x, msg.point.y, msg.point.z])
    if not self.initialized:
        self.x[:3] = z; self.initialized = True; self.last_t = t; return
    dt = max(1e-3, t - self.last_t); self.last_t = t
    F = self._F(dt)
    self.x = F @ self.x                                # Praediktion
    self.P = F @ self.P @ F.T + self.Q
    S = self.H @ self.P @ self.H.T + self.R            # Korrektur
    K = self.P @ self.H.T @ np.linalg.inv(S)
    self.x = self.x + K @ (z - self.H @ self.x)
    self.P = (np.eye(6) - K @ self.H) @ self.P
    self._publish(msg.header.stamp, dt)
```

Listing 24.9: p0b_stereo/ball_tracker.py (Kern)

_publish sendet /ball/estimate und integriert den Zustand *ohne* Messung N Schritte vorwärts – das ergibt /ball/prediction (nav_msgs/Path), die Grundlage fürs Abfangen in Projekt 2. Ein /tracker/reset (std_srvs/Trigger) nullt den Filter.

```
src/p0b_stereo/
|- setup.py                # angepasst: entry_points -> ball_tracker
`- p0b_stereo/ball_tracker.py # neu (Kalman + Vorhersage + reset-Service)
```

Listing 24.10: ros2_ws-Delta nach Schritt 5 (# neu / # angepasst)

24.3.7 Schritt 6 – Marker in RViz und alles starten

marker_viz macht Messung (rot), Schätzung (grün) und Vorhersage (blaue Linie) als MarkerArray sichtbar:

```
def on_meas(self, msg):    # rote Kugel an der gemessenen Position
    self._emit(self._sphere('measured', msg.point, (1, 0.1, 0),
                             msg.header.frame_id, msg.header.stamp, 0.08))

def on_pred(self, msg):    # blaue Linie entlang der Vorhersage
    m = Marker(); m.header = msg.header; m.ns = 'prediction'
    m.type = Marker.LINE_STRIP; m.action = Marker.ADD
    m.scale.x = 0.02; m.color.b = 1.0; m.color.a = 1.0
    m.points = [ps.pose.position for ps in msg.poses]
    self._emit(m)
```

Listing 24.11: p0b_stereo/marker_viz.py (Kern)

Eine Launch-Datei startet alles in *einem* Befehl – Gazebo, /clock-Brücke, Bildbrücke, eine statische TF (damit der Kamerarahmen in RViz existiert – Lehre aus 0.A), die drei Knoten und RViz, alle mit use_sim_time:

```

return LaunchDescription([
    gz,          # ros_gz_sim + worlds/stereo.sdf (gz_args: '-r')
    clock_bridge, # /clock (gz -> ROS)
    image_bridge, # /stereo/left/right/image
    static_tf,    # world -> stereo_left_optical
    detector, tracker, viz, # parameters=[{'use_sim_time': True}]
    rviz,         # -d config/stereo.rviz
])

```

Listing 24.12: launch/stereo.launch.py (Knotenliste)

```

cd ~/ros2_ws
colcon build --packages-select p0b_stereo
source install/setup.bash
ros2 launch p0b_stereo stereo.launch.py

```

Listing 24.13: Terminal 1 – bauen, sourcen, alles starten

Die Launch-Datei startet *auch* die beiden Brücken aus Schritt 2 (Bild + /clock). Startet man die Knoten *einzeln* (z. B. zum Debuggen), müssen diese Brücken in einem eigenen Terminal laufen – aber **nicht zusätzlich** zur Launch, sonst doppelte Publisher:

```

ros2 run ros_gz_image image_bridge /stereo/left/image /stereo/right/image
ros2 run ros_gz_bridge parameter_bridge /clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock

```

Listing 24.14: Brücken (nur bei manuellem Start nötig)

```

source ~/ros2_ws/install/setup.bash
ros2 topic echo /ball/point # gemessene 3D-Position
ros2 service call /tracker/reset std_srvs/srv/Trigger {}

```

Listing 24.15: Terminal 2 – mitlesen / Filter zuruecksetzen

Tip: Den Ball in Gazebo mit der Maus greifen und bewegen – Messung, Schätzung und Vorhersage folgen in RViz. **Sichtfeld im Blick behalten:** RViz zeigt die beiden Kamerabilder (/stereo/left/right/image); der Ball wird nur verfolgt, solange er in *beiden* sichtbar ist. Verlässt er das Bild, erkennt ball_detector nichts mehr und die Marker bleiben auf der letzten Position stehen – kein Fehler. Das schwarze Rig-Kästchen zeigt die Kameraposition in Gazebo.

```

src/p0b_stereo/
|- setup.py          # angepasst: entry_points -> marker_viz, data_files
|- p0b_stereo/marker_viz.py # neu (MarkerArray fuer RViz)
|- launch/stereo.launch.py # neu (alles in einem Start)
`- config/stereo.rviz   # neu (Fixed Frame, MarkerArray, Image)

```

Listing 24.16: ros2_ws-Delta nach Schritt 6 (# neu / # angepasst)

24.4 Implementierungs-Referenz: Parameter und Synchronisation

Die rclpy-Grundlagen (Publisher, Subscription, Service, Launch) stehen in der Referenz von Projekt 0.A. In 0.B kommen zwei neue Bausteine dazu.

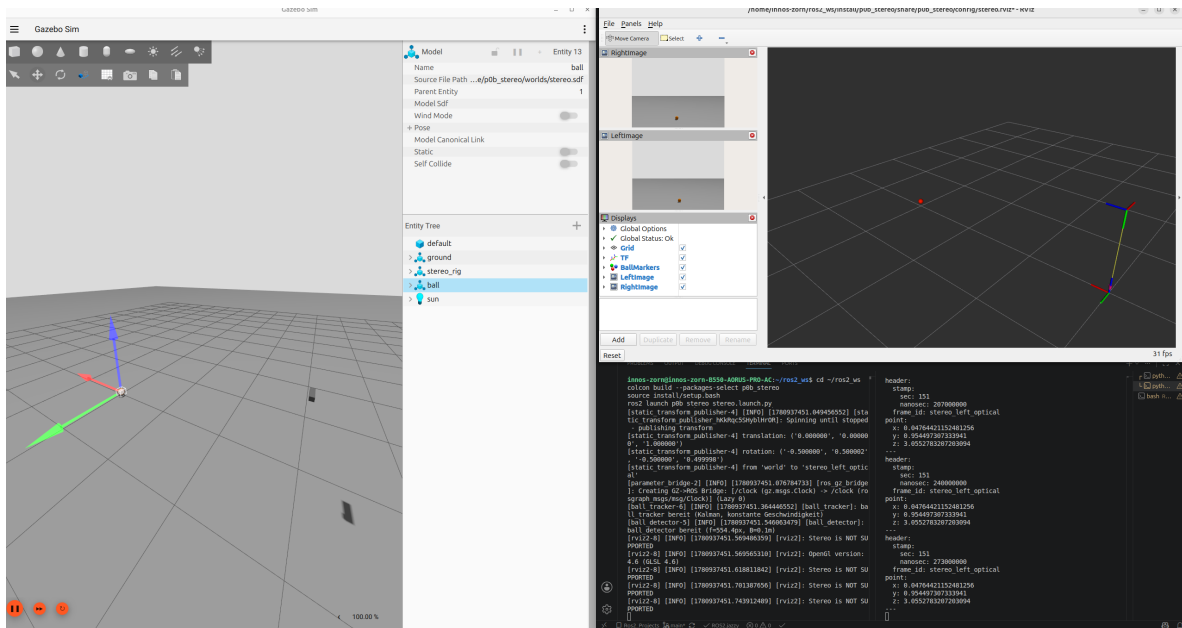


Abbildung 24.3: Projekt 0.B in Betrieb: links Gazebo mit Stereo-Rig und Ball, rechts RViz mit den Markern (gemessen rot, Kalman-Schätzung grün, Vorhersage blau) und den beiden Kamerabildern; im Terminal die `/ball/point`-Ausgabe. Der Filter glättet das Messrauschen und schätzt voraus.

24.4.1 Parameter – `declare_parameter` / `get_parameter`

Statt Werte (HSV-Bereich, Basisbreite, Brennweite) fest zu verdrahten, deklariert man sie als *Parameter*: so sind sie beim Start oder zur Laufzeit änderbar, ohne den Code anzufassen.

```
self.declare_parameter('baseline', 0.10)      # Name + Default(-typ)
B = self.get_parameter('baseline').value      # Wert auslesen
```

Gesetzt wird z. B. in der Launch-Datei (`parameters=[{'baseline': 0.12}]`) oder per CLI (`-ros-args -p baseline:=0.12`). Tutorial: [docs.ros.org – Parameters in a Class \(Python\)](https://docs.ros.org/en/Noetic/Parameters-in-a-Class-Python).

24.4.2 Zwei Topics zeitlich koppeln – `message_filters`

Die Triangulation braucht das linke *und* rechte Bild *desselben* Moments. Ein gewöhnliches `create_subscription` pro Kamera liefert die Bilder unabhängig – man wüsste nicht, welche zusammengehören. Der `ApproximateTimeSynchronizer` verknüpft beide Subscriptions und ruft den Callback erst, wenn ein zeitlich passendes *Paar* vorliegt:

```
from message_filters import Subscriber, ApproximateTimeSynchronizer
left = Subscriber(self, Image, '/stereo/left/image')
right = Subscriber(self, Image, '/stereo/right/image')
sync = ApproximateTimeSynchronizer([left, right], queue_size=10, slop=0.05)
sync.registerCallback(self.on_pair) # on_pair(left_msg, right_msg)
```

`slop` ist die maximal erlaubte Zeitdifferenz (s) zwischen den beiden Zeitstempeln. Doku: [docs.ros.org – message_filters](https://docs.ros.org/en/Noetic/Message-Filters).

24.5 Sichtfeld visualisieren und Sim-to-Real

24.5.1 Wie das Sichtfeld definiert ist

Jede Kamera in der SDF hat `<horizontal_fov>` (horizontaler Öffnungswinkel) und `<image><width/height>`. Daraus folgt die gesamte Abbildungsgeometrie:

- **Brennweite (Pixel):** $f = \frac{w/2}{\tan(hfov/2)}$ – genau die Größe, die `ball_detector` zur Triangulation berechnet ($640 \text{ px}, 60^\circ \Rightarrow f \approx 554$).
- **Vertikaler Öffnungswinkel** über das Seitenverhältnis: $vfov = 2 \arctan(\tan(hfov/2) \cdot h/w)$.
- **Hauptpunkt** $(c_x, c_y) = (w/2, h/2)$ – ideale Kamera, ohne Verzeichnung.

Bisher *sehen* wir das Sichtfeld nur indirekt über die beiden Kamerabilder.

24.5.2 Das Frustum als Pyramide zeigen (fov_viz)

Der Knoten `fov_viz` macht das Sichtfeld im 3D sichtbar: pro Kamera eine Pyramide (Apex am Kamerazentrum, Grundfläche auf der Fernebene) als `LINE_LIST`-Marker auf `/stereo/fov_markers`. Die Ecken folgen direkt aus `hfov`, Seitenverhältnis und Reichweite:

```
tan_h = np.tan(hfov/2.0); tan_v = tan_h * h/w          # Halbwinkel
X, Y = far*tan_h, far*tan_v                            # halbe Fernflaeche
corners = [Point(x=apex_x + sx*X, y=sy*Y, z=far)        # optisch: x rechts, y unten
            for sx, sy in [(-1,-1), (1,-1), (1,1), (-1,1)]]
# LINE_LIST: Apex->4 Ecken + Fernrechteck; linke Kamera apex_x=0, rechte apex_x=B
```

Listing 24.17: `fov_viz.py` (Kern der Frustum-Berechnung)

In `RViz` erscheinen zwei überlappende Pyramiden (grün = links, blau = rechts). Ihr *Schnittvolumen* ist der Bereich, in dem *beide* Kameras den Ball sehen – nur dort kann trianguliert werden. `fov_viz` ist in `stereo.launch.py` enthalten und als *CameraFOV*-Display in `stereo.rviz` eingebunden.

```
src/p0b_stereo/
|- p0b_stereo/fov_viz.py      # neu (Frustum-Marker beider Kameras)
|- setup.py                  # angepasst: entry_points -> fov_viz
|- launch/stereo.launch.py    # angepasst: fov_viz gestartet
`- config/stereo.rviz         # angepasst: CameraFOV- + RightImage-Display
```

Listing 24.18: `ros2_ws-Delta` (# neu / # angepasst)

Damit das in Simulation Entwickelte später auf einer echten USB-Stereokamera (z. B. einem ELP-Dual-Lens-Modul mit Global-Shutter) läuft, gleicht man drei Größen mit dem realen Gerät ab – dann teilen Sim und Realität dieselbe Geometrie:

1. **Öffnungswinkel/Brennweite:** `<horizontal_fov>` auf den Wert des realen Objektivs setzen (Datenblatt oder Kalibrierung, Kapitel 12) – damit stimmt f .
2. **Auflösung:** `<image><width/height>` auf die reale Sensorauflösung pro Auge.
3. **Basisbreite B :** den Kameraabstand in der SDF (`<pose>`) und den `baseline`-Parameter auf den realen Linsenabstand des Moduls.

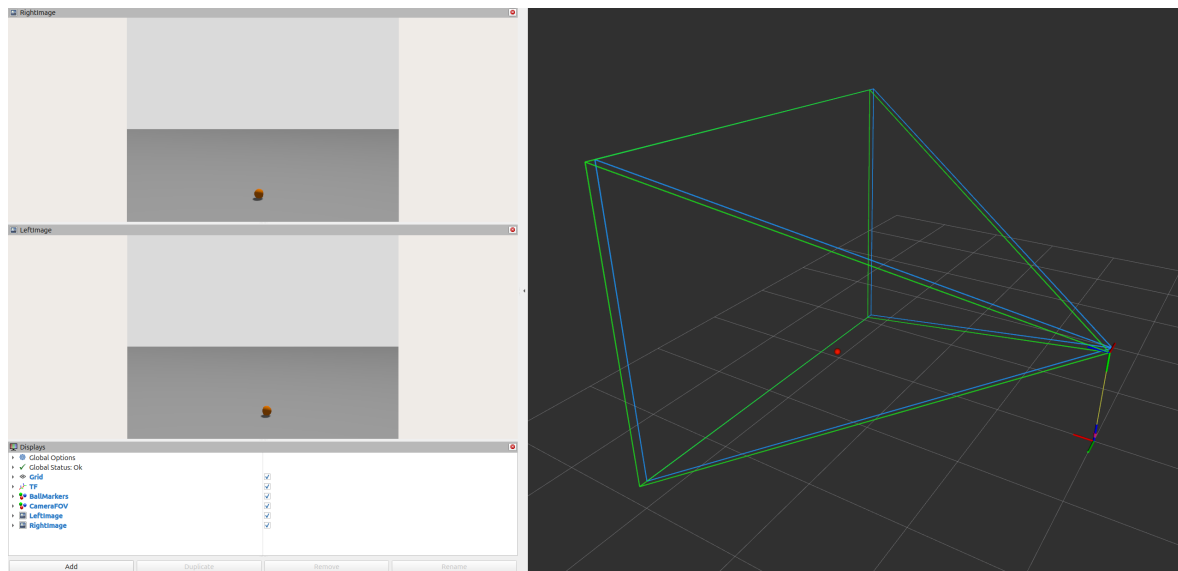


Abbildung 24.4: Die von `fov_viz` gezeichneten Sichtfeld-Pyramiden beider Kameras in RViz (grün = links, blau = rechts). Das Überlappungsvolumen ist der Bereich, in dem *beide* Kameras den Ball sehen – nur dort liefert die Triangulation eine 3D-Position.

Was die Simulation (noch) nicht abbildet

Reale Module haben **Linsenverzeichnung**: Gazebo kann sie über `<distortion>` (Brown-Conrady $k_1, k_2, \dots$) nachbilden – sauberer ist, *beide* Seiten zu kalibrieren und die Bilder zu entzerren (Kapitel 12). Ein **Global-Shutter** (wie beim genannten Modul) passt gut zum einfachen Pinhole-Modell; ein Rolling-Shutter erzeugt bei schnellen Bällen Schräglagen, die wir hier nicht simulieren. Auch reale Belichtung und Bildrauschen fehlen – die HSV-Schwellen müssen am Ende an echten Bildern nachjustiert werden.

24.6 Häufige Fehler und Checkliste (Projekt 0.B)

Stolpersteine – vieles aus 0.A übertragbar

- **/clock nicht gebrückt / use_sim_time fehlt.** Wie in 0.A: ohne /clock-Brücke und `use_sim_time:=true` fehlt die gemeinsame Zeit – RViz zeigt nichts oder “No transform”.
- **Kamerarahmen existiert nicht in RViz.** `<optical_frame_id>` in der SDF setzen *und* eine statische TF (`world→stereo_left_optical`) publizieren – sonst “Fixed Frame does not exist”.
- **Kein Kamerabild (EGL/Headless).** Die Gazebo-Sensors brauchen einen funktionierenden Renderer (`ogre2`); auf der RTX 3060 hilft die EGL-Einstellung aus dem Simulationskapitel – sonst bleiben die Bild-Topics leer.
- **HSV-Bereich zu eng/weit.** Kein Blob $\Rightarrow$ keine Punkte. Mit `rqt_image_view` und der Maske abstimmen; Beleuchtung der Welt beachten.
- **Disparität ≤ 0 .** Bei vertauschter Links/Rechts-Zuordnung wird d negativ; wir nehmen $|u_L - u_R|$ und verwerfen $d \approx 0$ (Objekt zu weit).
- **Bilder nicht synchron.** Ohne `ApproximateTimeSynchronizer` trianguliert man Links/Rechts aus verschiedenen Frames – springende Tiefe.
- **NumPy-Skalare in ROS-Felder.** `msg.point.x` erwartet ein Python-float – mit `float(...)` casten.
- **Unnötige Custom-Interfaces.** Standardnachrichten nutzen (`geometry_msgs`, `nav_msgs/Path`, `visualization_msgs`) – spart das `rosidl-Drumherum` aus 0.A.
- **“Stereo is NOT SUPPORTED” von RViz.** Harmlos – gemeint ist OpenGL-3D-Brillen-Stereo, nicht die Stereokamera. Ignorieren.
- **Marker stehen still / kein /ball/point mehr.** Der Ball ist nicht in *beiden* Kamerabil-dern. Die Image-Panels in RViz beobachten und den Ball im gemeinsamen Sichtfeld halten – erwartetes Verhalten, kein Bug.

Schnelle Prüfbefehle

```
ros2 topic hz /stereo/left/image      # liefert die Kamera Bilder?
ros2 topic echo /ball/point           # wird erkannt + trianguliert?
ros2 topic echo /ball/estimate        # schätzt der Kalman-Filter?
rqt_graph                             # haengt die Kette zusammen?
```

24.7 Was du danach beherrschst

- **Sensorik in Simulation:** Gazebo-Kamerabilder über `ros_gz` nach ROS 2 bringen.
- **Bildverarbeitung:** ein Objekt per `OpenCV/cv_bridge` robust erkennen.
- **Stereo-Geometrie:** Entfernung aus Disparität selbst triangulieren.
- **Zustandsschätzung:** einen Kalman-Filter implementieren, der glättet *und* prädiziert (Kapitel 2).
- **Visualisierung:** Mess- und Schätzgrößen als RViz-Marker prüfen – die direkte Vorstufe zu Projekt 2.

Teil VII

Die vier Projekte: Architektur und

rqt_graph

Kapitel 25

Projekt 1 – Stepper-Roboter mit Trajektorienbefehl vom PC

Ziel: Vom externen PC eine gewünschte Trajektorie senden, der Roboter führt sie aus. Da es ein langer Auftrag mit Fortschritt/Abbruch ist, ist die natürliche Schnittstelle eine **Action**.

25.1 ROS 2-Bausteine

Typ	Konkret
Nodes	trajectory_commander (PC, Action-Client) · controller_manager + joint_trajectory_controller + joint_state_broadcaster · Hardware-Interface (Sim: gz_ros2_control; Real: micro-ROS→ESP32)
Actions	control_msgs/FollowJointTrajectory (Goal: Bahn; Feedback: Ist-Position; Result: Erfolg)
Topics	/joint_states (Encoder/StallGuard) · /joint_trajectory_controller/... · /tf
Services	Controller laden/umschalten (/controller_manager/switch_controller)
Parameter	Gelenklimits, Getriebeübersetzung, Reglerverstärkungen

25.2 rqt_graph (vereinfacht)

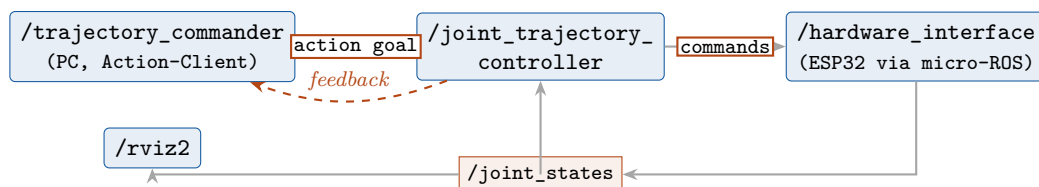


Abbildung 25.1: Projekt 1: Der PC schickt eine Trajektorie als Action-Goal; der Controller kommandiert das Hardware-Interface (ESP32) und liest /joint_states zur Regelung zurück.

Datenfluss: PC → Action-Goal (Bahn) → joint_trajectory_controller interpoliert → Sollwerte ans Hardware-Interface → ESP32 erzeugt Schritte am TMC2209, liest StallGuard/Encoder → /joint_states zurück → Regelschleife schließt sich. In Simulation ersetzt gz_ros2_control den ESP32 – der PC-Code bleibt unverändert.

Kapitel 26

Projekt 2 – Tischtennisroboter

Ziel: Ballflug per Stereokamera + Kalman schätzen, Roboter fährt an die Abschlagposition und schätzt Abschlagzeitpunkt und -trajektorie.

Grundlagen aus den Einstiegsprojekten werden vorausgesetzt

Die *Bausteine* sind in den Einstiegsprojekten eingeführt und werden hier nur referenziert: **Stereo-Triangulation und Kalman-Tracking** in Projekt 0.B (Kap. 24); **ros2_control, URDF/Xacro, Topics, Services und Actions** in Projekt 0.A (Kap. 23). Dieses Kapitel gliedert sich in **Vorüberlegungen** (Architektur, Mechanik, Simulierbarkeit, CAD/URDF) und **Vorgehensweise** (was wir in welcher Reihenfolge umsetzen).

26.1 Vorüberlegungen

Bevor gebaut wird, klären wir die Software-Bausteine, wo gerechnet wird, den mechanischen Aufbau und seine Grenzen in der Simulation sowie die nebenläufige Ablauflogik.

26.1.1 ROS 2-Bausteine

Typ	Konkret
Nodes	<code>stereo_camera</code> (Treiber) · <code>stereo_image_proc</code> (Disparität/Tiefe) · <code>ball_detector</code> (Farb-/Blob-Erkennung → 3D-Position) · <code>ball_tracker</code> (EKF, Flugvorhersage) · <code>strike_planner</code> (Abfangpunkt + Zeit + Schlagbahn) · <code>robot_controller</code> (Projekt-1-Stack)
Topics	<code>/left/image_raw</code> , <code>/right/image_raw</code> · <code>/points2</code> · <code>/ball/position</code> · <code>/ball/predicted_trajectory</code> · <code>/robot/target_pose</code>
Actions	<code>FollowJointTrajectory</code> (Schlagbewegung)
Parameter	Kamerakalibrierung, Ballfarbe/-radius, EKF- Q/R , Tischgeometrie

26.1.2 Architektur und Wahrnehmungskette

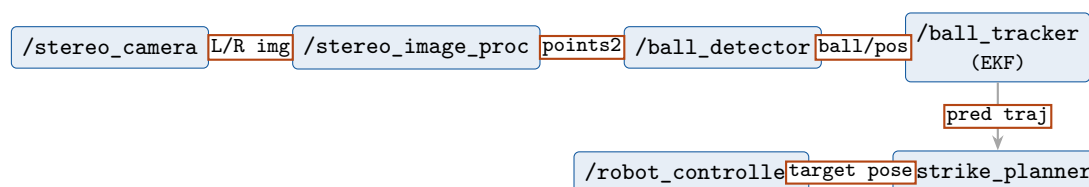


Abbildung 26.1: Wahrnehmungskette von der Stereokamera über den EKF bis zum Schlagplaner und Roboter.

Kernidee: Der EKF (Kapitel 2) bekommt die verrauschten 3D-Ballpositionen und extrapoliert per reiner Prädiktion bis zur Schlagebene; daraus folgen Abfangpunkt und -zeit. **Hauptsächlich reales Problem:** Latenz und Timing – in der Simulation triviale Zeit, real kritisch.

26.1.3 Kamera-Architektur – wo wird gerechnet?

Lohnt es sich, die Stereokamera an den ESP32 / Raspberry Pi Zero zu hängen und die Rohdaten per ROS-Service oder einfachem UDP/WLAN weiterzuleiten – oder entstehen Zeit-/Rechenleistungsprobleme?

26.1.3.1 Die Bandbreitenrechnung entscheidet

Die synchronisierte Stereokamera liefert ein side-by-side-Bild (z. B. $2 \times 1280 \times 720$). Was das pro Sekunde bedeutet:

Datenstrom	pro Frame	bei 60 fps
Stereo roh, YUYV (2 B/px), 2560×720	3,69 MB	≈ 1769 Mbit/s
Stereo MJPEG-komprimiert ($\approx 1:10$)	0,37 MB	≈ 177 Mbit/s
Nur 3D-Ballposition (Header + 3 Floats)	≈ 80 B	$\approx 0,1$ Mbit/s

Dem gegenüber die realistischen Linkkapazitäten:

Verbindung	real nutzbar
ESP32 WLAN (TCP, real)	$\approx 10\text{--}30$ Mbit/s
Pi Zero (2,4 GHz WLAN)	einige zehn Mbit/s, hoher Jitter
Pi 4 WLAN (Dualband)	$\approx 100\text{--}300$ Mbit/s, mit Jitter
Pi 4 Gigabit-Ethernet (Kabel)	1000 Mbit/s, deterministisch
USB 2 / USB 3	480 / 5000 Mbit/s

26.1.3.2 Verdikt pro Plattform

ESP32 – nein.

Zwei harte Gründe: (1) Der ESP32 ist *kein* USB-Host für UVC-Kameras. (2) $177\text{--}1769$ Mbit/s liegen um Größenordnungen über seinem WLAN-Durchsatz. Der ESP32 gehört auf die **Echtzeit-Motorebene**, nicht an die Kamera.

Pi Zero – technisch möglich, aber schlechte Idee.

Der Zero 2 W kann mit OTG-Adapter eine UVC-Kamera einlesen, aber: schwache CPU, nur 2,4-GHz-WLAN, und er wäre allein mit dem Datenschaukeln gesättigt. Für einen *latenzkritischen* Balltracker ist WLAN-Jitter Gift.

Pi 4 – ja, und das ist der richtige Ansatz.

Der Pi 4 hat USB 3, Gigabit-Ethernet und eine brauchbare Quad-Core-CPU. Er kann die Kamera einlesen *und die Bildverarbeitung lokal machen*.

Das richtige Muster: am Rand rechnen, nur das Ergebnis senden

Schiebe nicht die Pixel durchs Netz, sondern die **Erkenntnis**. Lass den Pi 4 (direkt an der Kamera) die Stereo-Erkennung machen und nur die **3D-Ballposition** publizieren – das sind $\approx 0,1$ Mbit/s, trivial über jedes Netz. ROS 2/DDS transportiert das Topic netzwerktransparent zum PC. Für deterministische Latenz: **Pi 4 per Kabel-Gigabit-Ethernet** an den PC.

26.1.3.3 Sparse statt dense: der Ball braucht keine volle Tiefenkarte

Eine dichte Disparitätskarte (SGBM über das ganze Bild) ist teuer. Für die Ballverfolgung genügt **sparse Triangulation**: den Ball (heller Blob) in *beiden* rektifizierten Bildern detektieren und über $Z = fb/d$ nur die Tiefe *dieses einen Punktes* bestimmen. Genau diese Pixel-zu-3D-Triangulation haben wir in Projekt 0.B (Kap. 24) implementiert; sie läuft auf dem Pi 4 locker bei 60–120 fps.

26.1.3.4 ROS-Service, UDP oder Topic?

- **ROS-Service: falsch** für Streaming. Services sind synchrones Request/Response (blockierend, one-shot) – für kontinuierliche Ströme ungeeignet.
- **ROS-Topic: richtig.** Ergebnis (Ballposition) als RELIABLE-Topic; falls doch Bilder, `image_transport` mit `compressed` und QoS `BEST_EFFORT`.
- **Rohes UDP/GStreamer: nur als letzter Ausweg** – du verlierst die ROS-Integration. Für berechnete Ergebnisse unnötig.

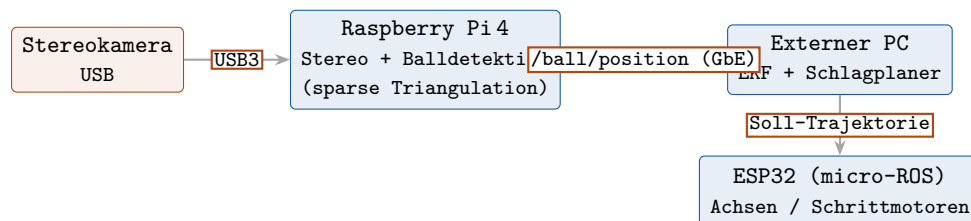


Abbildung 26.2: Empfohlene Architektur: schwere Pixelarbeit am Pi 4 nahe der Kamera, nur die destillierte Ballposition wandert (kabelgebunden) zum PC.

26.1.4 Mechanischer Aufbau: Portalsystem (Gantry)

26.1.4.1 Grundkonzept

Statt eines Delta-Roboters ein **kartesisches Portalsystem**: große Linearachsen positionieren den Schläger grob im Raum, ein kleiner Arm am Ende führt die Schlagbewegung aus und stellt den Schlägerwinkel. Bezug Tischtennisplatte (ITTF): 2,74 m × 1,525 m, Höhe 0,76 m, Netz 15,25 cm. Die **X-Achse** überspannt die Plattenbreite (1,525 m).

26.1.4.2 Geschachtelte Achsen

- **X-Achse** (Plattenbreite, horizontal): zwei parallele Führungsstäbe, Zahnriemenantrieb, X-Schlitten. *Glatte Hohlstahl-Stäbe + GT2-Zahnriemen*, leichter Schlitten.
- **Z-Achse** (Höhe, vertikal): nach demselben Prinzip, montiert *im* X-Schlitten. Der Abstand der X-Führungsstäbe definiert den **Hub der Z-Achse**.
- **Y-Achse** (Tiefe): entweder starr (fester Ausleger) oder als dritte Linearachse auf dem Z-Schlitten.
- **Schlagarm**: auf dem Y/Z-Schlitten ein kleiner Roboterarm mit Schlägereinspannung.

Zahnriemen, nicht Keilriemen

Für *Positionierung* brauchst du einen **Zahnriemen** (GT2), keinen Keilriemen. Ein Keilriemen überträgt per Reibschluss und *schlupft* – damit verlierst du Positionsgenauigkeit. Zahnriemen sind formschlüssig und der Standard in 3D-Druckern/Plottern.

26.1.4.3 Antriebstopologie und Hardware-Anschluss

Element	Umsetzung
X-/Z-/Y-Motoren	NEMA-17, möglichst <i>rahmenfest</i> (Riemen treibt Schlitten)

Treiber	TMC2209 (UART) je Achse, StallGuard als Last-/Endlagenhinweis
Endschalter	optische/mechanische Endstops je Achse zum Homing
Strom/Spannung	INA226-Modul je Treiberzweig (I ² C), optional
Controller	ESP32 (micro-ROS): bündelt STEP/DIR bzw. UART, liest Endstops + INA226
Versorgung	24 V Netzteil für Motoren, getrennte 5 V-Logik; Sternmasse
Schlagarm-Motoren	klein, idealerweise basisnah (Riemen-/Seilzug zu distalen Gelenken)

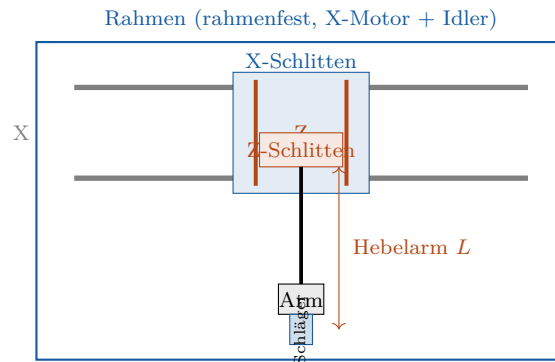


Abbildung 26.3: Frontblick: X-Schlitten trägt die im Rod-Abstand geschachtelte Z-Achse; der Schlagarm hängt als Ausleger (Hebelarm L).

26.1.5 Das Hebelarm- und Schwingungsproblem

Der Arm hängt mit Abstand L unter der X-Achse. Bei harten Beschleunigungen zur Zielposition entstehen Reaktionskräfte und -momente, die den Ausleger zum Schwingen anregen.

26.1.5.1 Die Physik

Beschleunigt der Schlitten mit a , wirkt auf die bewegte Masse m die Trägheitskraft $F = m a$. Über den Hebelarm L ergibt das ein **Biegemoment**

$$M = F \cdot L = m a L. \quad (26.1)$$

Der Ausleger verhält sich wie ein einseitig eingespannter Balken (Kragträger) mit Steifigkeit $k = \frac{3EI}{L^3}$ und erster **Eigenfrequenz**

$$f_1 = \frac{1}{2\pi} \sqrt{\frac{k}{m_{\text{eff}}}} = \frac{1}{2\pi} \sqrt{\frac{3EI}{m_{\text{eff}} L^3}}. \quad (26.2)$$

f_1 fällt mit $L^{3/2}$ – ein längerer Ausleger ist drastisch weicher. Anregung nahe f_1 führt zu Resonanz und Nachschwingen, was die Schlaggenauigkeit ruiniert.

26.1.5.2 Gegenmaßnahmen

1. **Bewegte Masse minimieren:** Motoren rahmenfest (Riemen-/Seilzug, CoreXY-Logik), Arm aus Aluminium/CFK, Schwerpunkt nah an der Achse.
2. **Steifigkeit maximieren:** große Schienenquerschnitte (gestützte Linearschienen), Doppelführung, kurzer Ausleger, triangulierter Rahmen.
3. **Ruckbegrenzte Profile (S-Kurve):** sanftes Beschleunigen regt hohe Frequenzen weniger an.
4. **Input Shaping / Resonanzkompensation:** der Steuerbefehl wird so vorgeformt, dass die Anregung bei f_1 ausgelöscht wird – wie Bambu/Klipper per ADXL345. Die wirksamste softwareseitige Maßnahme.

5. Dämpfung: Strukturdämpfung, abgestimmter Tilger, höhere Riemenvorspannung.

Anknüpfung an den FEM-Hintergrund

Bestimme f_1 und die Modenformen per **Modalanalyse** (SolidWorks Simulation / FEM). Das liefert die Frequenz, auf die du das Input Shaping abstimmst, und zeigt, welche Strukturteile zu versteifen sind.

26.1.6 Simulierbarkeit in Gazebo und RViz

Die Sim-Real-Trennung über `ros2_control` (Kap. 21) und der Gazebo-Workflow sind aus 0.A/0.B bekannt – hier nur die *Grenzen*.

26.1.6.1 Was Gazebo kann – und was nicht

Gazebo bildet ab (Starrkörperdynamik)	Gazebo bildet <i>nicht</i> ab
Anfahren der Zielposition (prism. + rot. Gelenke)	Strukturelle Biegung der Stäbe
Schlagbewegung des Arms	Riemenelastizität als kontinuierliche Elastizität
Gelenkmomente/-kräfte (auslesbar)	Modale Resonanz/Eigenschwingung
Dynamische Kopplung (Armträgheit ↔ Portal)	Rahmenflattern, Mikrovibrationen
Ball-Schläger-Kontakt + Reaktionskräfte	exakte Spin-/Absprungphysik (nur genähert)

Der Kernpunkt

Gazebo behandelt Links als **starre Körper**. Die befürchtete Schwingung kommt aber aus *struktureller Nachgiebigkeit* (Stabbiegung, Riemenelastizität) – starre Links biegen sich nicht, also zeigt Standard-Gazebo die Resonanz *nicht*.

26.1.6.2 Wege, die Nachgiebigkeit doch abzubilden

- **Klumpen-Nachgiebigkeit in Gazebo:** ein steifes Feder-Dämpfer-Gelenk zwischen Segmenten (Riemenelastizität als prismatische Feder). Näherung der ersten Mode, kein volles Modalverhalten.
- **Echte Strukturvibration:** FEM / flexible Mehrkörpersimulation – SolidWorks (modal), MATLAB **Simscape Multibody**, Adams; Co-Simulation mit ROS möglich.
- **RViz:** reine Visualisierung – Kräfte als Marker, wenn ein anderer Knoten sie berechnet.

Empfohlene Aufteilung

Gazebo für Regelung, Trajektorientracking, Momentenbudget und Ball-Schläger-Kontakt. **FEM/Simscape** für die strukturelle Resonanz (f_1). Beides verbindet die Input-Shaping-Frequenz.

26.1.7 CAD-Module und URDF

Wie man eine URDF aus Links, Joints und einem `ros2_control`-Block aufbaut, steht in Projekt 0.A (Kap. 23) und im CAD→URDF-Kapitel (Kap. 22). Hier das Gantry-Spezifische.

26.1.7.1 Notwendige CAD-Module mit Designanforderungen

Modul	Funktion	Designanforderung
Rahmen / Basis	trägt X-Achse + X-Motor, ortsfest	sehr steif (Alu-Profil), trianguliert; URDF-Wurzel
X-Achse	horizontale Verfahrachse	Hohlstahl-Stäbe + GT2-Riemen; Motor rahmenfest; > 1,6 m
X-Schlitten	trägt + verfährt die Z-Achse	leicht, biegesteif; Stababstand = Z-Hub
Z-Achse	vertikale Verfahrachse	gleiches Prinzip; Hub = X-Stababstand
Z-Schlitten	trägt Y/Arm	geringe Masse (bewegte Masse!)
Y-Achse (optional)	Tiefenzustellung	nur falls nötig; sonst starrer Ausleger
Armbasis	Anbindung Arm an Schlitten	steife Einspannung; kurzer Hebel L
Armglieder (2–3 DOF)	Schwung + Schlägerwinkel	leicht (Alu/CFK); Motoren basisnah
Schlägereinspannung	hält den Schläger	starr, definierte Schlägerebene
Endeffektor-Frame	Schlagpunkt + Normale	TF-Frame: Schlägerflächenmitte + Normale

26.1.7.2 Kinematische Kette

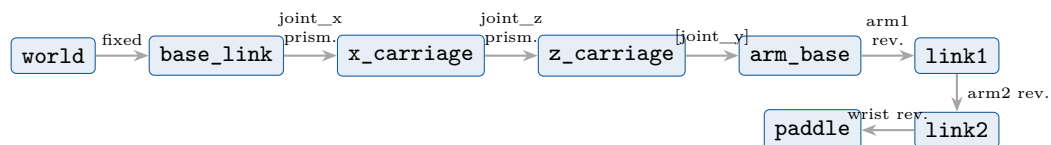


Abbildung 26.4: Kinematische Kette: drei (optional vier) Linear-/Drehübergänge bis zur Armbasis, dann der Schlagarm bis zum Schläger.

26.1.7.3 URDF/Xacro-Gerüst

```
<link name="base_link"> <!-- Wurzel: Rahmen fest mit der Welt --> </link>

<joint name="joint_x" type="prismatic">          <!-- X ueber die Plattenbreite -->
  <parent link="base_link"/> <child link="x_carriage"/>
  <origin xyz="0 0 1.0"/> <axis xyz="1 0 0"/>
  <limit lower="0" upper="1.5" effort="60" velocity="3.0"/>
</joint>
<link name="x_carriage"> ... </link>

<joint name="joint_z" type="prismatic">          <!-- Z vertikal, Hub = X-Stababstand -->
  <parent link="x_carriage"/> <child link="z_carriage"/>
  <axis xyz="0 0 1"/>
  <limit lower="-0.4" upper="0.0" effort="50" velocity="2.5"/>
</joint>
<link name="z_carriage"> ... </link>

<joint name="joint_arm1" type="revolute">        <!-- Schlagarm (arm2, wrist analog) -->
  <parent link="arm_base"/> <child link="link1"/>
  <axis xyz="0 1 0"/>
  <limit lower="-2.0" upper="2.0" effort="8" velocity="12"/>
</joint>
<link name="link1"> ... </link>

<joint name="joint_paddle" type="fixed">        <!-- Schlaeger-Endeffektor -->
  <parent link="link2"/> <child link="paddle"/>
  <origin xyz="0 0 0.12"/>
</joint>
<link name="paddle"> ... </link>
```

Listing 26.1: Gantry + Schlagarm als URDF (gekürzt)

Riemenübersetzung der Linearachsen

Eine prismatische Riemenachse wandelt Motordrehung in Linearweg: *Weg pro Umdrehung* = Zähnezahln $\times$ Riementeilung. Ein 20-Zahn-GT2-Rad (2 mm Teilung) ergibt 40 mm/Umdrehung; mit 200×16 Microsteps folgt $40/3200 = 0,0125$ mm/Microstep. Diese Umrechnung Motor $\leftrightarrow$ mm gehört ins Hardware-Interface; in der URDF stehen am Joint die *linearen* Limits.

26.1.8 Ablauf- und Parallellogik

Wahrnehmung, Schätzung und Stellgrößen laufen **nebenläufig** mit sehr unterschiedlichen Raten – als separate ROS 2-Knoten (teils auf verschiedenen Rechnern), die nur über Topics gekoppelt sind und sich nicht gegenseitig blockieren (Producer/Consumer).

Knoten	Rate	Aufgabe
ball_detector	Kamera-FPS (30–120 Hz)	Stereo-Erkennung $\rightarrow$ 3D-Punkt (Pi 4)
ball_tracker (EKF)	pro Messung	Zustand updaten, Flug prädictieren
strike_planner	pro EKF- Update	Auftreffpunkt + Zeit + Schlagbahn
robot_controller ESP32 (micro-ROS)	≈ 100 Hz Echtzeit (kHz)	Trajektorie interpolieren, Stellgrößen STEP/DIR-Erzeugung, Endstops

Frühe Reaktion statt Warten auf die perfekte Schätzung

Der `strike_planner` veröffentlicht bereits nach den *ersten* EKF-Schätzungen eine vorläufige `/robot/target_pose`; der Roboter fährt sofort grob in Richtung Abfangzone. Jede neue Ballmessung verfeinert die Schätzung – die `target_pose` wird laufend aktualisiert, und der Regler folgt der bewegten Sollgröße. So wird die knappe Flugzeit voll genutzt, statt auf eine endgültige Schätzung zu warten.

26.2 Vorgehensweise

Wir bauen die **Wahrnehmung zuerst vollständig in Simulation** und erst danach die Robotermechanik und -regelung. So lässt sich das Balltracking und die Auftreffpunkt-Schätzung isoliert verifizieren (direkt auf Projekt 0.B aufbauend), bevor die Mechanik dazukommt.

26.2.1 Phase 1 – Kamera und Balltracking in Simulation

26.2.1.1 Tischtennisplatte als CAD/SDF-Modell

Zuerst wird die Tischtennisplatte konstruiert (CAD → Mesh/Primitive) und als *statisches* Modell in die Gazebo-Welt eingefügt: Platte $2,74 \times 1,525 \times 0,76$ m mit Netz. Sie dient als Referenzgeometrie und definiert die **Zielebene** (siehe unten). Die Stereokamera-Welt aus Projekt 0.B (Kap. 24) wird um dieses Modell ergänzt.

26.2.1.2 Gegnerischen Ballschlag parametrisch simulieren

Ein Knoten (oder Gazebo-Plugin) startet den Ball mit einer *parametrierbaren* Anfangsbedingung – so sind Schläge reproduzierbar:

```
start_position: [x0, y0, z0]      # Abschlagort auf der Gegenseite
speed:          v0                 # Betrag der Abschlaggeschwindigkeit [m/s]
azimuth:        phi                # horizontaler Abschlagwinkel [rad]
elevation:      theta              # vertikaler Abschlagwinkel [rad]
# optional (spaeter): spin_axis, spin_rate
```

Listing 26.2: Parameter eines simulierten Gegnerschlags

Daraus folgt die Anfangsgeschwindigkeit $\mathbf{v}_0 = v_0 (\cos \theta \cos \phi, \cos \theta \sin \phi, \sin \theta)$, die dem Ball als Twist gesetzt wird; die Schwerkraft übernimmt Gazebo.

26.2.1.3 Stereokamera-Tracking des Balls

Die Wahrnehmungskette aus 0.B (`ball_detector` → `/ball/point`) trackt den fliegenden Ball und liefert verrauschte 3D-Positionen im Kamerasystem.

26.2.1.4 Auftreffpunkt über schiefen Wurf und EKF

Ohne Luftwiderstand ist der Flug ein **schiefer Wurf** mit bekannter Erdbeschleunigung $\mathbf{g} = (0, 0, -g)$:

$$\mathbf{r}(t) = \mathbf{r}_0 + \mathbf{v}_0 t + \frac{1}{2} \mathbf{g} t^2. \quad (26.3)$$

Der EKF-Zustand ist $\mathbf{x} = [\mathbf{r}, \mathbf{v}]^\top$ (6D), Messung sind die 3D-Stereopositionen, das Prozessmodell ist die konstante Beschleunigung $\mathbf{g}$. Die Ballmasse ist bekannt – sie wird erst relevant, sobald Luftwiderstand/Spin das Modell *nichtlinear* machen (dann ist der EKF zwingend, beim reinen Wurf genügt ein linearer KF). Mit jeder neuen Messung werden die Wurfparameter (effektiv $\mathbf{r}_0, \mathbf{v}_0$ bzw. der aktuelle Zustand) aktualisiert; reine Prädiktionsschritte extrapolieren den Flug.

Schätzung nur bis zur Tischmitte – danach unveränderbar. Der EKF korrigiert die Bahn nur, solange der Ball auf der *Gegnerseite* ist (bis zur Tischmitte, $y = 0$). Überschreitet die Schätzung die Mitte, wird sie **eingefroren** – Abfangpunkt und -zeit gelten ab dann als fest. Drei Gründe: (1) Der Roboter beginnt eine *frühe* Abfangbewegung und verfeinert sie bis zur Mitte, fährt danach aber *ruhig* zu Ende – statt kurz vor dem Auftreffpunkt noch schnelle, mechanisch kaum umsetzbare Korrekturen zu machen. (2) In der zeitkritischen Endphase fällt die EKF-Rechenlast weg. (3) Bis zur Mitte ist die Schätzung ohnehin konvergiert. Ein neuer Schlag (Ball wieder weit auf der Gegnerseite) setzt das Filter zurück.

Aufprall auf der Platte (Restitution). Trifft der Ball die Tischfläche, ist das ein *Stoß* statt einer glatten Parabel. Ein realer Tischtennisball verliert dabei wenig Energie: aus der ITTF-Prüfung (Fall aus 30,5 cm, Rücksprung 24–26 cm) folgt ein Restitutionskoeffizient $e = \sqrt{h_{\text{zurück}}/h_{\text{fall}}} \approx 0,9$ – *nahezu* elastisch, aber nicht ideal ($e = 1$). Im Flugmodell wird der Aufprall als **stückweise Reflexion** eingebaut: schneidet die prädiizierte Bahn die Tischebene ($z = h_{\text{Tisch}}$) innerhalb der Platte, kehrt sich die Normalkomponente um und wird gedämpft,

$$v_z \rightarrow -e v_z, \quad (v_x, v_y \text{ bei Reibungsfreiheit unverändert}), \quad (26.4)$$

danach läuft die Parabel weiter. **Reibung** beim Aufprall wirkt nur tangential und damit fast ausschließlich auf den *Spin*; auf die translatorische Bahn ist sie ohne Spin vernachlässigbar. Ein ideal elastischer Stoß ($e = 1$) ist eine brauchbare erste Näherung, $e \approx 0,9$ ist genauer und verschiebt Rücksprunghöhe und damit Abfangzeit spürbar. In der Simulation liefert Gazebo dieses Verhalten über den `restitution_coefficient` der Kollisionsfläche (Schritt 2).

26.2.1.5 Zielebene, Plattenrand und Fehlschuss-Erkennung

Zwei Ebenen sind relevant. **(1) Die Tischfläche** ($z = h_{\text{Tisch}} = 0,76$ m, horizontal): Der Schnittpunkt der prädiizierten Bahn mit ihr ist der *Aufprallpunkt* auf der Platte. Liegt er *innerhalb* des Plattenrands auf der Roboterhälfte, ist der Schlag gültig (dort findet der Restitutions-Bounce statt); liegt er außerhalb, geht der Ball ins Aus. **(2) Die Abfangebene** (vertikal, z. B. $y = y_R$ an der hinteren Plattenkante der Roboterseite): ihr Durchstoßpunkt *nach* dem Bounce ist die eigentliche **Abfangposition** des Schlägers,

$$t^* = \frac{y_R - y_0}{v_{y0}}, \quad x^* = x(t^*), \quad z^* = z(t^*), \quad (26.5)$$

also (x^*, y_R, z^*) und Abfangzeit t^* (mit dem gespiegelten Geschwindigkeitsvektor nach dem Tischkontakt). Als **Fehlschuss** markiert wird (Flag/Diagnostic): wenn der Ball das gemeinsame Sichtfeld beider Kameras verlässt (keine Triangulation mehr – vgl. FOV-Kegel, Kap. 24), *oder* der Aufprallpunkt außerhalb der Platte liegt, *oder* die Abfangposition außerhalb des Arbeitsraums, *oder* ein **Netztreffer** vorhergesagt wird.

Netztreffer und Netzroller. Schneidet die prädiizierte Bahn die *Netzebene* ($y = 0$) *unterhalb* der Netzoberkante ($z < 0,9125$ m), geht der Ball ins Netz – der Wurf wird ebenfalls als **ungültig** erkannt. Auch ein *Netzroller* (der Ball berührt die Netzkante und fällt knapp dahinter) gilt als nicht abfangbar: die dabei stark verlangsamte, abgelenkte Bahn lässt dem Roboter weder Bewegungsraum noch Reaktionszeit, ihn regulär zu treffen. Solche Bälle werden gar nicht erst angefahren.

Aufschlag- vs. Ballwechselregeln. Bisher haben die Beispielwürfe nur einen *Ballwechsel* nachgebildet (der Ball kommt über das Netz und trifft zuerst die Roboterseite). Ein regelkonformer **Aufschlag** sieht anders aus: Der Ball muss *zuerst genau einmal auf der Aufschlagseite* ($y > 0$, beim Gegenspieler) aufkommen und *danach* auf der Roboterseite ($y < 0$). Erst dann ist der Aufschlag gültig. **Der Roboter schlägt nie selbst auf** – der Aufschlag kommt immer vom Spieler; der Roboter ist hier ausschließlich Rückschläger. Daraus folgen die Fehlschuss-Fälle des Aufschlags:

erster Aufprall *nicht* auf der eigenen Seite, ein *zweiter* Aufprall erneut auf der Aufschlagseite, oder ein Netztreffer. Nach einem gültigen Aufschlag wird auf die **Ballwechselregeln** umgeschaltet: der ankommende Ball muss genau einmal auf der Roboterseite aufkommen (ein zweiter Aufprall ohne Rückschlag ist ein Doppelaufprall). Dieselbe kleine Zustandsmaschine, die schon zwischen REAKTIV und PAUSE trennt, verwaltet damit auch die Phase *Aufschlag* bzw. *Ballwechsel*.

Sichtbare Begrenzungswand als Güte-Anzeige. Um die Qualität von EKF und Flugmodell *direkt* beurteilen zu können, wird am Plattenrand der Roboterseite (an der Abfangebene $y = y_R$) eine **durchsichtige Begrenzungswand** eingezeichnet – rein visuell, der Ball fliegt hindurch. Auf ihr wird der vom EKF geschätzte *Auftreffpunkt* als Marker eingeblendet, der sich mit jeder Messung aktualisiert. So sieht man unmittelbar, *wo* das Modell den Durchstoßpunkt erwartet und *wo* der reale Ball die Wand tatsächlich durchbricht – die Abweichung ist die Modellgüte. Ein **Fehlschuss** wird an derselben Stelle sichtbar gemacht (die Wand färbt sich rot) und beim nächsten Ballwechsel wieder zurückgesetzt.

Abbruch und Rückkehr in die Initialstellung. Wird ein Wurf – noch *während* des Anfahrens – als ungültig erkannt (Aus, Netz, außer Reichweite), muss die laufende Roboterbewegung **abgebrochen** werden: das aktive Goal wird gecancelt, der Roboter stoppt die Abfangbewegung und fährt in seine **Initialstellung** (Home) zurück – bereit für den nächsten Ballwechsel. Genau dafür ist der **Action-Mechanismus** gemacht (Projekt 0.A, Kap. 23): ein Goal ist jederzeit *cancelbar*, und der Action-Server reagiert mit einem definierten Abbruch – in Simulation und Realität identisch.

Betriebsmodi: reaktiv vs. Pause. Der Roboter darf *nicht immer* auf erkannte Bälle reagieren. Eine kleine **Zustandsmaschine** trennt:

- **REAKTIV** – ein Ballwechsel läuft: die Tracker-Schätzungen werden in Roboter-Goals umgesetzt (frühe Abfangbewegung, laufende Verfeinerung).
- **PAUSE** – zwischen den Ballwechseln (Ball holen, Aufschlag vorbereiten): der Tracker darf weiterlaufen, löst aber *keine* Roboterbewegung aus. So werden **Fehlreaktionen** ausgeschlossen, etwa wenn jemand den Ball aufhebt oder über den Tisch greift.

Diese Moduslogik gehört bereits in den *Entwurf*. Da die Roboterbefehle ohnehin über **Actions und Goals** laufen, ist das passende Konstrukt schon vorhanden: im PAUSE-Modus werden schlicht keine Goals gesendet bzw. angenommen, im REAKTIV-Modus sendet der `strike_planner` Goals und aktualisiert bzw. cancelt sie. Derselbe Mechanismus trägt von der Simulation in die reale Anlage.

26.2.1.6 Ausblick: Luftwiderstand

Beim leichten Tischtennisball (Masse $m \approx 2,7$ g, Durchmesser 40 mm) ist der Luftwiderstand *nicht* vernachlässigbar – das kleine Verhältnis von Masse zu Querschnittsfläche führt zu spürbarer Verzögerung. Die Widerstandskraft wirkt entgegen der Geschwindigkeit und wächst quadratisch:

$$\mathbf{F}_D = -\frac{1}{2} \rho C_D A \|\mathbf{v}\| \mathbf{v}, \quad A = \pi r^2, \quad (26.6)$$

mit Luftdichte $\rho \approx 1,2 \text{ kg/m}^3$, Widerstandsbeiwert $C_D \approx 0,4$ (Kugel) und Querschnittsfläche A . Die Bewegungsgleichung lautet damit

$$\dot{\mathbf{v}} = \mathbf{g} - k \|\mathbf{v}\| \mathbf{v}, \quad k = \frac{\rho C_D A}{2m}, \quad (26.7)$$

und ist **nichtlinear** in $\mathbf{v}$ – genau deshalb wird hier der **EKF** (statt eines linearen KF) gebraucht: Das Prozessmodell wird pro Schritt linearisiert (Jacobi-Matrix der Beschleunigung nach dem Zustand). Die bekannte Ballmasse m geht direkt in k ein. Wirkung: Die reale Flugbahn ist *flacher*

und kürzer als die reine Wurfparabel – der Auftreffpunkt verschiebt sich, weshalb der Term für eine genaue Abfangzeit-Schätzung relevant ist. Der Schritt vom reinen Wurf zum Modell mit Luftwiderstand ist die natürliche erste Verfeinerung; der Spin (folgender Abschnitt) baut darauf auf.

26.2.1.7 Ausblick: Ballspin (Magnus)

Das Modell wird so aufgebaut, dass es um **Spin** erweiterbar ist: Die Magnus-Kraft $\mathbf{F}_M \propto \boldsymbol{\omega} \times \mathbf{v}$ ergänzt das Prozessmodell (nichtlinear $\rightarrow$ EKF/UKF), der Zustand wird um die Drehrate $\boldsymbol{\omega}$ erweitert. Messbar wird der Spin, indem **Markierungen auf dem Ball** getrackt werden, um Rotationsrichtung und -geschwindigkeit zu schätzen – eine spätere Ausbaustufe.

26.2.2 Phase 2 – Roboteranbindung

26.2.2.1 Frühe Abfangbewegung (iterative Verfeinerung)

Gemäß der Ablauflogik fährt der Roboter bereits nach den ersten Schätzungen los (grobe `target_pose`) und verfeinert das Ziel mit jeder neuen Messung. Das minimiert die effektive Reaktionszeit innerhalb der kurzen Ballflugdauer.

26.2.2.2 Stellgrößen und Motorsteuerung

Die geschätzte `target_pose` (Abfangpunkt + Schlägerorientierung) wird per inverser Kinematik in Achswerte umgesetzt: x, z (und optional y) aus der kartesischen Zielposition, die Armgelenke für den Schlägerwinkel. Der `joint_trajectory_controller` interpoliert, das Hardware-Interface (micro-ROS $\rightarrow$ ESP32 $\rightarrow$ TMC2209) erzeugt die Schritte – derselbe `ros2_control`-Stack wie in Projekt 0.A und Projekt 1. Hier zählt das **Timing**: in der Simulation trivial, real der kritische Faktor.

26.2.2.3 Regelstrategie: entkoppelte Achsregelung statt MIMO

Das Szenario wirkt komplex, lässt sich aber **entkoppeln**. Das Portal (Gantry) hat mechanisch nahezu orthogonale, unabhängige Achsen x und z (und optional y). Jede Achse bekommt daher einen *eigenen* Regler – **SISO-PID**, in der Praxis als Kaskade aus Positions- und Geschwindigkeitsschleife – statt einer gekoppelten MIMO-Regelung. Die Querkopplung (etwa die Trägheit des bewegten Arms auf die Portalachse) ist klein und wird als Störgröße ausgeregelt; bei Bedarf hilft eine **Vorsteuerung** (Feedforward) aus dem bekannten Soll-Bahnprofil. So zerfällt das Problem in drei lose gekoppelte Teile:

1. **Grobpositionierung** (Portal x, z): große, ruhige Bewegung in die Abfangregion – entkoppelte PID je Achse.
2. **Schlägerorientierung** (Armgelenke): Vor-/Rückhandstellung und Anstellwinkel des Schlägers.
3. **Schlag** (getaktete Schwungbewegung): kurz, schnell, auf den Auftreffzeitpunkt t^* getriggert.

Positionierung und Schlag werden auch *zeitlich* getrennt: erst früh und ruhig positionieren, dann im richtigen Moment schlagen. Das entkoppelt die langsame, genaue Anfahrt von der schnellen, getakteten Schlagbewegung – beide müssen nicht im selben Regelkreis gelöst werden.

26.2.2.4 Schläger-Reichweite: Position grob, Schlag fein

Der Roboter muss *nicht* exakt auf den geschätzten Auftreffpunkt fahren. Der Schläger sitzt am Ende eines **Schlagarms**, der einen Arbeitsraum aufspannt – der Ball ist treffbar, sobald er *irgendwo* in diesem Arbeitsraum liegt. Zwei Konsequenzen: (1) Die Positionier-Toleranz wächst um die Schläger-/Armreichweite; ein EKF-Restfehler von wenigen Zentimetern (Kap. 26.3.6) liegt klar darunter. (2) Der Arm wird so ausgelegt, dass **Vorhand und Rückhand** spielbar sind. Das vergrößert die seitliche Reichweite um die Portalposition und *verkürzt die nötige Reaktionszeit*: statt

das Portal weit zur Ballseite zu fahren, genügt oft ein Armschwung auf die entsprechende Seite. Vor- oder Rückhand folgt aus der Lage des Abfangpunkts relativ zur Portalposition.

26.2.2.5 Inverse Kinematik bei laufendem EKF-Update

Da der EKF den Abfangpunkt mit jeder Messung aktualisiert (bis zur Tischmitte, Kap. 26.3.6), stellt sich die Frage, ob eine *Neuberechnung der inversen Kinematik pro Update* performant ist. Antwort: **ja, problemlos** – weil die Kinematik hier einfach ist. Für das Portal ist die IK *trivial*: die kartesischen Ziele x, z sind (bis auf Offsets) direkt die prismatischen Achswerte, keine Iteration. Die wenigen Armgelenke für den Schlägerwinkel haben eine *geschlossene* (oder niederdimensional-numerische) Lösung. Eine IK-Auswertung kostet so Mikrosekunden – bei 60 Hz EKF-Updates völlig unkritisch. Das ist der Vorteil des Portalkonzepts gegenüber einem redundanten 6-DOF-Arm, dessen mehrdeutige, iterative IK eine ständige Neuberechnung teurer machte.

26.2.2.6 Auftreffzeitpunkt, Timing und Balltypen

Für den Schlag genügt der Auftreffpunkt nicht – es braucht auch den Auftreffzeitpunkt t^* , um den Schwung so zu takten, dass der Schläger den Ball zum richtigen Moment am richtigen Ort trifft. t^* fällt im EKF ohnehin an (Kap. 26.3.7). Entscheidend ist die **Spreizung der Reaktionszeiten**: ein langsamer, hoher Bogen gibt viel Zeit (Bahn lange verfeinerbar, später Schlag), ein schneller, flacher Ball wenig. Bei knappem Zeitfenster *committet* der Roboter früh bzw. schlägt direkt, statt die Schätzung weiter zu verfeinern – daher die Ball-Klassifikation in Kap. 26.3.7.

26.3 Umsetzung

Hier folgen die konkreten Implementierungsschritte – jeder für sich lauffähig, bevor der nächste daraufgesetzt wird. Wie in den Einstiegsprojekten lebt der Code in einem eigenen Paket `p2_tabletennis` im gemeinsamen Workspace `~/ros2_ws`.

26.3.1 Schritt 1 – Tischtennisplatte als STL in Gazebo einbinden

Das CAD-Modell der Platte (aus SolidWorks o. Ä. als `.stl` exportiert) wird als statisches Mesh-Modell in eine Gazebo-Welt eingebunden. Zwei Eigenheiten von STL-Dateien sind dabei zu beachten:

- **Einheiten:** STL trägt keine Einheit. Der Export ist hier in *Millimetern* (Bounding-Box $1525 \times 2740 \times 912$) – Gazebo rechnet in *Metern*, also `<scale>0.001 0.001 0.001</scale>`.
- **Orientierung:** Im CAD zeigt Y nach oben (die Platte sitzt auf der $y = 0$ -Ebene), in ROS/Gazebo zeigt Z nach oben. Eine Modell-`<pose>` mit Roll $+90^\circ$ (1,5708) dreht Y -oben auf Z -oben.

```
<model name="table">
  <static>true</static>
  <pose>0.173 0 0 1.5708 0 0</pose>  <!-- x: Breite zentrieren; roll +90: Y->Z -->
  <link name="link">
    <visual name="v"><geometry><mesh>
      <uri>package://p2_tabletennis/meshes/Tischtennisplatte.stl</uri>
      <scale>0.001 0.001 0.001</scale>          <!-- mm -> m -->
    </mesh></geometry></visual>
    <collision name="c"><geometry><mesh>
      <uri>package://p2_tabletennis/meshes/Tischtennisplatte.stl</uri>
      <scale>0.001 0.001 0.001</scale>
    </mesh></geometry></collision>
  </link>
</model>
```

Listing 26.3: `worlds/tabletennis.sdf` – die Platte als Mesh-Modell

Gazebo löst package:// nur über GZ_SIM_RESOURCE_PATH

Gazebo kennt keine ROS-Pakete. Damit `package://p2_tabletennis/...` auflöst, hängt die Launch-Datei den *Eltern-Ordner* des Paket-share an den Suchpfad – Gazebo sucht dort den ersten URI-Teil als Unterordner. Deshalb **immer über die Launch-Datei starten**, nicht mit nacktem `gz sim`.

```
from launch.actions import AppendEnvironmentVariable
pkg_share = get_package_share_directory('p2_tabletennis')
AppendEnvironmentVariable('GZ_SIM_RESOURCE_PATH', os.path.dirname(pkg_share))
# ... IncludeLaunchDescription(gz_sim.launch.py, gz_args=[world, '-r'])
```

Listing 26.4: launch/table.launch.py (Kern)

```
cd ~/ros2_ws
colcon build --packages-select p2_tabletennis
source install/setup.bash
ros2 launch p2_tabletennis table.launch.py
```

Listing 26.5: Bauen, sourcen, Welt starten

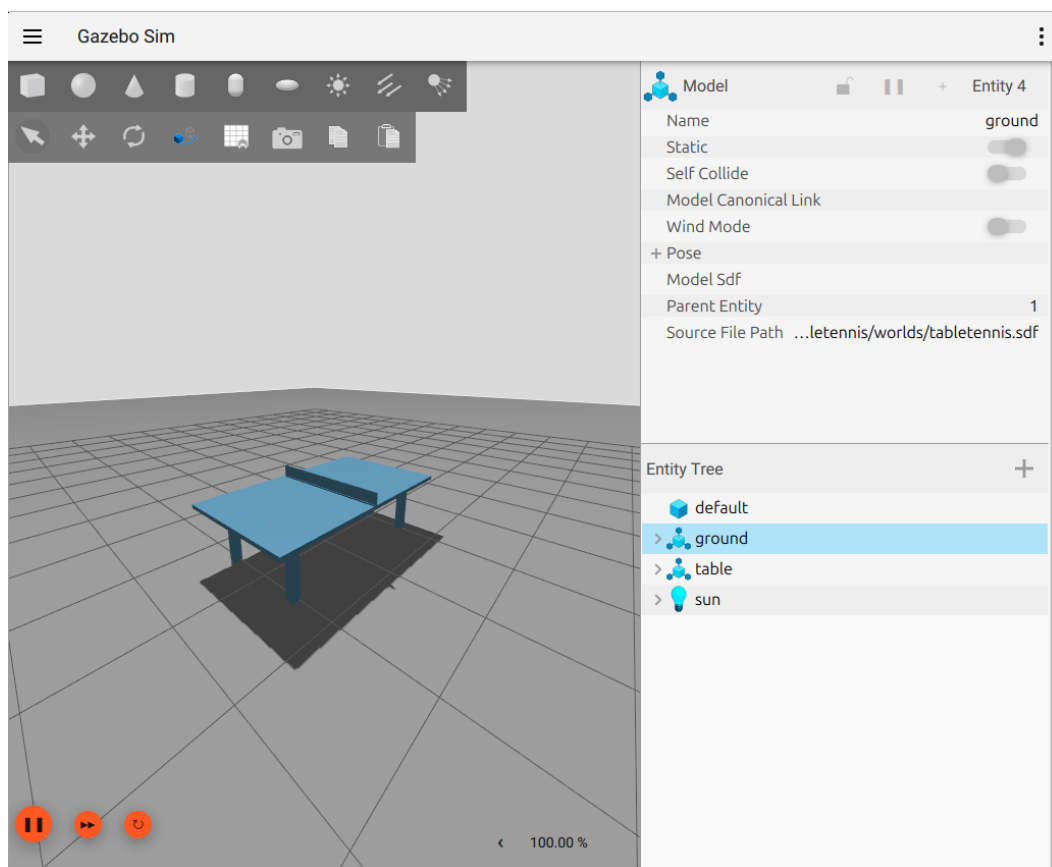


Abbildung 26.5: Die eingebundene Tischtennisplatte in Gazebo: korrekt in Metern skaliert (Spielfläche bei $z = 0,76\text{ m}$) und über Roll $+90^\circ$ aufrecht gestellt. Sie bildet die Referenzgeometrie und die spätere Zielebene.

```
src/p2_tabletennis/
|- meshes/Tischtennisplatte.stl    # neu (STL aus dem CAD, in mm)
|- worlds/tabletennis.sdf         # neu (Platte als Mesh-Modell + Boden + Licht)
|- launch/table.launch.py        # neu (Gazebo + GZ_SIM_RESOURCE_PATH)
`- setup.py                      # angepasst: data_files (worlds/launch/meshes)
```

Listing 26.6: ros2_ws-Delta nach Schritt 1 (# neu / # angepasst)

26.3.2 Schritt 2 – Stereo-Rig und Ball in die Welt integrieren

Die Wahrnehmung aus Projekt 0.B (Kap. 24) wird übernommen: das Stereo-Rig und der Ball kommen in `tablettennis.sdf`, die Knoten (`ball_detector`, `ball_tracker`, `marker_viz`, `fov_viz`) werden unverändert aus `p0b_stereo` wiederverwendet.

Welt erweitern. Stereo-Rig (Blick entlang $+y$ zur Gegenseite, Basisbreite 0,10 m) und der orange Ball (40 mm, 2,7 g) kommen hinzu. Damit der Ball **nicht durch die Platte fällt**, bekommt die Spielfläche eine *Box-Kollision* (robuster als ein konkaves Mesh) mit Restitution:

```
<model name="table_top"><static>true</static>
  <pose>0 0 0.74 0 0 0</pose>                                <!-- Oberkante z=0.76 -->
  <link name="link"><collision name="c">
    <geometry><box><size>1.525 2.74 0.04</size></box></geometry>
    <surface>
      <bounce><restitution_coefficient>0.9</restitution_coefficient><threshold>0</threshold></
      bounce>
      <friction><ode><mu>0.4</mu><mu2>0.4</mu2></ode></friction>
    </surface>
  </collision></link>
</model>

<model name="ball"><pose>0 0.5 1.2 0 0 0</pose>
  <link name="link">
    <inertial><mass>0.0027</mass> ... </inertial>
    <collision name="c"><geometry><sphere><radius>0.02</radius></sphere></geometry>
      <surface><bounce><restitution_coefficient>0.9</restitution_coefficient>
        <threshold>0</threshold></bounce></surface></collision>
    <visual name="v"> ... orange ... </visual>
  </link>
</model>
```

Listing 26.7: Spielflächen-Kollision + Ball (Auszug aus `tablettennis.sdf`)

Aufprall: kein Durchfallen, fast elastischer Stoß

Eine *Box-Kollision* der Spielfläche (statt des konkaven Tisch-Meshes) verhindert Tunneling; mit dem kleinen Zeitschritt (1 ms) bleibt der Ball sicher auf dem Tisch (Test: Start $z = 1,2$, nach Sekunden noch springend bei $z \approx 0,83$ – nicht durchgefallen). Der `restitution_coefficient` 0,9 entspricht dem realen Ball (nahezu elastisch, vgl. 26.2.1.4); Reibung beim Aufprall ist ohne Spin vernachlässigbar.

Netz: Box-Kollision statt eigenem STL. Das Tisch-Mesh enthält das Netz nur *visuell* – ohne Kollision fliegt der Ball hindurch. Ein eigener STL-Körper fürs Netz ist dafür *nicht* nötig (und als dünner, konkaver Collider sogar tunneling-anfällig); robuster ist – wie bei der Spielfläche – eine dünne *Box-Kollision* am Netzort ($y = 0$, Höhe 15,25 cm über der Platte):

```

<model name="net"><static>true</static>
  <pose>0 0 0.83625 0 0 0</pose>          <!-- 0.76 + 0.1525/2 -->
  <link name="link"><collision name="c">
    <geometry><box><size>1.525 0.01 0.1525</size></box></geometry>
    <surface>
      <bounce><restitution_coefficient>0.1</restitution_coefficient><threshold>0</threshold></
      bounce>
      <friction><ode><mu>0.8</mu><mu2>0.8</mu2></ode></friction>
    </surface>
  </collision></link>
</model>

```

Listing 26.8: Netz-Kollision (Auszug aus tabletennis.sdf)

Die **niedrige Restitution** (0,1) und **hohe Reibung** nähern das weiche, energiezehrende Verhalten des Netzes an: der Ball prallt nicht wie von einer Wand ab, sondern verliert Energie und fällt. Eine genauere *Netzphysik* (nachgiebiges Gewebe) bräuhete ein Soft-Body- bzw. Feder-Masse-Netz – in Gazebo aufwändig; für die Spiel-Logik genügt meist ein *Netztreffer-Flag* (Let/Fehler) zusätzlich zur Fehlschuss-Erkennung (26.2.1.5).

Sichtkegel. Wie in Projekt 0.B (Kap. 24, Sichtfeld-Visualisierung) zeichnet `fov_viz` die Sichtkegel beider Kameras als `MarkerArray` auf `/stereo/fov_markers`; ihr Überlappungsbereich ist die Zone, in der trianguliert werden kann.

Pipeline per Launch. Eine Launch-Datei bündelt – mit allen Lehren aus 0.A/0.B – Gazebo, /clock-Brücke, Bildbrücke, statische Kamera-TF und die vier Knoten, alle mit `use_sim_time`:

```

detector = Node(package='p0b_stereo', executable='ball_detector', parameters=[SIM])
tracker  = Node(package='p0b_stereo', executable='ball_tracker', parameters=[SIM])
viz      = Node(package='p0b_stereo', executable='marker_viz',   parameters=[SIM])
fov      = Node(package='p0b_stereo', executable='fov_viz',     parameters=[SIM])
static_tf = Node(package='tf2_ros', executable='static_transform_publisher',
  arguments=['--x', '0', '--y', '-1.9', '--z', '1.1', '--roll', '-1.5708',
    '--frame-id', 'world', '--child-frame-id', 'stereo_left_optical'])
return LaunchDescription([resource, gz, clock_bridge, image_bridge,
  static_tf, detector, tracker, viz, fov, rviz])

```

Listing 26.9: launch/tabletennis.launch.py (gekürzt)

Checkliste 0.A/0.B – in dieser Launch beachtet

/clock gebrückt und `use_sim_time:=true` auf allen Knoten (sonst “No transform”); statische TF `world`→`stereo_left_optical`, damit der Kamerarahmen existiert; `ogre2/EGL` fürs Kamerarendering; `ApproximateTimeSynchronizer` für L/R-Bilder; Ball orange (⇒ HSV-Bereich aus 0.B passt); `package://` über `GZ_SIM_RESOURCE_PATH`.

Terminals und Befehle.

```

cd ~/ros2_ws
colcon build --packages-select p2_tabletennis
source install/setup.bash
ros2 launch p2_tabletennis tabletennis.launch.py

```

Listing 26.10: Terminal 1 – bauen + alles starten (Gazebo + RViz + Pipeli

```
source ~/ros2_ws/install/setup.bash
ros2 topic echo /ball/point          # gemessene 3D-Position (Triangulation)
ros2 topic echo /ball/estimate       # Kalman-Schätzung
```

Listing 26.11: Terminal 2 – Ballposition anzeigen

In Gazebo erscheinen Platte, Rig und der (fallende/springende) Ball; in RViz die Sichtkegel, die Marker (gemessen/geschätzt/Vorhersage) und beide Kamerabilder. Der *parametrische Gegnerschlag* (Anfangsgeschwindigkeit statt freier Fall) folgt später; bis dahin lässt sich der Ball in Gazebo mit der Maus bewegen.

```
src/p2_tabletennis/
|- worlds/tabletennis.sdf          # angepasst: Stereo-Rig + Ball + Box-Kollision (Platte + Netz)
|- launch/tabletennis.launch.py   # neu (Welt + Bruecken + Pipeline + RViz)
|- config/tabletennis.rviz        # neu (Marker, FOV, Kamerabilder; Fixed Frame world)
`- package.xml                   # angepasst: exec_depend p0b_stereo u.a.
```

Listing 26.12: ros2_ws-Delta nach Schritt 2 (# neu / # angepasst)

26.3.3 Schritt 3 – Balltracking: fliegender Ball

Damit die Pipeline einen echten Flug verfolgt, muss der Ball über den Tisch fliegen – der Gegnerschlag aus 26.2.1.2. Gazebo bietet *keine* API, einem freien Körper eine Anfangsgeschwindigkeit zu geben (nur `set_pose`). Wir bewegen den Ball daher **kinematisch** entlang einer parametrischen Wurfparabel: der Knoten `opponent_serve` integriert Position und Geschwindigkeit (mit Tisch-Bounce, vgl. 26.2.1.4) und setzt die Ballpose $\sim 50x/s$ über den gebrückten `SetEntityPose`-Service. Vorteil: reproduzierbar und mit bekannter Ground Truth – ideal, um Erkennung und EKF gegenzuprüfen. (`self.p(n)` unten ist Kürzel für `self.get_parameter(n).value.`)

```
def start_serve(self):
    # v0 aus speed/azimuth/elevation
    s, sp = self.p('start'), self.p('speed')
    az, el = self.p('azimuth'), self.p('elevation')
    self.pos = [float(s[0]), float(s[1]), float(s[2])]
    self.vel = [sp*cos(el)*cos(az), sp*cos(el)*sin(az), sp*sin(el)]
    self.last_t = None

def tick(self):
    if self.pos is None: return
    now = self.get_clock().now().nanoseconds * 1e-9 # dt aus (Sim-)Zeit!
    if self.last_t is None: self.last_t = now; return
    dt = now - self.last_t; self.last_t = now
    self.vel[2] -= self.p('g') * dt
    for i in range(3): self.pos[i] += self.vel[i] * dt
    if self.pos[2] <= TZ and self.vel[2] < 0 and on_table(self.pos):
        self.pos[2] = TZ; self.vel[2] *= -self.p('restitution') # Bounce
    self.set_ball(self.pos) # SetEntityPose-Service (async)
```

Listing 26.13: opponent_serve – Kern (Wurf + Bounce)

Lehre: dt aus der Uhr – und use_sim_time braucht /clock

Mit *festem* dt pro Tick flog der Ball zu schnell, weil der Timer unter `use_sim_time` beim Sim-Zeit-Aufholen mehrfach feuert; korrekt ist dt aus der *tatsächlichen* Uhr-Differenz. Und: läuft `opponent_serve` mit `use_sim_time`, aber *ohne* gebrückte `/clock`, feuert sein Timer nie – der Ball bleibt stehen (genau der Checklistenpunkt aus 24.6). Die `/clock`-Brücke ist in der Launch deshalb zwingend.

Den fliegenden Ball verarbeitet die **unveränderte Pipeline aus Schritt 2**: `ball_detector` (HSV + sparse Triangulation, 0.B) $\rightarrow$ `/ball/point`; `ball_tracker` (Kalman) $\rightarrow$ `/ball/estimate` und `/ball/prediction`; `marker_viz` zeigt alles in RViz. Verlässt der Ball das gemeinsame Sichtfeld, stoppt `/ball/point` – die Marker frieren auf der letzten Position ein (Fehlschuss-Indiz, vgl. 26.2.1.5).

Checkliste 23.5/24.6 – in Schritt 3 beachtet

`/clock` + `use_sim_time` (sonst kein Servetimer); Pose-Felder mit `float(...)` gecastet (NumPy-Skalare in ROS-Feldern); *kein* Custom-Interface – Auslösung über `std_srvs/Trigger`; der Callback heißt `on_serve`, nicht `handle` (rclpy-Kollision); HSV-Bereich, L/R-Synchronisation und Disparität unverändert aus 0.B.

Terminals und Befehle.

```
cd ~/ros2_ws
colcon build --packages-select p2_tabletennis
source install/setup.bash
ros2 launch p2_tabletennis tablettennis.launch.py
```

Listing 26.14: Terminal 1 – alles starten (Welt + Pipeline + Gegnerschlag + ...)

```
source ~/ros2_ws/install/setup.bash
ros2 topic echo /ball/point      # gemessene 3D-Position (Triangulation)
ros2 topic echo /ball/estimate   # Kalman-Schaetzung
```

Listing 26.15: Terminal 2 – Messung und Schaetzung mitlesen

```
source ~/ros2_ws/install/setup.bash
ros2 service call /opponent/serve std_srvs/srv/Trigger "{}"
```

Listing 26.16: Terminal 3 (optional) – Schlag manuell auslösen

`opponent_serve` schlägt standardmäßig alle 5 s automatisch auf (Parameter `auto_period`); `speed`, `azimuth`, `elevation` und `start` lassen sich anpassen, damit der Ball auf der Roboterhälfte aufkommt und im Sichtfeld bleibt.

```
src/p2_tabletennis/
|- p2_tabletennis/opponent_serve.py # neu (parametrischer Gegnerschlag)
|- config/set_pose_bridge.yaml      # neu (Service-Bruecke set_pose)
|- launch/tablettennis.launch.py    # angepasst: set_pose-Bruecke + opponent_serve
|- setup.py                         # angepasst: entry_point opponent_serve
`- package.xml                      # angepasst: ros_gz_interfaces, std_srvs ...
```

Listing 26.17: `ros2_ws`-Delta nach Schritt 3 (# neu / # angepasst)

26.3.4 Schritt 4 – Flugschätzung mit EKF und Luftwiderstand

Der lineare 0.B-Tracker wird durch ein **EKF** ersetzt (`flight_ekf`), das den nichtlinearen Flug aus 26.2.1.4/26.2.1.6 schätzt: Zustand $\mathbf{x} = [\mathbf{r}, \mathbf{v}]^\top$ (6D), Prozessmodell

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{g} - k \|\mathbf{v}\| \mathbf{v}, \quad k = \frac{\rho C_D A}{2m} \approx 0,131 \text{ m}^{-1}. \quad (26.8)$$

Wegen des $\|\mathbf{v}\|\mathbf{v}$ -Terms ist das nichtlinear – der EKF linearisiert pro Schritt mit der Jacobi-Matrix; für den Geschwindigkeitsblock gilt

$$\frac{\partial \dot{\mathbf{v}}}{\partial \mathbf{v}} = -k \left(\|\mathbf{v}\| I + \frac{\mathbf{v} \mathbf{v}^\top}{\|\mathbf{v}\|} \right). \quad (26.9)$$

```
def ekf_step(self, z, dt):
    F = self._F(self.x, dt)          # Jacobi der Dynamik
    self.x = self._f(self.x, dt)      # Praediktion (nichtlinear, mit Drag)
    self.P = F @ self.P @ F.T + self.Q
    S = self.P[:3, :3] + self.R       # Korrektur, H = [I 0]
    K = self.P[:, :3] @ np.linalg.inv(S)
    self.x = self.x + K @ (z - self.x[:3])
    self.P = (np.eye(6) - K @ H) @ self.P
```

Listing 26.18: `]flight_ekf` – EKF-Kern (Position gemessen, $H=[I \ 0]$)

Physik im Welt-Frame – Messungen per TF transformieren

Gravitation, Tischebene ($z = 0,76$) und Abfangebene leben im **Welt-Frame**, die Messungen (`/ball/point`) kommen aber im *optischen Kameraframe*. Der EKF transformiert sie daher per TF nach world (`tf_buffer.transform(msg, 'world')`) und rechnet/publiziert in world. In 0.B war das egal (konstante Geschwindigkeit, kein Schwerfeld) – hier ist es zwingend.

Auftreffpunkt vorhersagen. Nach jedem Update wird der Zustand *ohne* Messung vorwärts integriert (mit Tisch-Bounce) bis zur Abfangebene $y = y_R$; das ergibt Abfangpunkt und -zeit. Netztreffer und Aus werden dabei wie in 26.2.1.5 erkannt.

```
def predict_impact(self):
    x = self.x.copy(); pts = [x[:3].copy()]
    for _ in range(STEPS):
        yprev = x[1]; x = self._f(x, dt)
        if x[2] <= TZ and x[5] < 0 and on_table(x): # Tisch-Bounce
            x[2] = TZ; x[5] *= -E
        if yprev > 0 >= x[1] and x[2] < NET_TOP: # Netztreffer -> ungueltig
            return pts, None
        if x[1] <= yR: # Abfangebene erreicht
            return pts, x[:3].copy()
    return pts, None
```

Listing 26.19: `predict_impact` – Bounce, Netz, Abfangebene

Veröffentlicht werden `/ball/estimate` und `/ball/prediction` (wie 0.B, sodass `marker_viz` weiterläuft) sowie `/ball/intercept` und ein Marker (`/ball/intercept_marker`, gelber Zylinder in RViz).

Einfrieren an der Tischmitte (Umsetzung). Gemäß 26.2.1.4 korrigiert der EKF nur bis zur Mitte ($y = 0$); danach steht die Schätzung fest und es wird *nicht weitergerechnet* (Performance, ruhige Endanfahrt). Ein neuer Schlag setzt zurück:

```
def on_point(self, msg):
    z = self._to_world(msg)          # TF: optical -> world
    if self.frozen:                  # Schaetzung steht fest
        if z[1] > self.reset_y: self._reset() # nur neuer Schlag setzt zurueck
        else: return                 # sonst: kein EKF-Rechnen mehr
    ...                               # init / ekf_step / publish
    if self.x[1] <= self.freeze_y:    # Tischmitte erreicht
        self.frozen = True           # -> Abfangpunkt unveraenderbar
```

Listing 26.20: `flight_ekf` – Einfrieren und Reset

Verifiziert: das Einfrieren erfolgt bei $y \approx 0$; danach bleiben Schätzung und Abfangpunkt identisch (keine weiteren Updates), ein Ball wieder bei $y > 1,0$ löst den Reset aus.

Zwei EKF-Stolpersteine, die wir live trafen

(1) **Geschwindigkeit initialisieren:** mit Startgeschwindigkeit 0 braucht der EKF zu lange – wir setzen $\mathbf{v}$ aus der *Differenz der ersten zwei Messungen* ($\mathbf{v} = (\mathbf{z}_1 - \mathbf{z}_0)/\Delta t$). (2) **Q strukturieren:** ein großes Q auf *allen* Zuständen macht die Schätzung verrauscht; klein auf der Position (das Modell $\mathbf{r} + \mathbf{v} \Delta t$ ist exakt), moderat auf der Geschwindigkeit. Damit liegt die *frühe* Abfangpunkt-Vorhersage (nach ~ 16 Messungen) bei wenigen bis ~ 20 cm und verfeinert sich mit jeder weiteren Messung (frühe Reaktion, vgl. 26.1.8).

Checkliste 23.5/24.6 – in Schritt 4 beachtet

Mess-Frame korrekt behandelt (TF optical→world); use_sim_time + /clock; Pose-/Punktfelder mit float(...) gecastet; *kein* Custom-Interface (PointStamped, nav_msgs/Path, Marker); Methoden neutral benannt (ekf_step, nicht handle). Zur Konsistenz nutzt auch opponent_serve jetzt Luftwiderstand (gleiche k), und die Schlagparameter sind so gewählt, dass der Ball *über* das Netz kommt.

Terminals und Befehle.

```
cd ~/ros2_ws
colcon build --packages-select p2_tabletennis
source install/setup.bash
ros2 launch p2_tabletennis tablettennis.launch.py
```

Listing 26.21: Terminal 1 – alles starten (jetzt mit flight_ekf)

```
source ~/ros2_ws/install/setup.bash
ros2 topic echo /ball/estimate      # EKF-Schaetzung (im world-Frame)
ros2 topic echo /ball/intercept     # vorhergesagter Abfangpunkt
```

Listing 26.22: Terminal 2 – Schaetzung und Abfangpunkt mitlesen

In RViz zeigt der gelbe Zylinder den vorhergesagten Abfangpunkt, die blaue Linie die prädiizierte (geboundete) Bahn. Geht der Schlag ins Netz oder ins Aus, verschwindet der Abfang-Marker (ungültig).

```
src/p2_tabletennis/
|- p2_tabletennis/flight_ekf.py      # neu (EKF + Luftwiderstand + Auftreffpunkt)
|- p2_tabletennis/opponent_serve.py # angepasst: Luftwiderstand + Schlag ueber Netz
|- launch/tablettennis.launch.py    # angepasst: flight_ekf statt O.B-ball_tracker
|- config/tablettennis.rviz          # angepasst: Marker /ball/intercept_marker
`- setup.py / package.xml            # angepasst: flight_ekf, tf2_*-Abhaengigkeiten
```

Listing 26.23: ros2_ws-Delta nach Schritt 4 (# neu / # angepasst)

26.3.5 Schritt 5 – Aufschlagregeln, Fehlschuss und Begrenzungswand

Schritt 5 setzt die Tischtennis-Regeln aus 26.2.1.5 um und macht das Ergebnis *sichtbar*. Ein neuer Knoten match_referee (Schiedsrichter) erkennt die Aufpraller des Balls, prüft sie gegen die Aufschlag- bzw. Ballwechselregeln und steuert die Visualisierung. Zusätzlich erhält opponent_serve einen echten **Aufschlag**-Modus – denn die bisherigen Würfe waren Ballwechsel, keine Aufschläge.

Regelkonformer Aufschlag im Gegnerknoten. Der Roboter schlägt nie selbst auf; der Aufschlag kommt immer vom Spieler. opponent_serve kennt daher zwei Modi (Services /opponent/serve

und /opponent/rally) und meldet jeden Schlag auf /opponent/event, womit der Schiedsrichter seine Zustandsmaschine zurücksetzt. Die Aufschlagparameter sind so gewählt (offline geprüft), dass der Ball zuerst auf der eigenen Seite ($y \approx +0,5$), dann über das Netz ($z \approx 0,97 > 0,9125$) auf die Roboterseite ($y \approx -0,5$) springt.

Realistisches Spielermodell. Ein echter Spieler steht *nicht* immer mittig und schlägt *nicht* immer geradeaus. Statt eines festen Azimuts zieht jeder Schlag daher eine **seitliche Spielerposition** x_{start} und einen **Zielpunkt** x_{Ziel} auf der Robotertischhälfte; die horizontale Richtung folgt aus der Geometrie. So landen die Bälle auf dem Tisch, kommen aber aus wechselnden Winkeln – auch sehr schräg. Offline geprüft (Spieler $\pm 0,5$ m, Ziel $\pm 0,7$ m): 100 % gültige Schläge, Auftreffpunkt streut bis $\sim \pm 1,25$ m.

```
def _launch(self, mode):
    s, speed, el = self._mode_params(mode)          # serve: 3.0/5deg, rally: 4.5/6deg
    start_x = s[0] + uniform(-self.start_x_spread, self.start_x_spread) # Spielerpos.
    target_x = uniform(-self.target_x_spread, self.target_x_spread)      # Zielpunkt
    az = atan2(self.aim_y - s[1], target_x - start_x) # Richtung Start->Ziel
    self.pos = [start_x, s[1], s[2]]
    self.vel = [speed*cos(el)*cos(az), speed*cos(el)*sin(az), speed*sin(el)]
    self.event_pub.publish(String(data=mode))        # Schiedsrichter zuruecksetzen
```

Listing 26.24: opponent_serve – Spielermodell (Position + Ziel) und Schiedsrichter

Aufprall-Erkennung aus den Messungen. Der Schiedsrichter darf *nicht* die EKF-Schätzung verwenden – die ist ab der Tischmitte eingefroren (Schritt 4) und sähe den Aufprall auf der Roboterseite gar nicht. Stattdessen transformiert er die Messungen /ball/point per TF nach world und erkennt einen Aufprall als *lokales z-Minimum* nahe der Tischhöhe (Vorzeichen von v_z kippt von – nach +), innerhalb der Platte:

```
def _check_bounce(self):
    (p0,t0),(p1,t1),(p2,t2) = self.buf              # letzte drei Messungen
    vz_old = (p1[2]-p0[2])/(t1-t0); vz_new = (p2[2]-p1[2])/(t2-t1)
    if not (vz_old < -0.2 and vz_new > 0.2):        return # kein Umkehrpunkt
    if p1[2] > self.tz + 0.10:                      return # nicht auf Tischhoehe
    if abs(p1[0]) > self.hw or abs(p1[1]) > self.hl: return # ausserhalb Platte
    if t1 - self.last_bounce_t < 0.15:             return # Entprellung
    self.last_bounce_t = t1
    self._on_bounce('player' if p1[1] > 0 else 'robot')
```

Listing 26.25: match_referee – Aufprall = z-Minimum auf Tischhöhe

Regel-Zustandsmaschine. Aus der Reihenfolge der Aufprall-Seiten ergibt sich der Verdikt. Ein Netztreffer (Bahn schneidet $y = 0$ unter 0,9125 m) wird zusätzlich beim Überqueren der Netzebene erkannt.

```
def _on_bounce(self, side):
    self.bounces.append(side); n = len(self.bounces)
    if self.phase == 'SERVE':
        if n == 1 and side == 'robot': self._fault('nicht auf Aufschlagseite')
        elif n == 2 and side == 'robot': self._valid_serve() # -> Phase RALLY
        elif n == 2: self._fault('2x eigene Seite')
    else: # RALLY
        if n == 1 and side == 'robot': self._status('gueltiger Aufprall')
        elif n == 1: self._fault('falsche Seite')
        else: self._fault('Doppelaufprall')
```

Listing 26.26: match_referee – Aufschlag-/Ballwechselregeln

Begrenzungswand und Auftreffpunkt in Gazebo. Die Wand ist als *durchsichtiges, kollisionsfreies* Box-Visual am Plattenrand ($y = -1,30$, der Abfangebene) in die Welt eingebaut – der Ball fliegt hindurch. Ihre Breite (2,93 m) ist die **Reichweite des Roboters**: Tischbreite (1,525 m) plus je 0,70 m seitlich (realistisches Amateurspiel), also $x \in [-1,46, +1,46]$ m – so sind auch sehr schräge Bälle abfangbar. Sagt der EKF einen Auftreffpunkt *außerhalb* dieser Reichweite voraus, gilt der Ball als nicht erreichbar (kein Abfang-Marker, vgl. 26.2.1.5: Abfangposition außerhalb des Arbeitsraums). Da Gazebo zur Laufzeit kein bequemes Umfärben über ROS erlaubt, nutzt der Schiedsrichter einen Trick *ohne* Materialeingriff: Es gibt *zwei* Wände – eine grüne (wall_ok) und eine rote (wall_fault, sonst unter dem Boden geparkt). Bei einem Fehlschuss werden sie über dieselbe set_pose-Brücke wie der Ball (Schritt 3) getauscht; ein neuer Ballwechsel tauscht zurück. Der vom EKF geschätzte Auftreffpunkt (/ball/intercept) wird als flache Scheibe live auf die Wand gesetzt.

```
def _set_wall(self, fault):
    # Tausch statt Umfärben
    self._move('wall_ok', 0, self.wall_y, self.park_z if fault else self.wall_z)
    self._move('wall_fault', 0, self.wall_y, self.wall_z if fault else self.park_z)

def on_intercept(self, msg):
    # EKF-Schaetzung auf die Wand
    if self.fault: return
    self._move('impact_marker', msg.point.x, self.wall_y, msg.point.z,
               qx=0.7071, qw=0.7071) # roll 90: Scheibe liegt flach
```

Listing 26.27: match_referee – Wand umfärben & Auftreffpunkt setzen (set_pose)

Schiedsrichter liest Messungen, nicht die EKF-Schätzung

Die Aufprall-Erkennung läuft auf /ball/point (per TF nach world), *nicht* auf /ball/estimate. Grund: Der EKF friert seine Schätzung an der Tischmitte ein (Schritt 4) und liefert für die Roboterseite – wo der entscheidende zweite Aufprall liegt – gar keine Updates mehr. Die rohe Messung sieht die ganze Bahn.

Checkliste 23.5/24.6 – in Schritt 5 beachtet

Mess-Frame korrekt (TF optical→world); use_sim_time + /clock an allen Knoten; set_pose wird erst nach service_is_ready() aufgerufen (kein Blockieren); Pose-Felder mit float(...) gecastet; *keine* Custom-Interfaces (PointStamped, std_msgs/String, ros_gz_interfaces/SetEntityPose); Knoten und Methoden neutral benannt (match_referee, process_world_point). Die Wand ist *ohne Kollision* modelliert – sonst würde der Ball an ihr abprallen statt hindurchzufliegen.

Terminals und Befehle.

```
cd ~/ros2_ws
colcon build --packages-select p2_tabletennis
source install/setup.bash
ros2 launch p2_tabletennis tablettennis.launch.py
```

Listing 26.28: Terminal 1 – alles starten (jetzt mit Schiedsrichter + Wand)

```
source ~/ros2_ws/install/setup.bash
ros2 service call /opponent/serve std_srvs/srv/Trigger # regelkonformer Aufschlag
ros2 service call /opponent/rally std_srvs/srv/Trigger # Ballwechsel-Schlag
ros2 topic echo /match/status # bereit / gueltig / FEHLSCHUSS
```

Listing 26.29: Terminal 2 – Aufschlag/Ballwechsel auslösen, Verdikt mitlesen

In Gazebo zeigt die Scheibe auf der durchsichtigen Wand den EKF-Auftreffpunkt; der reale Ball durchbricht die Wand daneben – die Differenz ist die Modellgüte. Bei einem ungültigen Aufschlag (z. B. /opponent/serve mit einem Ballwechsel-artigen Wurf, oder ein Ball ins Netz) färbt sich die Wand rot, beim nächsten Schlag wieder grün.

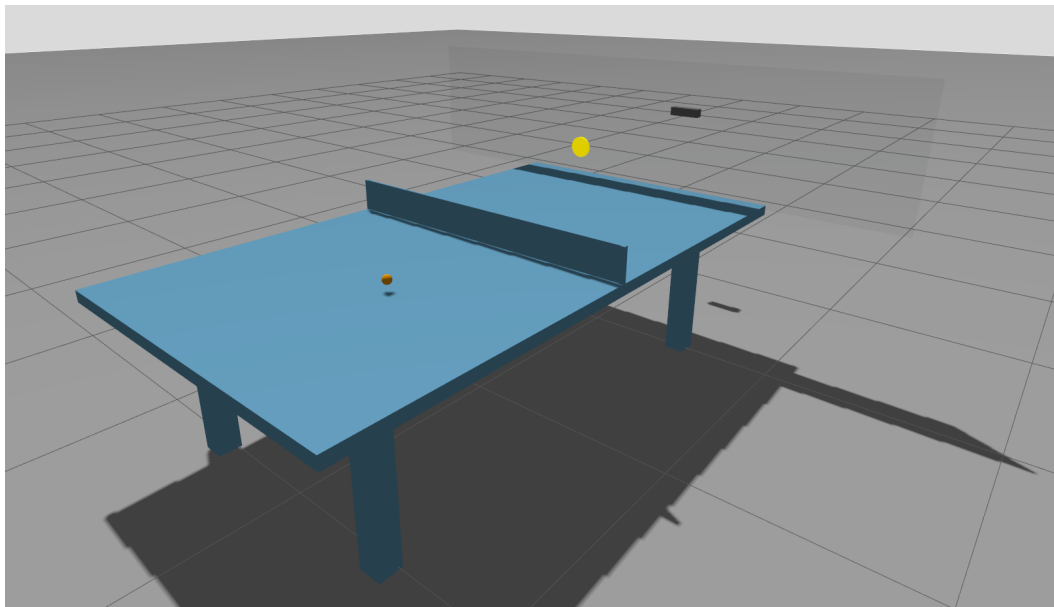


Abbildung 26.6: Schritt 5 in Gazebo: die durchsichtige Begrenzungswand am Plattenrand der Roboterseite (Breite = Reichweite, Tisch $\pm 0,70$ m) mit dem vom EKF geschätzten Auftreffpunkt als Scheibe. Der reale Ball durchbricht die Wand – die Abweichung zur Scheibe ist die Modellgüte; ein Fehlschuss färbt die Wand rot.

```
src/p2_tabletennis/
|- p2_tabletennis/match_referee.py # neu (Regeln + Fehlschuss + Wand-Visualisierung)
|- p2_tabletennis/opponent_serve.py # angepasst: Aufschlag-/Ballwechsel-Modus + /opponent/event
|- worlds/tabletennis.sdf # angepasst: wall_ok/wall_fault + impact_marker (visual-only)
)
|- launch/tabletennis.launch.py # angepasst: match_referee gestartet
~- setup.py / package.xml # angepasst: match_referee, std_msgs
```

Listing 26.30: ros2_ws-Delta nach Schritt 5 (# neu / # angepasst)

Offline verifiziert (Regel-Logik mit synthetischen Bahnen, gleiches Drag-Modell): gültiger Aufschlag (player→robot) → Wechsel auf Ballwechsel; erster Aufprall Roboterseite → Fehlschuss; zweiter

Aufprall erneut eigene Seite → Fehlschuss; Aufschlag ins Netz → Fehlschuss (Netztreffer); gültiger Ballwechsel (ein Aufprall Roboterseite) → kein Fehlschuss.

26.3.6 Genauigkeit der Flugschätzung: Messrauschen, Kamerastandort, FPS

Beim Testen fiel zweierlei auf: der eingefrorene Auftreffpunkt lag *mal sehr genau, mal deutlich daneben* (hohe Streuung), und es gab nur 5–10 Korrekturschritte bis zur Tischmitte. Die Ursachenanalyse – offline per Monte-Carlo (300 Würfe je Konfiguration, gleiches Drag-Modell, realistisches Stereo-Rauschmodell) – lieferte mehrere lehrreiche Erkenntnisse. Sie sind typisch für Stereo-Tracking und deshalb hier festgehalten.

Erstens: die Bildrate war 30 Hz, nicht mehr. Die Kameras standen auf `<update_rate>30</update_rate>`. Der Ball braucht von der Gegner-Grundlinie bis zur Tischmitte ($y = 0$) rund 0,3 s – bei 30 Hz sind das eben nur ~ 9 Frames, minus zwei für die Initialisierung. Die 5–10 Korrekturen waren also *korrekt*, kein Fehler. Die Korrekturzahl ist durch $\text{Bildrate} \times \text{Flugzeit}$ begrenzt, **nicht** durch Rechenlast oder Terminal-Logging (beides ist hier vernachlässigbar; eine Umleitung des Loggings in eine Datei ändert nichts an Genauigkeit oder Frame-Zahl).

Zweitens: das Tiefenrauschen der Triangulation dominiert. Stereo misst *Tiefe* (Entfernung entlang der Blickachse) über die Disparität – mit einem Fehler, der **quadratisch mit der Entfernung** wächst und mit der *Basisbreite* sinkt:

$$\sigma_{\text{lat}} = \frac{\sigma_{\text{px}}}{f} Z, \quad \sigma_{\text{tiefe}} = \frac{\sqrt{2} \sigma_{\text{px}}}{f B} Z^2, \quad (26.10)$$

mit Brennweite f (aus FOV und Bildbreite, hier ≈ 554 px), Pixelrauschen σ_{px} , Basisbreite B und Entfernung Z . Bei $B = 0,10$ m und $Z \approx 3,2$ m (Ball an der Gegner-Grundlinie, Kamera hinter dem Roboter) ist das verheerend:

	lateral (x, z)	Tiefe (Blickachse)
$Z = 3,2$ m, $B = 0,10$ m	2,9 mm	131 mm
$Z = 3,2$ m, $B = 0,30$ m	2,9 mm	44 mm
$Z = 1,9$ m, $B = 0,10$ m	1,7 mm	46 mm

Die rohe Tiefenmessung schwankt also um ± 13 cm – das ist direkt das beobachtete „mal genau, mal daneben“. Der EKF nahm bisher ein *skalares* $R = (5 \text{ mm})^2$ an und vertraute der verrauschten Tiefe damit ~ 25 -fach zu stark.

Drittens: mehr FPS allein ist nicht „gratis“. Naiv die Bildrate zu erhöhen, *ohne* R zu korrigieren, macht es **schlechter**: Das Filter verfolgt mit mehr Messungen nur das Tiefenrauschen genauer. Im Monte-Carlo stieg die Streuung des Auftreffpunkts von $\sigma \approx 5,7$ cm (30 Hz) auf 9,3 cm (120 Hz, skalares R). Zudem verschlechtert sich der Geschwindigkeits-Init aus zwei Frames bei hoher FPS, weil der Zeitabstand schrumpft ($\sigma_v \propto 1/\Delta t$).

Drei Korrekturen am Schätzer. Daraus folgen drei gezielte Eingriffe in `flight_ekf` (rein softwareseitig, keine Hardware-Änderung):

1. **Entfernungs- und richtungsabhängiges R :** groß entlang der Blickachse $\mathbf{d}$ (Kamera→Ball), klein quer dazu, $R = \sigma_{\text{tiefe}}^2 \mathbf{d}\mathbf{d}^\top + \sigma_{\text{lat}}^2 (I - \mathbf{d}\mathbf{d}^\top)$. Die Kamerapose kommt per TF.
2. **Prozessrauschen $Q \propto \Delta t$** (pro Sekunde statt pro Schritt) – so injiziert eine höhere FPS nicht mehr Rauschen pro Zeit; die Schätzung wird *FPS-unabhängig*.
3. **Geschwindigkeits-Init über ein festes Zeitfenster** (z. B. 0,05 s) statt aus zwei aufeinanderfolgenden Frames – entkoppelt die Init-Güte von der Bildrate.

```

def _meas_R(self, z):
    # z: Messung im Welt-Frame
    if self.cam_pos is None: return self.R # Fallback (skalar)
    r = z - self.cam_pos; Z = norm(r); d = r/Z
    sig_lat = (self.pix/self.focal)*Z # quer zur Blickachse
    sig_dep = (sqrt(2)*self.pix/(self.focal*self.baseline))*Z*Z # entlang Blickachse
    return sig_lat**2*(I - outer(d,d)) + sig_dep**2*outer(d,d)

def ekf_step(self, z, dt, R):
    F = self._F(self.x, dt); self.x = self._f(self.x, dt)
    self.P = F @ self.P @ F.T + self.Q_rate * dt # Q pro Sekunde -> *dt
    S = self.P[:3,:3] + R; K = self.P[:, :3] @ inv(S)
    self.x = self.x + K @ (z - self.x[:3]); self.P = (I - K@H) @ self.P

```

Listing 26.31: flight_ekf – entfernungsabhängiges R und $Q \propto \Delta t$

Wirkung (Monte-Carlo der echten Implementierung, hinten montiert, $B = 0,10$ m): die *Streuung* des Auftreffpunkts sinkt und ist nun über die Bildrate *stabil*.

	median	σ (Streuung)	p90
vorher (30 Hz, skalar R)	4,3 cm	5,7 cm	11,1 cm
neu, 30 Hz	5,2 cm	2,1 cm	8,0 cm
neu, 60 Hz (eingestellt)	4,7 cm	2,6 cm	8,2 cm
neu, 120 Hz	4,0 cm	2,4 cm	7,7 cm

Die Bildrate wurde auf **60 Hz** gesetzt – mehr Korrekturen (jetzt ~ 17 bis zur Mitte) bei stabiler Streuung. Wichtig: Die Sensor-Rate gilt in *Simulationszeit*; kann der Renderer 60 Hz nicht in Echtzeit liefern, sinkt der Real-Time-Factor, die Frame-Zahl pro Flug bleibt aber erhalten. Gazebo muss dafür nicht weiter angepasst werden. Die Bildrate steckt allein in den zwei `<update_rate>`-Zeilen der Welt-SDF (beide Kameras); umschalten ohne weitere Code-Änderung (der EKF ist FPS-unabhängig):

```

cd ~/ros2_ws
# beide Vorkommen ersetzen, z.B. auf 120 Hz:
sed -i 's|<update_rate>[0-9]*</update_rate>|<update_rate>120</update_rate>|g' \
    src/p2_tabletennis/worlds/tabletennis.sdf
colcon build --packages-select p2_tabletennis

```

Listing 26.32: FPS umschalten (30 / 60 / 120 Hz)

Kamerastandort: hinten, seitlich oder beim Spieler? Naheliegender wäre, die Kamera *seitlich* zu stellen (der Ball flöge dann quer zur Blickachse, die Hauptbewegung y würde lateral und damit mm-genau gemessen) oder *näher zum Spieler* (frühe Schätzungen näher $\rightarrow$ bessere Tiefe am Anfang). Beide Ideen haben wir durchgerechnet – und beide bringen **nichts** bzw. verschlechtern:

- **Seitlich** ($\sigma \approx 20$ cm statt ≈ 4 cm hinten): schlechter! Denn der gesuchte Auftreffpunkt liegt auf einer *festen* Ebene $y = y_R$; ausgegeben werden seine Koordinaten x^*, z^* . Steht die Kamera seitlich, wird ausgerechnet x^* (die Quer-zur-Platte-Position) zur *Tiefe* und damit grob verwechselt.
- **Hinter dem Spieler** ($\sigma \approx 5$ cm, praktisch wie hinten): Die frühen Schätzungen werden zwar genauer (Ball nah), aber der Zustand *an der Tischmitte* – wo eingefroren wird – ist wieder ~ 2 m entfernt; die Geometrie zur Abfangebene bleibt gleich. Dazu stünde die Kamera dort, wo der Spieler steht und aufschlägt (Verdeckung).
- **Hinter dem Roboter (aktuell)**: am besten. *Beide* Ausgabe-Koordinaten x^*, z^* sind die gut aufgelösten Lateralachsen; die schwache Tiefenachse (y) wird durch die Auswertung auf der festen Ebene $y = y_R$ weitgehend „absorbiert“.

Fazit: **Die Kamera bleibt hinter dem Roboter.** Der große Hebel war nicht der Standort und auch nicht die Basisbreite (die bei ehrlichem R kaum noch etwas bringt: $B=0,10$ vs. $0,30$ m liegt im Rauschen), sondern das *Messrauschmodell* und $Q \propto \Delta t$. Eine zweite Stereo-Kamera zum Tiefenabgleich wäre die saubere Lösung gegen das Tiefenrauschen, scheidet hier aber aus Budgetgründen aus.

Sieht die Kamera die breite Reichweite noch? Mit der verbreiterten Reichweite (x bis $\pm 1,46$ m) und dem seitlich spielenden Gegner liegt die Sorge nahe, die hintere Kamera müsse *weiter nach hinten*, um die Ränder noch aufzulösen. Die Simulation (4000 Schläge, Spieler $\pm 0,5$ m, Ziel $\pm 0,7$ m) zeigt: **nicht nötig**. Die EKF-Phase reicht nur bis zur Tischmitte ($y \geq 0$); dort ist der Ball noch bei $|x| \lesssim 0,8$ m – die weite seitliche Ablage entsteht erst *danach*. Bei der aktuellen Position ($y = -1,9$) sind **100 %** der gültigen Schläge während der EKF-Phase im Sichtfeld; eine Position weiter hinten ($y = -2,3/-2,7$) deckt zwar ebenfalls 100 % ab, verschlechtert aber die Tiefe (größeres Z). *Aber:* Der *vorhergesagte* Auftreffpunkt ist bei sehr schrägen Randbällen ungenauer – mittig $\sigma \approx 1,7$ cm (p90 7 cm), am Rand ($x^* \approx 1,1$ m) $\sigma \approx 5-7$ cm (p90 bis 22 cm). Ursache ist *nicht* die Kamera, sondern die *open-loop-Extrapolation* der weiten Querbewegung von der Tischmitte bis zum Rand (Hebel auf v_x plus Bounce). Eine Kamera weiter hinten würde das nicht beheben (sie sieht den weiten Durchstoßpunkt aus der Nähe ohnehin nicht und verschlechtert die Tiefe). Der Restfehler liegt im Rahmen der Schläger-/Reichweitentoleranz; eine spätere Verbesserung wäre, den Einfrierpunkt für extreme Querbälle etwas nach hinten zu legen.

Ausblick: variationsärmere Schätzung. Mehrere Ansätze würden die Streuung über das hier Erreichte hinaus weiter senken:

- **Glätter statt reiner Vorwärts-EKF:** Ein Fixed-Lag- bzw. RTS-Glätter (oder ein Batch-/Least-Squares-Fit der ganzen Bahn) nutzt *alle* Messungen bis zur Tischmitte auch *rückwärts* – die früh noch verrauschte Geschwindigkeit wird durch die späteren Messungen nachträglich korrigiert.
- **Bahnmodell als Constraint:** Der Flug ist eine Parabel mit Luftwiderstand mit nur wenigen Parametern ($\mathbf{r}_0, \mathbf{v}_0$). Ein überbestimmter parametrischer Fit ist robuster als die rekursive Punkt-für-Punkt-Schätzung.
- **Restitution/Drag mitschätzen:** e und k als Zustände (oder online kalibriert) verringern den Modellfehler, der sich am Bounce stark auf den Auftreffpunkt überträgt.
- **UKF/IEKF statt EKF:** weniger Linearisierungsfehler im nichtlinearen Drag-/Bounce-Modell.
- **Ausreißer-Abweisung:** ein Mahalanobis-Gate gegen Fehltriangulationen (verdeckter oder nur teilerkannter Ball) – einzelne Ausreißer treiben sonst die Varianz.
- **Bessere Messung:** Subpixel-Detektion (kleineres σ_{px}) oder höhere Auflösung (größeres f) senken das Rauschen linear; eine zweite Kamera-Perspektive zum Tiefenabgleich wäre die physikalisch sauberste Lösung (hier aus Budgetgründen ausgeschlossen).
- **Adaptiver Einfrierpunkt:** für extreme Querbälle die Mitte-Grenze etwas nach hinten legen – weniger Open-Loop-Extrapolation, kleinerer Randfehler.

Austauschbare Schätzermodelle (Architektur). Mehrere dieser Ansätze sind eigentlich *alternative Schätzer* auf denselben Eingangsdaten. Das lässt sich sauber als **austauschbare Architektur** bauen, ohne die übrige Pipeline anzufassen: Der Tracker-Knoten konsumiert `/ball/point` und veröffentlicht `/ball/estimate`, `/ball/prediction`, `/ball/intercept` usw. – jeder Schätzer mit *derselben* Schnittstelle ist ein Drop-in. Hinter einer gemeinsamen Strategie-Schnittstelle (`reset()`, `add(z,t)`, Zustand $\mathbf{x}$) ließen sich verschiedene Modelle kapseln; ein Parameter `model` wählt das aktive Modell – per **Launch-Argument** (`estimator:=ekf`) oder zur Laufzeit (`ros2 param set /flight_ekf model batch`). Die gemeinsame Logik (TF, Einfrieren an der Mitte, Auftreffpunkt/-zeit, Reichweite, Klassifikation) bliebe ungeteilt. So sind Modelle *auf identischen Eingaben* direkt vergleichbar, z. B.:

- **ekf** – der hier umgesetzte rekursive EKF (Schritt 4).
- **batch** – ein **parametrischer Fit der ganzen Bahn**: der Flug ist eine Parabel mit Luftwiderstand mit wenigen Parametern ($\mathbf{r}_0, \mathbf{v}_0$). Ein überbestimmter, mit dem Stereo-Rauschmodell gewichteter Least-Squares-/ Gauss-Newton-Fit ist robuster gegen einzelne Ausreißer als die rekursive Punkt-für-Punkt-Schätzung.

So ließe sich etwa der EKF gegen den Batch-Fit auf demselben Wurf benchmarken (Streuung, Rechenlast), ohne den restlichen Stack zu ändern – eine naheliegende nächste Ausbaustufe.

Lehren – Stereo-Tracking

(1) Tiefe ist der schwache Freiheitsgrad: Fehler $\propto Z^2/(fB)$. (2) Das Messrauschen *ehrlich* (anisotrop, entfernungsabhängig) modellieren, sonst überschätzt der EKF die Tiefe. (3) Q pro Sekunde ansetzen ($\cdot \Delta t$), damit FPS-Änderungen das Filter nicht verstimmen. (4) Mehr FPS hilft nur *mit* korrektem R . (5) Den Standort an der Frage festmachen, *welche* Koordinaten gebraucht werden – hier liegen sie quer zur Tiefe, also gehört die Kamera hinter den Roboter.

```
src/p2_tabletennis/
|- worlds/tabletennis.sdf          # angepasst: Kameras 60 Hz; Wand auf Reichweite 2.93 m
|- p2_tabletennis/flight_ekf.py    # angepasst: R entfernungsabh., Q*dt, Fenster-Init,
    half_reach
`- p2_tabletennis/opponent_serve.py # angepasst: Spielermodell (seitl. Position + Zielpunkt)
```

Listing 26.33: ros2_ws-Delta dieser Verfeinerung (# angepasst)

26.3.7 Schritt 6 – Auftreffzeitpunkt und Ball-Klassifikation

Für den späteren Schlag (Schritt 7) genügt der Auftreffpunkt nicht – der Schwung muss auf den Auftreffzeitpunkt getaktet werden. Die Vorwärtsintegration des EKF (`predict_impact`, Schritt 4) liefert die Restflugzeit t^* ohne Zusatzaufwand: es wird einfach die integrierte Zeit bis zum Erreichen der Abfangebene mitgezählt.

```
def predict_impact(self):
    x = self.x.copy(); t_acc = 0.0
    for _ in range(STEPS):
        x = self._f(x, dt); t_acc += dt          # Zeit mitzaehlen
        ... # Tisch-Bounce / Netz / Reichweite wie Schritt 4/5
        if x[1] <= yR:                          # Abfangebene erreicht
            return pts, x[:3].copy(), t_acc      # Punkt + Restflugzeit t*
    return pts, None, None
```

Listing 26.34: `flight_ekf` – Restflugzeit t^* aus der Vorhersage

Veröffentlicht wird `/ball/time_to_impact` (`std_msgs/Float32`).

Klassifikation schneller Bälle. Die Reaktionszeit spreizt stark: ein langsamer, hoher Bogen lässt viel Zeit, ein schneller, flacher Ball wenig. Anhand von t^* wird der Ball klassifiziert (Schwelle `fast_time`, hier 0,40 s); liegt t^* darunter, ist das Reaktionsfenster knapp.

```
fast = t_impact < self.fast_time                # knappes Reaktionsfenster?
self.tti_pub.publish(Float32(data=t_impact))
self.class_pub.publish(String(data='schnell' if fast else 'normal'))
```

Listing 26.35: `flight_ekf` – Ball-Klassifikation (schnell/normal)

Sinn der Klassifikation: Ist das Reaktionsfenster so knapp, dass die ersten Schätzungen für ein sicheres späteres Verfeinern nicht reichen, soll der Roboter *früh committen* – direkt eine Abfangbewegung anfahren bzw. schlagen, statt auf weitere EKF-Verfeinerung zu warten (die das knappe Fenster aufzehren würde). Bei *normalen* Bällen bleibt Zeit, den Abfangpunkt bis zur Tischmitte zu verfeinern (Schritt 4). Die Schwelle ist ein Auslegungsparameter; der eigentliche „direkt schlagen“-Entscheid gehört in die Roboterlogik (Schritt 7).

Terminals und Befehle.

```
source ~/ros2_ws/install/setup.bash
ros2 topic echo /ball/time_to_impact    # Restflugzeit t* [s]
ros2 topic echo /ball/ball_class        # schnell / normal
```

Listing 26.36: Terminal 2 – Restflugzeit und Ballklasse mitlesen

Offline verifiziert: flacher Ball ($v_y = -4$) $\rightarrow t^* \approx 0,36$ s („schnell“), hoher Bogen ($v_y = -2$, $v_z = 2,5$) $\rightarrow t^* \approx 0,73$ s („normal“).

```
src/p2_tabletennis/
`- p2_tabletennis/flight_ekf.py    # angepasst: predict_impact liefert t*;
                                   # /ball/time_to_impact + /ball/ball_class
```

Listing 26.37: ros2_ws-Delta nach Schritt 6 (# angepasst)

26.3.8 Schritt 7 – Roboteranbindung

[Dokumentation folgt.] Umsetzung der Regelstrategie aus 26.2.2: entkoppelte PID-Achsregelung des Portals, Schlägerorientierung (Vor-/Rückhand), inverse Kinematik und auf t^* getaktete Schlagbewegung über den ros2_control-Stack.

Kapitel 27

Projekt 3 – Mobiler Roboter: Odometrie, Navigation, Bewegungserkennung

Ziel: Mit Stereokamera, Odometrie und Pfadalgorithmen navigieren und Bewegung erkennen.

27.1 ROS 2-Bausteine

Typ	Konkret
Nodes	stereo_camera · visual_odometry oder Radodometrie · robot_localization (EKF: Odom+IMU) · slam_toolbox (Karte) · nav2 (Planer+Controller) · motion_detector (optischer Fluss/Hintergrundsubtraktion)
Topics	/odom · /tf · /scan bzw. /points2 · /map · /cmd_vel
Actions	nav2_msgs/NavigateToPose
Parameter	Kostenkarte, Controller-Frequenz, EKF-Konfiguration, Roboterfußabdruck

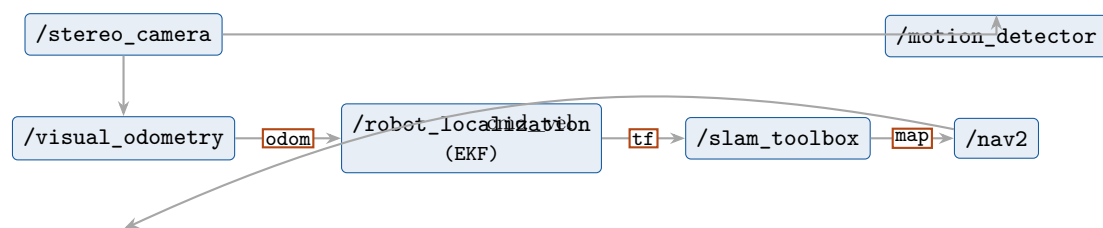


Abbildung 27.1: Projekt 3: Odometrie → EKF-Fusion → SLAM → Nav2; Bewegungserkennung als paralleler Zweig.

Kapitel 28

Projekt 4 – CAD-basierte Posenerkennung und Greifen

Ziel: Bauteile per Stereokamera über sich matchende CAD-Modelle erkennen, deren 6D-Lage bestimmen und das Greifen daran anpassen.

28.1 ROS 2-Bausteine

Typ	Konkret
Nodes	stereo_camera · part_detector (ORB/SIFT-Matching gegen gerenderte CAD-Ansichten <i>oder</i> Punktwolke+ICP) · pose_estimator (solvePnP/ICP → 6D-Pose) · moveit2 (Greifplanung) · robot_controller
Topics/Msgs	vision_msgs/Detection3DArray · Pose über /tf
Actions	MoveIt 2 MoveGroup (Planen + Ausführen)
Parameter	CAD-Modellpfade, Matching-Schwellen, ICP-Iterationen, Greifvorgaben

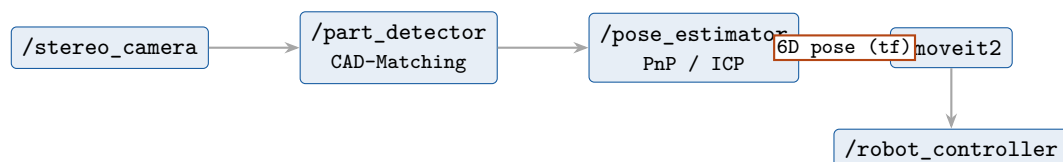


Abbildung 28.1: Projekt 4: CAD-Matching → 6D-Posenschätzung → MoveIt 2-Greifplanung, angepasst an die erkannte Werkstücklage.

28.2 Zwei Wege zur 6D-Pose

Für das CAD-Matching gibt es einen klassischen und einen lernbasierten Pfad – beide enden im selben `pose_estimator` und liefern $(\mathbf{R}, \mathbf{t})$ über /tf an MoveIt 2:

1. **Klassisch (kein Training):** ORB/SIFT-Matching gegen gerenderte CAD-Ansichten → `solvePnPRansac`, *oder* Stereo-Punktwolke + ICP gegen die CAD-Punktwolke. Schnell aufzusetzen, aber fragil bei texturlosen/glänzenden/symmetrischen Teilen.
2. **Gelernte Surface Embeddings (SurfEmb-Linie):** das im Theoriekapitel beschriebene Zwei-Turm-Netz. Robuster bei genau diesen Industrieteilen, dafür Trainingsaufwand. **Empfehlung:** klassisch starten, lernbasiert nachrüsten, wenn die Teile die klassischen Deskriptoren überfordern.

Die theoretischen Grundlagen beider Wege (Rotationsrepräsentationen, kontrastives Lernen, Two-Tower-Modell, PnP+RANSAC, grobe/feine Registrierung, PointNet++) stehen im Theoriekapitel zur 6-DoF-Posenschätzung.

28.3 Inferenz-Pipeline (lernbasiert)

1. **Detektion/Segmentierung** des Objekts im Stereobild → Bildausschnitt (RGB oder RGB-D) plus bekanntes CAD-Modell als Eingang.
2. **Embedding:** Query-Tower bettet die Pixel ein, Key-Tower die CAD-Oberflächenpunkte – in denselben Merkmalsraum.
3. **Matching** im Embedding-Raum (nächster Nachbar) → dichte 2D-3D-Korrespondenzen.
4. **Pose-Hypothesen:** PnP+RANSAC sampeln und bewerten (Scoring), beste Hypothese wählen.
5. **Verfeinerung** per ICP → finale $(\mathbf{R}, \mathbf{t})$, Modell→Roboterframe.

Punktwolken-Zusatzschritte (3D-3D-Pfad): Hintergrund entfernen, Voxel-Downsampling, statistischer Ausreißerfilter – vor der Registrierung.

Labels sind quasi gratis

Das CAD-Modell wird in bekannten Posen gerendert → Ground-Truth-Pose *und* die Pixel-Punkt-Korrespondenzen liegen automatisch vor. Kein manuelles Annotieren, keine Symmetrie-Labels. Das synthetische Training wird per **Domain Randomization** und PBR-Rendering (Isaac Sim) gegen den Sim-to-Real-Gap abgehärtet, optional mit kleinem realem Feintuning-Satz.

28.4 Stereo als metrische Skala

Die Stereokamera ist hier nicht nur Bildquelle: Sie löst die **Skalenmehrdeutigkeit** der Mono-Ansicht. Tiefe geht entweder als RGB-D-Zusatzkanal ins Netz oder wird zur Punktwolke rückprojiziert. Erst dadurch steht $(\mathbf{R}, \mathbf{t})$ in echten Millimetern – die Voraussetzung dafür, dass MoveIt 2 überhaupt greifen kann. **Vorsicht:** glänzendes Metall liefert ungültige Tiefe; saubere Kalibrierung, Rektifizierung und Tiefen-Entrauschung sind Pflicht.

28.5 Bewegte Teile: Pose-Tracking mit Kalman

Liegt das Teil nicht still (Förderband, armmontierte Kamera), ist die bildweise Netz-Pose veräuscht und zitterig. Ein **Kalman-Filter** (vgl. Projekt 1/3 und das Kalman-Kapitel) macht aus Einzelschuss ein *Tracking*: Zustand = Pose (ggf. + Geschwindigkeit), Messung = bildweise Netz-Pose; Prädiktion → Korrektur glättet Jitter, verwirft Ausreißer-Frames und überbrückt kurze Verdeckungen. Da Rotationen nichtlinear sind: EKF/UKF oder Filtern auf einer geeigneten Rotationsdarstellung. *Dieselbe Kalman-Kompetenz wie beim Tischtennisball – nur auf die 6D-Pose statt die Ballposition angewandt.*

28.6 Hyperparameter und Evaluation

Stellgröße	Wirkung
Embedding-Dimension	Ausdrucksstärke des Merkmalsraums
Kontrastive Temperatur τ	Schärfe der Zuordnung im Verlust
#Negative / Backbone-Tiefe	Trennschärfe bzw. Kapazität
LR / Batch / Optimierer	Trainingsdynamik

Augmentierungsstärke		Robustheit gegen Sim-to-Real-Gap
RANSAC-Iterationen	+	Ausreißerrobustheit der Posenlösung
Inlier-Schwelle		
#Pose-Hypothesen	/	Genauigkeit vs. Laufzeit
Crop-Auflösung		

Metriken: ADD / ADD-S (ADD-S für symmetrische Teile), geodätischer Rotations- und euklidischer Translationsfehler, RMSE; im Forschungskontext die BOP-Metriken (VSD, MSSD, MSPD → Average Recall) auf Datensätzen wie T-LESS/ITODD. **Industrielle KPIs:** Taktzeit/Latenz, Pick-Erfolgsrate, Falsch-Positiv-Rate – das sind die Zahlen, die im Maschinenbeladungs-Einsatz zählen.

Teil VIII

Dokumentationsmethodik

Kapitel 29

Vorgehensweise festhalten

Eine konsistente Dokumentation der Projekte schlägt folgende Struktur und Werkzeuge vor:

Versionskontrolle

Alles in **Git**. Ein Monorepo oder ein Repo je Projekt mit klarer **README**.

Verzeichnisstruktur

`/docs` (Theorie, Entscheidungen), `/design` (CAD, URDF, Frames), `/hardware` (Stückliste/BOM, Verdrahtung, Pinbelegung), `/software` (Pakete, Launch-Files, Parameter-YAMLs), `/calibration` (Kamera-.yaml), `/logs` (Tests, `ros2 bag`-Aufzeichnungen).

Entscheidungsprotokoll (ADR)

Kurze Notizen “Warum TMC2209 statt A4988?”, “Warum Jazzy statt Humble?” – spart in Monaten viel Grübeln.

Code-Doku

Doxygen (C++) bzw. Docstrings + **Sphinx** (Python); `rostdoc2` für ROS 2-Pakete.

Reproduzierbarkeit

Launch-Files und Parameter-YAMLs sind Teil der Doku – sie machen ein Experiment wiederholbar. `rqt_graph`-Screenshots je Projekt beilegen.

Nachweise

`ros2 bag` zeichnet reale Läufe auf; damit kannst du Vision/Kalman *offline* debuggen und Sim vs. Real vergleichen.

Empfohlene Reihenfolge der Umsetzung

1. Setup (Teil III) → 2. URDF aus CAD + in Gazebo bewegen (Teil IV) → 3. `ros2_control` in Sim, Trajektorie abfahren (Projekt 1 simuliert) → 4. Stereokamera real anbinden + kalibrieren (Teil VI) → 5. Kalman/Vision auf simulierten *und* realen Daten → 6. Hardware-Interface tauschen, ESP32 + TMC2209 real → 7. projektspezifische Logik (Tischtennis, Navigation, Greifen). Jeder Schritt ist für sich testbar – das ist der Sinn der `ros2_control`-Trennung.

Literaturverzeichnis

- [1] Ali Barzegar, Oualid Doukhi und Deok-Jin Lee. “Design and Implementation of an Autonomous Electric Vehicle for Self-Driving Control under GNSS-Denied Environments”. In: *Applied Sciences* 11.8 (2021), S. 3688. DOI: [10.3390/app11083688](https://doi.org/10.3390/app11083688).
- [2] Rushi Deshmukh. *Visual Odometry: A Practical Guide*. Medium. 2023. URL: <https://medium.com/@rushideshmukh23/visual-odometry-a-practical-guide-71ddff1687cd>.
- [3] Jaime León. *Visual Odometry*. Online. 2021. URL: <https://jasleon.github.io/Visual-Odometry/>.
- [4] Simone Godio u. a. “Resolution and Frequency Effects on UAVs Semi-Direct Visual-Inertial Odometry (SVO) for Warehouse Logistics”. In: *Sensors* 22.24 (2022), S. 9911. DOI: [10.3390/s22249911](https://doi.org/10.3390/s22249911).
- [5] OpenCV. *Camera Calibration (OpenCV 4.x Python Tutorials)*. 2024. URL: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html.
- [6] Oliver Kahmen, Robin Rofallski und Thomas Luhmann. “Impact of Stereo Camera Calibration to Object Accuracy in Multimedia Photogrammetry”. In: *Remote Sensing* 12.12 (2020), S. 2057. DOI: [10.3390/rs12122057](https://doi.org/10.3390/rs12122057).
- [7] Shaharyar Ahmed Khan Tareen und Zahra Saleem. “A Comparative Analysis of SIFT, SURF, KAZE, AKAZE, ORB, and BRISK”. In: *International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*. IEEE, 2018.
- [8] Rasmus Laurvig Haugaard und Anders Glent Buch. “SurfEmb: Dense and Continuous Correspondence Distributions for Object Pose Estimation with Learnt Surface Embeddings”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2022.
- [9] Yi Zhou u. a. “On the Continuity of Rotation Representations in Neural Networks”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
- [10] Radu Bogdan Rusu, Nico Blodow und Michael Beetz. “Fast Point Feature Histograms (FPFH) for 3D Registration”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2009.
- [11] Rui Qiao u. a. “An Accurate, Efficient, and Stable Perspective-n-Point Algorithm in 3D Space”. In: *Applied Sciences* 13.2 (2023), S. 1111. DOI: [10.3390/app13021111](https://doi.org/10.3390/app13021111).
- [12] Rahul Raguram, Jan-Michael Frahm und Marc Pollefeys. “A Comparative Analysis of RANSAC Techniques Leading to Adaptive Real-Time Random Sample Consensus”. In: *European Conference on Computer Vision (ECCV), LNCS 5303*. Springer, 2008, S. 500–513.
- [13] Paul J. Besl und Neil D. McKay. “A Method for Registration of 3-D Shapes”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), S. 239–256.
- [14] Gerard Pons-Moll. *ICP: Iterative Closest Points (Lecture 4.1, Virtual Humans, Winter 23/24)*. University of Tübingen / MPI-Informatics, Vorlesungsfolien. 2024.
- [15] Jiaolong Yang u. a. “Go-ICP: A Globally Optimal Solution to 3D ICP Point-Set Registration”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 38.11 (2016), S. 2241–2254.
- [16] Charles R. Qi u. a. “PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017.

- [17] Charles R. Qi u. a. “PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017.
- [18] Felix Maral. *Understanding PointNet and PointNet++ for Semantic Segmentation of 3D Point Clouds*. Medium. 2023. URL: <https://medium.com/@felixmaral131/understanding-pointnet-and-pointnet-for-semantic-segmentation-of-3d-point-clouds-896f4efde55a>.
- [19] Tomáš Hodaň u. a. “BOP: Benchmark for 6D Object Pose Estimation”. In: *European Conference on Computer Vision (ECCV)*. 2018.
- [20] Maximilian Denninger u. a. “BlenderProc”. In: *arXiv preprint arXiv:1911.01911* (2019).
- [21] NVIDIA. *Omniverse Replicator (Synthetic Data Generation)*. 2024. URL: https://docs.omniverse.nvidia.com/extensions/latest/ext_replicator.html.
- [22] NVIDIA. *NVIDIA Isaac Sim Documentation*. 2024. URL: <https://docs.isaacsim.omniverse.nvidia.com/>.
- [23] Unity Technologies. *Unity Perception Package*. 2023. URL: <https://github.com/UnityTechnologies/com.unity.perception>.
- [24] Emanuel Todorov, Tom Erez und Yuval Tassa. “MuJoCo: A physics engine for model-based control”. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2012, S. 5026–5033.
- [25] Google DeepMind. *MuJoCo Documentation*. 2024. URL: <https://mujoco.readthedocs.io/>.
- [26] Mayank Mittal u. a. “Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments”. In: *IEEE Robotics and Automation Letters (RA-L)*. 2023.